Evaluation of 22nm Memristor-Based Main Memory (MBMM) in Commodity DIMM Architectures

The extended book: the complete record and knowledge base of the project. The book itself, the summary of the work with its references, is the separate document Project_Book.

Abstract

The memory supply-demand gap of late 2025, driven by the reallocation of DRAM wafers to High-Bandwidth Memory (HBM) for AI **[1], [2]**, necessitates alternative technologies for commodity main memory. This research evaluates 22nm Memristor-based Non-Volatile Memory (NVM) as a DRAM replacement in standard DIMMs. Utilizing a cross-layer pipeline bridging NVSim **[3]** and NVMain 2.0 **[4]**, I characterize 1T1R (transistor-gated) and 1S1R (selector-gated) architectures across six workloads - drawn from five benchmarks, with AlexNet split into read-dominant and write-dominant phases - spanning compute-bound, memory-streaming, and AI-inference behavior. Both cell types are simulated at one matched array organization (2048 x 2048 subarrays, sense-amp mux 64), so every cross-technology difference reported here is a device difference and not an organization difference.

The key quantitative results are: (1) In wall-clock latency at NVSim-projected device timings, the modeled ReRAM DIMM is *faster* than the DDR5-4800 baseline on the CPU traces: under compute-bound GCC the full-DIMM averages are 41.4 ns for 1T1R SLC and 36.8 ns for 1S1R SLC against 83.1 ns for DDR5. Only 11.25 ns of that ReRAM average is NVSim device time, charged once as tRCD plus tCAS; the remaining three quarters is NVMain's default protocol cycle counts at the assumed 800 MHz interface, an assumption carrying no ReRAM citation (Section 2.3), so the direction of the comparison is robust and its magnitude is model-dependent. That result is bounded by a published-silicon counterweight run on the same traces: with the fabricated-chip timings of Micron/Sony's 16 Gb 1T1R **[49]** and SanDisk/Toshiba's 32 Gb 1S1R **[48]** the same DIMM averages 11.9 us and 379 us under GCC and completes only 18% and 1% of the LBM window, so the latency verdict belongs to the projected device, not to any shipped part. On the write-dominated AlexNet OFMAP trace ReRAM is slower than DDR5 (353.1 and 391.5 ns against 234.3 ns), and under high-parallelism AI inference all technologies are queue-bound rather than device-bound. Under sustained LBM streaming every ReRAM track now completes the whole 250 ms window, as DDR5 does - 5,564,704 of 5,564,704 requests - against 50% for the legacy PCM baseline. (2) At the matched organization NVSim assigns the two cell types *identical* peripheral leakage - 108.384 mW per 1 Gb chip - so the transistor-versus-selector leakage gap reported in earlier versions of this book does not exist as a device effect; it was an artifact of two different array organizations (Section 3.1.2 and Appendix A). The full ReRAM DIMM therefore draws 6.94 W, essentially all static, for 8 GiB, against 0.797 W for a 16 GiB DDR5 module under GCC: 17.4x more power per gigabyte, and 12.7x to 17.4x across the suite for the SLC tracks (6.4x to 8.7x for MLC, which puts twice the capacity behind the same chips). Both module figures count the memory devices only: register clock driver, power-management IC, PHY and termination, and any ReRAM-side controller, are outside the model on both sides, and a fixed per-module overhead would compress this per-gigabyte ratio - while tightening, not relaxing, the gating comparison of Section 3.3. That ratio is also an *upper bound*, measured against a DDR5 baseline that realizes its JEDEC power-down credit while ReRAM's power-down energy is an explicit no-savings placeholder, so the model cannot price a ReRAM module that gates its periphery. Break-even needs 94.3% of the static component gated, which lies *inside*

the 94-98% idleness the 2-6% memory-bandwidth utilization reported for web-serving deployments [21] corresponds to: at the idle end of that band a gated 22nm module would draw 0.14 W against DDR5's 0.40 W for the same 8 GiB, and at the busy end 0.42 W against 0.40 W. The arithmetic therefore brackets parity for that deployment class, and what this book cannot do is price it, because no measured ReRAM power-down energy exists. The selector's remaining advantage over the transistor is die area (3.4x smaller per chip), not leakage. What the selector does buy, quantified here for the first time by an analytic layer on top of NVSim (Section 3.1.2), is a sneak-current budget that adds nothing at all to standby power and 7.56 mW per chip during an access at an ovonic-threshold-switch bound, or 0.076 mW at a best-published FAST-selector bound **[9]**, evaluated with the crossbar read model of **[51]** - but the present-day OTS bound does not keep a 2048-cell tile valid (its ceiling is 1666 cells a side), so 1S1R at this organization is conditional on selector quality. These results rest on a fidelity audit of the simulation flow (Section 3.1.6) that found fourteen silent failure modes in the standard NVSim-to-NVMain toolchain - a mixed-clock-domain efficiency metric, two silently ignored parameter families (leakage and access energy), a technology-blind static-power default, single-rank power reported as system power, a disabled power-down model, weak trace provenance, DDR5 refresh timing and supply parameters inherited unrescaled from a DDR3-era template, a PCM baseline running at twice its cited clock basis, heterogeneous host-CPU frequencies that replayed each technology against a different offered load and admitted trace population, an unsourced DDR5 CAS/RCD/RP timing placeholder, and unsourced ReRAM MLC read/write latency and energy penalty multipliers attributed to a citation that does not support them (later found to include a further, independent data error in the MLC read-latency figure, also corrected) - of which this work repaired thirteen, validating every repair against device-level anchors (0.0% error) or exact predicted arithmetic, and re-running the affected simulations (with idle-gating restored, DDR5 realizes real power-down savings, ReRAM's power-down energy is a disclosed no-savings placeholder, and PCM shows no power-down activity). (3) Write endurance is the binding constraint, and wear leveling decides it. Projecting the write distribution measured per line location in the simulation onto a 64 GiB module at the best-sourced 10^6 cycle rating **[47]**, ideal uniform leveling gives 38.6 years under GCC, 3.6 under LBM and 4.3 under STREAM, while no leveling at all gives 7.7, 23.2 and 69.5 *hours*: the whole lifetime claim lives in the leveling mechanism, not in the cell. A faithful deterministic Start-Gap remapper **[52]** over one whole-module region recovers none of that gap for these sparse footprints, because the hottest line fails before the gap has rotated once; randomizing the region assignment recovers part of it (0.20 years under LBM, 0.96 under STREAM). Stated in the form a system designer needs, the per-cell endurance required for a ten-year life at 64 GiB is 2.6×10^5 (GCC), 2.8×10^6 (LBM) and 2.3×10^6 (STREAM) under ideal leveling, and 6.3, 6.5 and 4.3×10^6 under randomized Start-Gap. Every endurance figure in this book is a *projection* from a measured write distribution, never a measured lifetime. (4) On the *pre-revision* dataset, read latency is invariant to a ±20% ReadVoltage perturbation with Power-Delay Product changing less than 4%. That sweep has not been re-run at the matched organization, so its absolute values are not comparable with the tables

above; its qualitative conclusion survives a fortiori, because the static share of module power is now 99.96% (Section 3.1.5).

The resulting picture is narrower and better grounded than the one this book carried in its 3 September version. Selector-gated 22nm SLC ReRAM is a density play (1.24x DDR5's die-level density as SLC, 2.47x as MLC) with competitive projected latency and a large, honest power deficit per gigabyte; the transistor-gated variant matches it on leakage and loses on area. Neither is a drop-in DDR5 replacement at 22nm. What changed since the 3 September version, claim by claim, is tabulated in the change appendix.

Scaling that module does not change the verdict, and the chip-count matrix of Section 3.2 says why. Across four architectures and six workloads, module leakage and capacity scale exactly linearly with chip count (1, 8, 16 and 64 times 108.384 mW of static power, to six digits) while latency barely scales at all: the three CPU traces are offered-rate limited and improve by only 6.4% to 10.4% across the three physically realizable points (eight, sixteen and sixty-four chips), with a total spread of at most 11.6% and no monotonic trend. The two AI read bursts gain 1.68x, and a control run at one channel shows that gain is entirely the second channel: with a single channel the 8-chip, 16-chip and full-DIMM points are the same latency to the digit, and the whole improvement is admission queueing at a second controller with device service time flat at 29.0 ns. Rank depth bought at most 7.5% on any of the six traces, only on the three offered-rate-limited CPU traces and only as a fall in the device component; on the three AI traces it bought exactly nothing, because their footprints never leave one rank.

Table of Contents

Abstract 2
Table of Contents 4
List of Figures 5
List of Tables 6
1. Introduction & Background 8
    1.1. Project Context 8
    1.2. ReRAM Fundamentals 8
    1.3. Related Work 9
2. Infrastructure & Methodology 11
    2.1. Decentralized Structure & Simulation Stack 11
    2.2. Validation & Protocol 12
    2.3. Hardware Baselines & Memory Models 13
    2.4. Workload Traces & Benchmarks 19
        2.4.1. Diagnostic Trace Dictionary 19
3. Results: Multi-Rank Architecture & Scaling 21
    3.1. Granular Workload Diagnostics (Latency & Power Bar Charts) 21
        3.1.1 Latency Analysis: A Fast Projected Array and its Silicon Counterweight 21
            The write pulse on the request path: a bounding run 26

3.1.2 Power Analysis: Static Leakage Dominance 35
Bit errors and error correction 39
3.1.3 Power-Delay Product (PDP): The Architectural Sweet Spot 45
3.1.4 Endurance Viability Analysis 52
3.1.5 Design Robustness: ReadVoltage Sensitivity 57
3.1.6 Simulation-Fidelity Audit: Where the Toolchain Could Not Be Trusted - and How It Was Repaired 59
3.2. Architectural Scaling: Chip Count, Channels and Ranks 64
3.3. Global Viability (Hero Graphs) 73
3.4. Near and Far Memory 82
3.4.1. The measured far-memory window 82
3.4.2. How Linux would see such a device 83
3.4.3. What CXL is and is not worth in bandwidth 84
3.4.4. What the one shipped precedent actually did 84
3.4.5. Device reference row 84
4. Conclusion and Future Work 86
4.1. Future Work: Simulator and Infrastructure Enhancements 89
4.2. Future Work: Architectural Extensions and System Scaling 91
4.3 Data Availability and Reproducibility 93
References 94
Appendix A: Simulation Parameters and Literature Grounding 103
Appendix B: Execution Pipeline and Command Interface 111
Appendix C: Reproducibility Statement 112
Appendix D: What Changed Since the 3 September Version 114

List of Figures

Dataset note: Figures 1-18 and 20-27 are rendered from the primary dataset of this revision (the 120-run matched-window matrix of Section 2.4), and each agrees with the table beside it. Figure 19 alone is retained from the pre-revision dataset, because the ReadVoltage sweep behind it was not re-run at the matched organization; its caption and Section 3.1.5 both say so. Every bar chart plots the six primary configurations only: the two published-silicon rows of Table 2 and the 64 B DDR5 cross-check of Section 2.3 are deliberately excluded from the figures, so no reading of Figures 1-6 is complete without Table 2's silicon rows beside it. The Pareto figures (20-25) plot all four architectures of Table 8, so the 8-to-16-chip step that Section 3.2 turns on is visible in them. Appendix D lists what moved since the 3 September version and why. "AlexNet Layer 1" throughout means SCALE-Sim's zero-indexed `layer1` *folder, which is AlexNet's second convolution layer (Section 2.4). Figure 28 is not a bar chart at all: it is a sketch of position drawn by* `tools/plot_far_memory_sketch.py` *from the same primary dataset plus two cited literature constants, and its caption says what it is and is not.*

• Figure 1: Average Total Latency under the compute-bound GCC (SPEC2017)

- Figure 2: Average Total Latency under the continuous memory-streaming LBM (SPEC2017)
- Figure 3: Average Total Latency under the parallel read-storm of GPT-2 Inference (IFMAP)
- Figure 4 & 5: Average Total Latency under AlexNet Layer 1 IFMAP (pure-read) and OFMAP (pure-write)
- Figure 6: Average Total Latency under the STREAM memory bandwidth benchmark.
- Figure 7: System Power Breakdown (Static / Dynamic / Refresh)
- Figure 8: System Power Breakdown (Static / Dynamic / Refresh)
- Figure 9: System Power Breakdown (Static / Dynamic / Refresh)
- Figure 10: System Power Breakdown (Static / Dynamic / Refresh)
- Figure 11 & 12: System Power Breakdown (Static / Dynamic / Refresh)
- Figure 13 & 14: Power-Delay Product under single-threaded compute (GCC SPEC2017)
- Figure 15 & 16: Power-Delay Product under heavy continuous memory-streaming workloads (LBM SPEC2017 and STREAM)
- Figure 17 & 18: Power-Delay Product under AlexNet Layer 1 IFMAP (Read-Heavy)
- Figure 19: ReadVoltage Sensitivity Analysis - 1T1R SLC Full DIMM (LBM and GCC, +/-20% sweep)
- Figure 20: Pareto scaling trajectory under the single-threaded GCC workload.
- Figure 21 & 22: Pareto scaling trajectories under highly parallel AI inference workloads (GPT-2 IFMAP and AlexNet IFMAP)
- Figure 23: Pareto scaling trajectory under write-torture conditions (AlexNet OFMAP)
- Figure 24 & 25: Pareto scaling trajectories under continuous memory-streaming workloads (LBM SPEC2017 and STREAM)
- Figure 26: Die-Level Density Comparison, normalized to DDR5 = 1.
- Figure 27: Overall System Efficiency (Geometric Mean PDP)
- Figure 28: Density against latency - a sketch of position (Section 3.4).

List of Tables

- Table 1: NVSim device-level characterization per 1 Gb chip (22nm FinFET LOP, 1.4 V ReadVoltage nominal, matched 2048 x 2048 subarrays at mux 64)
- Table 2: Average total latency (ns), full-DIMM configurations, 250 ms matched window, including the two published-silicon counterweights

- Table 3: Total module power (W) and power per gigabyte (W/GiB), full-DIMM, module-sum semantics
- Table 4: Power-Delay Product (W·ns = nJ), full-DIMM, and six-workload geometric means
- Table 5: Projected lifetime and required endurance (1T1R SLC full DIMM, 64 GiB, 10^6 cycles/cell, four wear-leveling policies)
- Table 6: Projected die-level density versus DDR5 (= 1.00) under quadratic feature-size scaling and 3D deck stacking
- Table 7: Cross-technology summary, full-DIMM configurations
- Table 8: Chip-count scaling (1 / 8 / 16 / 64 chips), average total latency and Power-Delay Product, 1T1R SLC
- Table 9: The write-pulse bracket - average total latency and end-to-end latency, full-DIMM configurations, default mode against bounding mode (Section 3.1.1)
- Table 10: Device reference row - this book’s modeled media beside measured devices (Section 3.4.5)
- Appendix D table: What changed since the 3 September version

1. Introduction & Background

1.1. Project Context

- **The External Memory Wall**: Traditional DRAM is struggling to meet the bandwidth requirements of modern processors due to physical scaling limits, specifically capacitor leakage and transistor short-channel effects **[5]**.
- **Economic Drivers**: By late 2025, the massive reallocation of DRAM wafers to AI-focused HBM production created a severe supply-demand gap for commodity main memory **[1], [2]**.
- **The Objective**: This project investigates whether memristor-based non-volatile memory (NVM) can effectively substitute traditional DRAM in standard DIMM modules to alleviate this constraint.
- **The Endurance Constraint**: However, deploying NVM as main memory introduces a critical challenge: endurance. Because physical wear-out limits are the primary bottleneck of memristive technologies, a viable DRAM alternative necessitates architectural interventions to prevent premature device failure.

1.2. ReRAM Fundamentals

Before the engineering choices, the device itself: a memristor (ReRAM cell) is a two-terminal metal-insulator-metal stack - typically an HfOx or TaOx oxide - whose electrical resistance *is* its stored state. A programming voltage grows or ruptures a conductive filament of oxygen vacancies through the insulating oxide, switching the cell between a low-resistance state (LRS) and a high-resistance state (HRS); a read applies a small sense voltage that leaves the filament undisturbed [14]. Because the filament persists without power, storage is non-volatile - no refresh, no standby charge maintenance - and because state is resistive rather than capacitive, the cell has no capacitor to scale, which is precisely the wall DRAM's planar cell faces [5]. Writes physically stress the filament, so endurance is finite (Section 3.1.4), and filament formation is stochastic, which is why the resistance targets and sensing margins below are engineering choices rather than free parameters.

- **Process Selection:** I standardized on 22nm FinFET LOP (Low Operating Power). This node allows for logic-compatible integration **[8]** while the LOP profile aligns with the thermal and power constraints of high-capacity main memory replacement. The choice also frames the entire comparison conservatively: commodity DDR5 is fabricated on 10 nm-class DRAM nodes (1γ [17]) that sit at the end of the capacitor's scaling roadmap [5], whereas 22nm is a mature, logic-compatible CMOS node with several shrink generations of headroom still ahead of it - the density figures in this evaluation are therefore a floor on ReRAM's potential, not a ceiling.
- **Resistance Design:** Cells are calibrated to high resistance targets ($10^5\Omega$ LRS and $10^9\Omega$ HRS) **[7]** to mitigate IR-drop on long bitlines, ensuring reliable sensing margins. Matsui et al. set the $10^5\Omega$ LRS floor at a 1024-cell bitline, where the retained fraction of the ideal bitline current is $\alpha = 0.99$ with a 22nm single-level Cu line of 1 Ω per cell pitch. The arrays simulated in this

book are 2048 cells on a side (Section 3.1.6 and Appendix A), one doubling past that design point: by the same expression, $\alpha = R_{\mathrm{LRS}}/(R_{\mathrm{LRS}} + k \cdot r_{\mathrm{BL}})$, a 2048-cell bitline retains $\alpha = 0.980$ on single-level interconnect and $\alpha = 0.990$ on double-level (0.5 Ω per pitch), so the chosen organization meets Matsui's own criterion exactly with a double-level line and stays within 1% of it otherwise. The 2048-cell side is not extrapolation: it is anchored to fabricated ground truth on both device topologies, Micron and Sony's 27nm 16 Gb 1T1R part **[49]** and SanDisk and Toshiba's 24nm 32 Gb cross-point, whose block is 2K bitlines by 4K wordlines **[48]**.

- **Device Structures & Sneak-Path Mitigation:** I evaluate two distinct hardware topologies to handle array sneak-paths:
  - **1T1R (Transistor-based):** Utilizes CMOS access transistors to completely eliminate sneak-paths. While this results in a logic-compatible but larger bit-cell area - a commonly-used, transistor-drive-limited engineering estimate of approximately $20F^2$ (derivation and demonstrated-endpoint caveats in Appendix A) - modern commercial foundries currently mass-produce 22nm embedded ReRAM (eReRAM) macros at megabyte scale **[24]**, inherently validating its manufacturability far beyond the 1Mb subarrays evaluated here.
  - **1S1R (Selector-based):** Suppresses local sneak-paths via highly non-linear thresholding. While academic 1S1R test-chips frequently restrict subarray sizes to manage leakage, industry milestones - such as Crossbar Inc.'s fabrication of a 4Mb cross-point array utilizing a super-linear threshold selector **[9]** - demonstrate the physical viability of massive crossbar scaling. Building on this foundation, combining the NVSim non-linear selector model **[3]** with the aforementioned ultra-high resistance calibration **[7]** provides a theoretical architectural projection capable of scaling to the 2048-cell bitlines simulated here. This reduces the cell to an idealized $4F^2$, enabling a significant footprint reduction for high-density DIMMs. NVSim models no selector physics of its own, so whether a 2048-cell tile is actually sneak-path-viable is settled separately, by the analytic selector layer of Section 3.1.2: at a present-day ovonic-threshold-switch quality it is not, and at the best published selector it is.
- **Area Realism over Financial Metrics:** To evaluate module cost and density objectively across these topologies, I utilize area-per-bit (F^2) rather than financial cost-per-bit, isolating architectural evaluation from fabrication economics and market volatility.

1.3. Related Work

The architectural case for replacing DRAM with a non-volatile main memory was first made at scale by three concurrent ISCA 2009 studies, all targeting phase-change memory. Lee et al. architected PCM as a scalable DRAM alternative, narrowing a raw 1.6x delay / 2.2x energy deficit to within 1.2x / 1.0x through row-buffer reorganization and partial writes [36]; Qureshi et al. hybridized a large PCM store behind a small DRAM buffer (~3% of PCM capacity) with lazy writes and fine-grained wear leveling [37]; and Zhou et al. attacked durability directly, extending projected PCM lifetime to 13-22 years via redundant-bit-write removal and row-shifting wear

leveling [38]. That generation established the field's enduring template - hide the write penalty, level the wear, buy density where latency cannot be hidden - but it evaluated DDR2/DDR3-era baselines, and its commercial embodiment, Intel's Optane/3D XPoint, was ultimately discontinued on cost economics rather than physics [23].

For the resistive technology evaluated here, the closest prior work is Xu et al.'s HPCA 2015 study of crossbar ReRAM as main memory, which characterized the IR-drop- and sneak-current-induced, data-dependent RESET latency of large crossbars and recovered to within ~10% of an ideal DRAM-only system through split-phase RESET and compression-based encoding [39]. Kültürsay et al. ran the parallel exercise for STT-RAM, finding an unmodified STT-RAM main memory uncompetitive but a lightly optimized one performance-comparable to DRAM at ~60% lower memory energy [40]. On the empirical side, Izraelevitz et al. published the first comprehensive third-party characterization of the only NVM main-memory module ever shipped - the Optane DC PMM - and found its behavior substantially more nuanced than the "slow, persistent DRAM" abstraction earlier emulation-based studies assumed [32]: a standing caution for every simulation-only evaluation, this one included (Section 2.2, Validation Scope). Concretely, on real silicon [32] measured idle random-read latency of 305 ns against 81 ns for local DRAM on the same platform (~3x, their Section 3.1.1), and a single-DIMM maximum bandwidth of 6.6 GB/s read / 2.3 GB/s write (their Abstract) - real, measured numbers this book's own simulated ReRAM/PCM figures are never directly benchmarked against, since Optane's selector-gated PCM architecture and this study's technologies differ in both material and device topology. They are reported here as the field's one genuine real-hardware anchor point, explicitly distinct from every simulated result in Section 3.

Against that body of work, this book's contribution is fivefold: (a) the baseline is commodity DDR5 with vendor-calibrated power, not the DDR2/DDR3-era DRAM of the founding studies; (b) the ReRAM inputs are a device-anchored 22nm NVSim characterization propagated into cycle-accurate NVMain simulation, rather than abstract latency/energy multipliers; (c) transistor-gated 1T1R and selector-gated 1S1R are compared head-to-head at a matched process node *and a matched array organization* - 2048 x 2048 subarrays at sense-amp mux 64 for both, anchored to fabricated parts of each topology **[48], [49]** - so that what the comparison isolates is the access device and nothing else. That control is the contribution, because it is what an earlier version of this work lacked: the large transistor-versus-selector leakage separation that version reported, and built its architectural verdict on, turned out to be a consequence of simulating the two cells at different mat counts, and it vanishes at a matched organization (Section 3.1.2, Appendix A; the superseded figures are tabulated in Appendix D); (d) the simulation toolchain itself is audited and repaired (Section 3.1.6), where prior studies take the NVSim-to-NVMain wiring on trust; and (e) endurance is projected from the write distribution measured per physical line location under real traces, and across four wear-leveling policies including a faithful Start-Gap remapper **[52]**, rather than from an analytic write rate under an assumption of ideal leveling. The complementary boundary is equally explicit: this work models the crossbar at NVSim's calibrated array-level abstraction (Section 1.2) and does not reproduce [39]'s circuit-level, data-dependent write timing

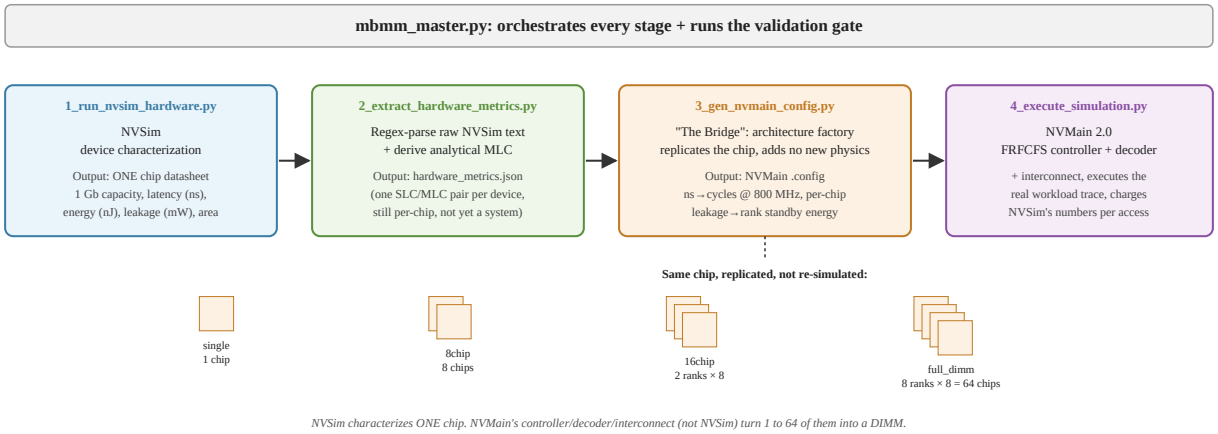
- the two levels of analysis answer different questions, and integrating them is part of the agenda in Section 4.1.

Every technology evaluated in this book is judged on the same three axes: **latency** (Section 3.1.1), **density** (Section 3.3), and **endurance** (Section 3.1.4) - with power as a fourth, closely related dimension (Sections 3.1.2-3.1.3). This framing is stated explicitly here because it also gives a home for admitting when an axis is *not* evaluated for a given technology - for instance, this study's PCM baseline was never carried through an endurance projection (Section 3.1.4), a disclosed scope choice rather than a missing result.

2. Infrastructure & Methodology

2.1. Decentralized Structure & Simulation Stack

- **The Bridge:** I developed a cross-layer pipeline that automatically translates device-level physical metrics from NVSim (Area, absolute Latency in nanoseconds, Energy) into cycle-accurate architectural parameters for NVMain 2.0 (JEDEC timing constraints such as Column Address Strobe latency (tCAS), Row-to-Column Delay (tRCD), Row Precharge time (tRP), and dynamic energy-per-access coefficients).
  Core memristive cell parameters and resistance targets were rigorously calibrated against established silicon fabrication literature. A full table of these electrical parameters and their corresponding references is provided in **Appendix A**.
- **Simulation Pipeline**: The pipeline follows a structured sequence: hardware characterization in NVSim → diagnostic extraction to JSON → system configuration generation → cycle-accurate trace execution in NVMain. NVSim characterizes exactly **one 1 Gb** chip (Table 1); NVMain never re-touches cell-level physics; it replicates that same chip **1, 8, 16, or 64** times per configuration and wraps the result in its own controller, decoder, and interconnect - components NVSim has no concept of. The diagram below illustrates this end to end.



NVSim characterizes ONE chip. NVMain's controller/decoder/interconnect (not NVSim) turn 1 to 64 of them into a DIMM.

Diagram: NVSim's device characterization (one chip) becomes an NVMain system via replication and NVMain's own controller logic - not part of the numbered Figure/List of Figures sequence, since it illustrates pipeline architecture rather than a simulation result.

- **Decoupled Benchmarking**: To ensure simulation stability, I bypassed brittle integrated wrappers in favor of a robust trace-based pipeline utilizing gem5 v25.1 memory access logs. I transitioned to 502.gcc_r and 519.lbm_r using gem5's X86O3CPU (Out-of-Order). The audit of Section 3.1.6 established that the traces used in earlier versions of this book carried no caches, no warm-up and no region of interest; every trace in this version has been regenerated with L1 and L2 caches enabled, a documented detailed region and a 10 ms warm-up discard (Section 3.1.6 item 6, Appendix A), and each reconciles exactly with gem5's own request counters - a documented provenance rather than a documented limitation for absolute latency magnitudes.

2.2. Validation & Protocol

- **The Gate-Keeper**: All architectural conclusions are gated by the **mbmm_master.py** script. This script mandates a validation flow of cleaning build artifacts, recompiling the C++ engines, and passing a sanity-check simulation before data is accepted.
- **Critical C++ Repairs**: To modernize the legacy simulation engines, I applied legacy compatibility patches:
  - **NVSim**: Enforced -std=c++11 to resolve namespace collisions between MemoryType::data and std::data in modern GCC.
  - **NVMain**: Repaired a fatal memory deallocation bug in FlipNWrite.cpp (mismatched new/delete) that triggered SIGSEGV failures.
  - **Build Migration**: Migrated the legacy build system from Python 2 to **Python 3** using 2to3.
  - **MLC Analytical Modeling**: Prior to simulating Multi-Level Cell (MLC) scaling, I established the physical and architectural differences between SLC and MLC - a technique with established multi-bit-per-cell precedent in the literature, demonstrated up to 3 bits per cell [13]. Recognizing that NVSim lacks native support for the complex Analog-to-Digital (ADC) sensing and Iterative Step-and-Verify (ISPV) programming required for MLC, I implemented an Analytical Penalty Method (1.5x read latency, 3.263x write latency, 1.1x read energy, 3.0x write energy, measured on the same EMBER macro's two publications - Upton et al. [6], ESSCIRC 2023, Table I, for the read-energy figure, and Levy et al. [31], IEEE JSSC 2024, Section V, for the read-latency and write-side figures - see Appendix A for the full derivation) rather than relying on NVSim's native MLC implementation, which triggers a floating-point exception in Mat.cpp's ISPV sensing logic.

- **Validation Scope**: The validation flow above establishes *internal* consistency, not hardware correlation. Input parameters are anchored to real silicon and vendor documents - the EMBER macro's published measurements [6], [31] and the vendor DDR5 IDD datasheets of Section 2.3 - and every repair in Section 3.1.6 is verified against NVSim's device-level output or exact predicted arithmetic (0.0% error). No end-to-end result, however, is validated against a measured ReRAM DIMM or an instrumented DDR5 system - no ReRAM main-memory module exists to measure. The “0.0% error” claims throughout this book are therefore pipeline-fidelity guarantees (the system level faithfully reflects its device-level inputs), not accuracy guarantees against physical hardware.

2.3. Hardware Baselines & Memory Models

- **The DDR5-4800 Baseline Reference:** The DDR5 configuration's device timings are taken from the JEDEC Solid State Technology Association, Standard JESD79-5 (DDR5 SDRAM) [10], with three documented exceptions and a handful of simulator-specific keys. The precise scope is this: *every DDR5-4800 device timing in the primary baseline is taken from JESD79-5, except that (a) where NVMain keeps a single rank-wide value and JEDEC splits the parameter into a short and a long form by bank group - tCCD, tWTR and tRRD - the **short** value is used, which is the choice that favors the DDR5 baseline, and (b) a small number of NVMain-specific keys with no DDR5 counterpart keep their template values, none of which is a device timing that binds except the one-cycle rank-to-rank bus-turnaround term tRTRS.* A key-by-key sweep of the configuration labels its 36 device and simulator keys (the timing parameters plus the clock, rate, refresh-policy and power-down-policy settings that govern them) as 22 JEDEC-sourced, 9 NVMain-specific with no DDR5 counterpart - three of which can never bind in this configuration: the DLL-off power-down exit tXPDLL, because DDR5 defines no such mode and NVMain never reaches that branch under fast exit; the self-refresh exit pair tXS and tXSDLL, because NVMain models no self-refresh entry at all; and the ODT switching term tOST, which binds only in the two-rank 64 B cross-check configuration - 2 parsed but read by no timing code at all, and the 3 short-versus-long judgements above. Two further disclosures belong with it. The values were transcribed from the JESD79-5 tables through a public mirror; on 2026-10-19 every one of them was checked against Micron’s DDR5 SDRAM core data sheet [101], the vendor’s full transcription of JESD79-5, and against two DDR5-4800 module datasheets, and all agree, including the three power-down entry formulas (research note `ddr5_jedec_timing_verification_2026-10-19.md`); the JEDEC text itself remains unread, since it is access-gated. And NVMain charges the activate-to-activate constraint tRRD, and advances its four-activate window, on every *refresh* as well as on every activate, which JEDEC does not do for an all-bank refresh; that is template behavior in the simulator, not a configuration choice, and it makes the modeled DDR5 marginally pessimistic. The core numbers are the 4800 MT/s speed bin at 1.1 V: tCAS-tRCD-tRP of 40-39-39 (Section 3.1.6, item 12), CWL = CL - 2 = 38 cycles, tWR 30 ns, tRTP 7.5 ns, tPD and tXP 7.5 ns, tRRD_S 8 cycles and tFAW 32 cycles. The write-path, power-down and activate-window timings were DDR3-1333 template cycle counts until this revision corrected them; Appendix A, “DDR5 timing and power corrections of this revision”, records what moved. The configuration also

explicitly models DDR5's defining architectural upgrade: *two independent 32-bit sub-channels per DIMM, each with its own command bus and 32 banks, serving a 64-byte access as a 16-beat burst on its 32-bit bus*. Earlier versions of this book asserted that model in prose while the configuration actually simulated was a single 64-bit channel with 32 devices and a 32 GiB capacity; the configuration now matches the prose, and the module it describes is a conventional 16 GiB DDR5-4800 RDIMM built from eight 16 Gb x8 devices. The subchannel split is what imposes the real JEDEC bus serialization penalty: a 64-byte line takes sixteen beats on a 32-bit subchannel rather than eight on a 64-bit channel, and the two subchannels schedule independently. A single-channel 64-byte cross-check configuration is retained to separate the effect of access granularity from the effect of module shape, and it was run on three traces. On the two CPU traces the two models are nearly equal: the cross-check averages 79.6 ns against the subchannel model's 83.1 ns under GCC and 75.9 against 78.3 ns under LBM, a 3 to 4% difference on a module that completes the identical request population either way. On the dense AlexNet output-map burst they separate sharply: the cross-check averages 69.2 ns in the controller and 63.6 µs end to end against 234.3 ns and 209.9 µs for the subchannel model, that is **3.4x faster in the controller and 3.3x faster end to end**. The gap is the serialization the subchannel split imposes, and it is real: sixteen beats on a 32-bit bus occupy the subchannel twice as long as eight beats on a 64-bit one, which only a burst dense enough to queue behind itself can expose. **The two-subchannel model remains the primary baseline throughout this book**, because it is the organization JESD79-5 actually specifies and the cross-check is not. The cross-check is moreover a *legacy-shape reference rather than a JEDEC DDR5 configuration*: a 64-bit channel serving a 64-byte line as an 8-beat burst is a DDR4 shape that DDR5 does not offer at all, and to keep that shape the configuration holds tCCD at 4 cycles, below the 8-cycle JEDEC floor for DDR5-4800. Its speed advantage over the two-subchannel baseline is therefore optimistic and is quoted here only as an upper bound on what access granularity and module shape are worth. The consequence is that every DDR5 figure in the dense-burst regime is a *conservative* baseline, and a reader who prefers the 64-byte shape should read Section 3.1.1's AlexNet output-map comparison as up to 3.3x harder on ReRAM than it appears. Refresh is modeled explicitly to the same standard: an all-bank refresh command every tREFI = 3.906 µs (the JEDEC 32 ms retention window spread over 8,192 commands), each occupying tRFC = 295 ns, at the DDR5 supply of 1.1 V, with refresh energy accumulated in dedicated rank-level counters - the source of the refresh decomposition in Section 3.1.2. These refresh and voltage parameters were corrected during the fidelity audit (Section 3.1.6, items 7-8) after being found inherited, unrescaled, from a DDR3-1333 template configuration.

- **The Microsoft PCM 2009 Baseline:** A legacy non-volatile memory baseline, named for its architectural framing in Lee et al. **[11]**, used to benchmark modern 22nm ReRAM against older NVM scaling constraints. Unlike ReRAM, PCM was never run through NVSim's device-characterization pipeline: its concrete timing and energy parameters are inherited unmodified from NVMain's own bundled example configuration (**pcm_microsoft_2009.config**), whose header attributes those specific numbers to a different device paper - Choi et al.'s 20nm, 1.8 V,

8 Gb PRAM (ISSCC 2012) **[41]** - not to Lee et al. This project did not independently re-derive or verify those inherited numbers. It runs at its cited derivation basis of CLK = 400 MHz - restored during the fidelity audit (Section 3.1.6, item 9) after a stray clock override was found running it twice as fast as its timing parameters intend - and runs the same 250 ms matched window as every other configuration (Section 3.1.6, item 11), with an identical admitted trace population. Neither its inherited config nor its cited source papers specify a particular access-device architecture (transistor- or selector-gated) - this baseline is a black-box timing/energy model, not a characterization of any specific PCM cell topology. This is a disclosed scope gap, not a hidden assumption: the real, shipped commercial embodiment of PCM - Intel/Micron's Optane/3D XPoint, architecturally confirmed by Intel/Numonyx's own IEDM 2009 device paper **[42]** and independently corroborated by a professional physical teardown of a retail unit - is a selector-gated 1S1R cross-point design (an Ovonic Threshold Switch in series with the phase-change element, no access transistor), not modeled here at all. Real, measured performance numbers from that shipped product are reported alongside this study's simulated results in Section 1.3.

- **The ReRAM DIMM Interface (800 MHz):** All ReRAM configurations run behind an 800 MHz (1600 MT/s) DIMM interface - a deliberate modeling choice, not a device limit, and one that reaches much further into the reported latencies than a bandwidth ceiling would. **Only two of the protocol timings in a generated ReRAM configuration come from NVSim**: tRCD plus tCAS, which together carry the device read latency exactly once (for 1T1R SLC, 4 + 5 cycles at 800 MHz, 11.25 ns against a 10.12 ns device read), and tWR, the write recovery time. tRP is fixed at one cycle by ruling, because a resistive read is non-destructive and has no row to restore. Every other protocol timing the simulation applies is a constant in *interface cycles*, never derived for ReRAM, and therefore scales in nanoseconds with whatever clock is chosen: tCWD 7, tRTP 5, tXP 6, tPD 6 and tWTR 5 are NVMain's own template defaults, which the generator never overrides at all, while tBURST 4 and tCCD 4 are this project's own generator constants, which happen to equal NVMain's defaults. Measured on the GCC average for the 1T1R SLC full DIMM, the NVSim-derived part is 9 of 33.1 memory cycles, 11.25 ns of 41.40; the remaining 30.2 ns is default cycle counts at 800 MHz. At 2400 MHz the same request costs 45.7 cycles, of which 25 are NVSim-derived: 10.42 ns of array time and 8.6 ns of everything else. That, and not a clock-independent device time, is why tripling the interface clock takes the average from 41.4 ns to 19.0. One further consequence is on the write side: NVMain's default write-pulse time tWP is zero and this project's generator does not override it, so the NVSim cell write time is booked as tWR, the write recovery time, and **reaches the request path only when a write closes its row**. Which branch a write takes decides it. Under the close-page policy these configurations use, `MemoryController::IsLastRequest` returns false as soon as another queued request targets the same rank, bank, row and subarray. A write that *is* the last queued request to its row becomes a write-with-precharge: NVMain adds tWR to that subarray's next activation and maxes the subarray's next precharge, next read and next write against it, so the next access to the subarray waits for the cell. A write followed by another queued request to the same open row takes the plain write branch instead, where tWR

reaches only the next precharge; the subarray's next read and next write carry the burst and the write-to-read turnaround (tWTR, NVMain's default of 5, identical in every configuration here) and no tWR at all. In *neither* branch does the cell write time delay the write's own completion event. The scope of the delay is the write's own subarray and not the whole bank, because NVMain's nextActivate, nextPrecharge, nextRead and nextWrite are subarray members and a bank holds ROWS / MATHeight subarrays - one for the SLC full DIMM, two for MLC. (NVMain reaches the tWP pulse only through its `UniformWrites` branch, which returns tWP itself; the alternative branch's 40- and 60-cycle tWP0 / tWP1 defaults, which *would* sit on the request path, are never reached in this book's configurations because `UniformWrites` defaults to true and no generated config sets it otherwise.) A write retires after tCWD + tBURST = 11 cycles = 13.75 ns on every ReRAM track, whatever NVSim priced the cell write at (15.26 ns for 1T1R SLC, 49.79 ns for 1T1R MLC, 80.23 ns for 1S1R MLC). Section 3.1.1 brackets that modeling choice with a bounding run that moves the same cycle count onto tWP, that is, onto the write's own completion. The rate is positioned between two precedents: twice the 400 MHz basis of NVMain's PCM reference lineage [11], and below the DDR4-2666 electrical interface carrying the DDR-T protocol on Intel's shipped Optane persistent-memory DIMMs [32], [53] - the industry precedent that NVM DIMM interfaces trail the contemporary DRAM PHY rather than match it. Where the choice matters is the bandwidth-bound AI regime of Section 3.1.1, where every technology is queue-bound rather than device-bound, so pairing the same media with a faster PHY is an available mitigation, not a hidden cost. That mitigation was measured rather than asserted. Re-running the ReRAM configurations at 1333 and 2400 MHz, with the media, the traces and the window unchanged and with the DDR5 and PCM baselines left at their own clocks, gives for the full DIMM: under GCC, 1T1R SLC 41.4 ns at 800 MHz, 27.9 at 1333 and 19.0 at 2400 (1S1R SLC 36.8 / 23.6 / 14.9); under LBM, 43.1 / 28.8 / 19.3 (1S1R SLC 38.1 / 24.5 / 15.2); and under the write-dominated AlexNet output-map burst, 353.1 / 263.1 / 200.4 ns in the controller and 486.1 / 366.5 / 283.3 µs end to end. Every configuration completes the identical request population at every clock, so these are service rates and not completion effects. Two readings follow. The interface is a *real* term in the modeled latency, worth 2.2x on the CPU traces between 800 and 2400 MHz and taking 1T1R SLC from 41.4 ns to 19.0 ns against a DDR5 baseline of 83.1 ns; and it is *not* the binding term in the dense-burst regime, where tripling the clock buys 1.8x in the controller and 1.7x end to end because the queue, not the bus, is what the burst is waiting on (Section 3.1.1). None of this changes the power verdict: interface clock does not enter the NVSim leakage figure at all, so a faster PHY buys latency at no modeled static-power cost and the per-gigabyte gap of Section 3.1.2 is unmoved. *This 800 MHz figure itself has no independent ReRAM-interface citation* - it is positioned only by relationship to the two precedents above, not derived from a ReRAM PHY specification. A targeted literature search (EMBER [6]/[31]'s “100 MHz” is confirmed, by both papers' own text, to be the macro's internal sense/write-circuit clock, not an external interface rate - no interface clock is stated for either device) found no independent ReRAM interface-speed citation to support it; the only real, datasheet-verified ReRAM chip I/O clocks located (Fujitsu's MB85AS8MT and MB85AS4MT SPI-interface parts, 10 MHz and 5 MHz

respectively) run 80-160x *slower* than 800 MHz, actively contradicting rather than merely failing to support this figure. 800 MHz should therefore be read as a deliberate, disclosed modeling assumption positioned between two DRAM/PCM-adjacent precedents, not a literature-derived ReRAM interface speed.

- The 2D and 3D DRAM Control Configurations: NVMain 2.0 ships with native 2d_dram (legacy planar, DDR3-era, 666 MHz) and 3d_dram (Wide I/O-style Through-Silicon Via, 1333 MHz) example models. These are idealized textbook abstractions rather than models of purchasable products: unlike the DDR5-4800 model, which faithfully pays JEDEC bus serialization and 16-beat burst transfer on a 32-bit subchannel, the example configurations bypass real-world interface overheads. They were retained in the simulation matrix as pipeline control variables - useful for verifying clock-domain normalization and for bounding theoretical throughput - but they are excluded from every reported cross-technology comparison, which is restricted to technologies with commodity referents: DDR5-4800, PCM, and the four ReRAM configurations. The 2D/3D runs are preserved in the released dataset (Appendix C) for completeness.
- **Module capacity, access granularity and channels:** every technology in this comparison is accessed at a *64-byte* granularity, which is the x86-64 line size, NVMain's hardwired address-translation grid, and the granularity a 16-beat DDR5 burst on a 32-bit subchannel delivers. The simulated ReRAM DIMM is **8 GiB**: 64 chips of 1 Gb, arranged as 2 channels x 4 ranks x 8 devices, with 8 banks of 2048 x 1024 sixty-four-byte locations per device. Every configuration's simulated capacity now equals the physical module it describes; earlier versions of this book generated a full-DIMM configuration whose decoded address space was 512 GB, 64x the module, which left the timing and energy results intact but changed which rank an address decoded to. The MLC tracks store two bits per cell and therefore describe a 16 GiB module of the same 64 chips. The DDR5 baseline is 16 GiB of eight 16 Gb x8 devices, so every power comparison in this book is made *per gigabyte*, never module against module. Two channels is the primary configuration, matched to DDR5's two subchannels; because channels are formed by splitting ranks rather than by adding capacity, only the 16-chip and full-DIMM ReRAM architectures have ranks to split, and the single-chip and 8-chip architectures remain single-channel.

Both ReRAM cell types are characterized at one *matched array organization*: 2048 x 2048 subarrays at a sense-amp column mux of 64, forced identically for 1T1R and 1S1R and verified against NVSim's own printed geometry on every run (Section 3.1.6, Appendix A). This is the single most consequential methodological change in this revision. In earlier versions the two cells were characterized at hand-set column muxes of 32 and 256, which drove NVSim to 256 mats for 1T1R against 2 for 1S1R, and almost every device-level difference those versions reported - a large leakage gap, a read-latency penalty for the selector, an outsized die-area advantage - was a consequence of that organization rather than of the access device (Appendix D). At the matched organization the picture is different and much simpler: leakage is *identical* (108.384 mW per chip for both, because NVSim assigns memristor cells zero leakage and

charges only peripheral circuitry, which scales with mat count), the selector is *faster* on reads (4.70 against 10.12 ns) and slower on writes (24.59 against 15.26 ns), and the die-area advantage is 3.4x rather than the 8.7x retired in Appendix D. Table 1 collects the characterization the system simulation inherits.

Table 1: NVSim device-level characterization per 1 Gb chip (22nm FinFET LOP, 1.4 V ReadVoltage nominal, matched 2048 x 2048 subarrays at mux 64).

Config.	Cell size	Read lat. (ns)	Write lat. (ns)	Read energy (nJ)	Write energy (nJ)	Leakage (mW/ chip)	Die area (mm^2)
1T1R SLC	$20F^2$	10.12	15.26	0.376	0.913	108.384	12.008
1T1R MLC*	$20F^2$	15.18	49.79	0.413	2.740	108.384	12.008
1S1R SLC	$4F^2$	4.70	24.59	0.329	1.078	108.384	3.540
1S1R MLC*	$4F^2$	7.05	80.23	0.361	3.234	108.384	3.540

**MLC rows apply the Analytical Penalty Method (1.5x read latency, 3.263x write latency, 1.1x read energy, 3.0x write energy - Section 3.1.6, item 13); leakage and die area are unchanged. Both cell types are forced to the same 2048 x 2048 subarray at mux 64, which is why their leakage is identical: NVSim charges memristor cells no leakage at all, so chip leakage is peripheral circuitry and follows mat count, not access-device type. The one device-level advantage that survives a matched organization is the selector's die area, 3.540 against 12.008 mm^2 per 1 Gb chip. Sources: results/rev2026-09_primary_csv/hardware_metrics.json; Sections 2.1-2.2.*

Two further characterizations are carried alongside Table 1 throughout this book, and both are deliberately conservative counterweights to it. The first is the *published-silicon sensitivity*: NVSim's numbers are an array-core projection, and no fabricated ReRAM part reads or writes in nanoseconds. Two configurations are therefore generated directly from fabricated-chip data sheets and run on the same traces and the same module geometry - Micron and Sony's 27nm 16 Gb 1T1R part at 2.3 us read and 11.7 us write **[49]**, and SanDisk and Toshiba's 24nm 32 Gb 1S1R cross-point at 40 us read and 230 us write **[48]**. They are labeled 1T1R SILICON and 1S1R SILICON and appear next to every latency result. The second is the *analytic selector layer* of Section 3.1.2, which supplies the sneak-path physics that NVSim does not model at all.

2.4. Workload Traces & Benchmarks

To rigorously evaluate the memory architectures across different access patterns, I selected a diverse suite of workloads spanning traditional single-threaded execution, pure bandwidth stress tests, and modern parallel AI:

- **SPEC CPU2017 (502.gcc_r & 519.lbm_r):** Generated via gem5 [26], from the SPEC CPU2017 suite [28], each run on its reference input. gcc was selected as a compute-bound, control-flow heavy workload with irregular memory accesses. In contrast, lbm (fluid dynamics) was selected as a memory-bound workload to test sustained, sequential memory throughput. A third SPEC workload, 505.mcf_r, was attempted and is *absent* from this evaluation: gem5 panics about 0.13 ms into the detailed region, inside the benchmark's own input reader, on both the March and the September attempts. Recovering it needs a rebuilt benchmark binary, so mcf is parked rather than reported, and no mcf number appears anywhere in this book.
- **STREAM Benchmark:** The real STREAM benchmark [27], compiled statically with OpenMP disabled at its default array size and traced through gem5 exactly as the SPEC workloads are, used to measure the practical upper bound of sustained memory bandwidth and expose baseline latency bottlenecks. Earlier versions of this book replayed a hand-written synthetic copy kernel under this name; that stand-in has been retired.
- **SCALE-Sim ML Traces (AlexNet & GPT-2):** Generated via the SCALE-Sim systolic array simulator. These workloads were selected to represent modern, highly parallel AI operations. AlexNet is split into IFMAP (read-dominant) and OFMAP (write-heavy "Write-Torture") to isolate the ReRAM write-latency penalty, while GPT-2 evaluates massive transformer-based memory-level parallelism (MLP). SCALE-Sim's output folder named `layer1` is zero-indexed, so the AlexNet traces used here and in every earlier version of this book are AlexNet's *second* convolution layer, not its first; the label is kept for continuity with the earlier results, and it means Conv2. All three AI traces are *microsecond bursts*, not sustained workloads: their whole captured span is 2.2 us (GPT-2 IFMAP), 60.5 us (AlexNet IFMAP) and 4.5 us (AlexNet OFMAP), against a 250 ms replay window. Any rate derived from them is a burst rate and is labeled as such wherever it is used (Section 3.1.4).

2.4.1. Diagnostic Trace Dictionary

The following specific traces were executed across all simulated architectures to isolate distinct memory behaviors:

- **gcc_spec2017 (Compute-Bound):** A standard CPU compilation workload whose control-flow-heavy instruction mix generates sparse, irregular memory traffic (in-window mix: 298,792 reads / 220,776 writes - 58/42, of a 545,566-record trace). **Purpose:** It leaves the main memory mostly idle, perfectly exposing the pure *Static Leakage Power* of the DIMMs. That idleness is real last-level-cache-filtered traffic: the gem5 run carries L1 and L2 caches (Section 3.1.6, item 6), so what reaches the memory controller is what the cache hierarchy actually missed.

- **lbm_spec2017 (Memory-Streaming):** A fluid dynamics simulation that constantly streams large data vectors from memory (in-window mix: 3,179,900 reads / 2,384,804 writes - 57/43, at roughly eleven times GCC's request density). **Purpose:** It highlights the *Dynamic Energy* penalty of switching topologies, as memory is constantly active.
- **stream (Vector-Streaming)**: The real STREAM benchmark [27] traced through gem5, measuring sustainable memory bandwidth under its four vector kernels and providing a baseline for continuous, predictable memory traffic (in-window mix: 3,996,254 reads / 1,998,124 writes - exactly two reads per write, the signature of a write-allocate copy/scale/add/triad mix).
- **alexnet_layer1_ifmap (CNN Read-Storm):** A neural network loading its Input Feature Map. Cycle-accurate traces for this and all subsequent neural network workloads were generated using the SCALE-Sim systolic array simulator **[12]**. **Purpose:** Exposes memory controller queueing delays under massive, parallel read requests (pure-read: 1,269,600 reads, zero writes, delivered as a 60.5 us burst).
- **alexnet_layer1_ofmap (CNN Write-Torture):** A neural network writing its computed activations back to memory. **Purpose:** Explicitly exposes the physical write-latency penalty of Iterative Step-and-Verify (ISPV) MLC programming (pure-write: 135,424 writes, zero reads, delivered as a 4.5 us burst).
- **gpt2_ifmap (Transformer-style Inference):** A General Matrix Multiply (GEMM) read trace representing transformer-style weight streaming. **Purpose:** The maximum-parallelism read test, comparing ReRAM's read-bandwidth behavior directly against modern dual-channel DDR5 (pure-read: 65,536 reads, zero writes, delivered as a 2.2 us burst). Provenance caveat (Section 3.1.6, item 6): unlike the AlexNet traces, whose SCALE-Sim TPU-v1 configuration is confirmed and reproducible, this trace's original generator configuration was never preserved; it is treated as a representative parallel-read stress pattern rather than a validated GPT-2 model trace.

Throughout this subsection the per-trace counts are *trace totals* from each trace’s provenance sidecar; the completed counts used in Section 3 and in Table 5 differ by the one or two requests still in flight at the window edge (for example 1,269,600 against 1,269,599 for AlexNet IFMAP).

Together, the six traces span the three canonical intensity classes: compute-bound with memory mostly idle (GCC), mixed sustained streaming (LBM and STREAM, 57/43 and 67/33 read/write in window), and pure single-operation stress at both extremes - read-intensive (GPT-2, AlexNet IFMAP) and write-intensive (AlexNet OFMAP). The distinction that matters for every rate in Section 3.1.4 is duration: the three gem5 traces run continuously for longer than the 250 ms replay window, while the three SCALE-Sim traces are microsecond bursts that the replay window contains many thousand times over. A per-second figure from a gem5 trace is a sustained rate; the same figure from an AI trace is that burst spread over the window, and is marked burst-derived wherever it appears. Under the matched-host replay of Section 3.1.6, item 11, every technology admits the identical request population for every trace.

3. Results: Multi-Rank Architecture & Scaling

3.1. Granular Workload Diagnostics (Latency & Power Bar Charts)

To fully understand the system-level tradeoffs of memristor-based main memory, I isolated the physical variables of latency and power across distinct workload profiles. The evaluation is organized around the canonical ReRAM trade-off profile - slower access than DRAM, zero refresh, higher density, and individually expensive write operations - and quantifies each axis in turn: latency in Section 3.1.1, the refresh-free but leakage-bound power identity (including where the write-energy cost actually surfaces) in Section 3.1.2, their balance in Section 3.1.3, write endurance in Section 3.1.4, and density in Section 3.3.

One framing note before the numbers, so the trust boundary is explicit from the outset: every figure in this chapter passed through a simulation-fidelity audit of the NVSim-to-NVMain toolchain that found **fourteen** silent failure modes, of which **thirteen** were repaired - each repair validated against device-level anchors or exact predicted arithmetic, and the affected simulations re-run - while **one** remains as a disclosed permanent limitation rather than a bug: one AI trace cannot be attributed to a preserved generator configuration. The simulator's idle-power gating, disabled in source when the audit began, is restored for every technology: DDR5 realizes real savings from it, while ReRAM's power-down energy is a disclosed no-savings placeholder, so no ReRAM power figure claims a gating benefit. The full audit, with the repair arithmetic, is Section 3.1.6; results below cite its item numbers where a specific repair or caveat applies.

3.1.1 Latency Analysis: A Fast Projected Array and its Silicon Counterweight

Latency is where this revision changes the book's story most sharply. At the corrected configurations and the regenerated traces, the modeled ReRAM DIMM is *faster* than the DDR5-4800 baseline on every workload in the suite except the pure write burst, and the operational constraint that dominates instead is the distance between NVSim's projected device timings and anything yet fabricated. To quantify both at the architectural level, I evaluated the memory configurations across compute-bound, streaming, and neural network traces, with two published-silicon configurations replaying the same traces as a permanent counterweight. A methodological note governs how performance is read throughout this section. In fixed-window trace replay the trace dictates the offered load, and after the matched-host correction of Section 3.1.6, item 11, every configuration admits the identical request population per workload: a uniform 250 ms wall-clock window for every technology, cut at the same trace timestamp, with one trace cycle equal to 1/3 ns on the regenerated traces' 3 GHz basis. Under the corrected traces and the corrected configurations, *all six workloads now complete in full on DDR5 and on all four ReRAM tracks*, so every latency average in this section compares identical populations. Two families are still service-limited and are read accordingly. The legacy PCM baseline completes 50.2% of LBM (2,794,480 of 5,564,704) and 46.1% of STREAM. The published-silicon configurations are service-limited almost everywhere, which is the point of including them: the Micron/Sony 1T1R timings complete 18.2% of LBM and the SanDisk/Toshiba 1S1R timings 1.0%.

Two corrections have to be stated before any number in this section is read against the 3 September version. The first, and the largest single correction of this revision, is a *double-count in the read path*: the configuration generator wrote the full NVSim device read latency into *both* tRCD and tCAS, so every read was charged the device twice over. The generator now splits the device read latency across activate and column access so that tRCD + tCAS sums to the device read latency rounded up to the next whole interface cycle (for 1T1R SLC, 4 + 5 cycles at 800 MHz, 11.25 ns against a 10.12 ns device read), and the generator's own validation refuses any configuration in which tRCD + tCAS differs from that rounded-up value, *ceil(device read latency in ns / interface cycle time in ns)*. The second is the trace regeneration of Section 3.1.6, item 6, which removed a fourfold record duplication and moved the replay window out of each benchmark's start-up burst. Both corrections move latency downward, and together they are why the ReRAM latencies below are roughly a third of the ones this book previously reported.

A third change is one of reporting rather than of simulation. NVMain's `averageTotalLatency` measures a request from the moment the memory controller *accepts* it into its queue to the moment it completes, so it cannot see time a request spends waiting to be accepted at all; under a trace that outruns the device, it stays flat while the backlog grows without bound. This book therefore reports a second statistic, *end-to-end latency*, added to NVMain for this revision: it measures from the request's own trace timestamp to its completion, in the same memory-clock domain, and it is the saturation measure. The two definitions are used consistently throughout: **total latency** is queue-entry to completion and is the like-for-like device comparison; **end-to-end latency** is trace-arrival to completion and is the measure of whether a configuration is keeping up with its offered load. Where a configuration is not saturated the two agree to within a fraction of a memory cycle - under GCC, 1T1R SLC posts 41.4 ns total against 40.9 ns end-to-end - and where it is saturated they diverge enormously: the legacy PCM baseline posts 5,136.8 ns total under LBM against 62.2 ms end-to-end, and the SanDisk-timed silicon configuration posts 324.2 us total against 123.6 ms end-to-end. A configuration whose two latencies have separated is not slow; it is failing to keep up.

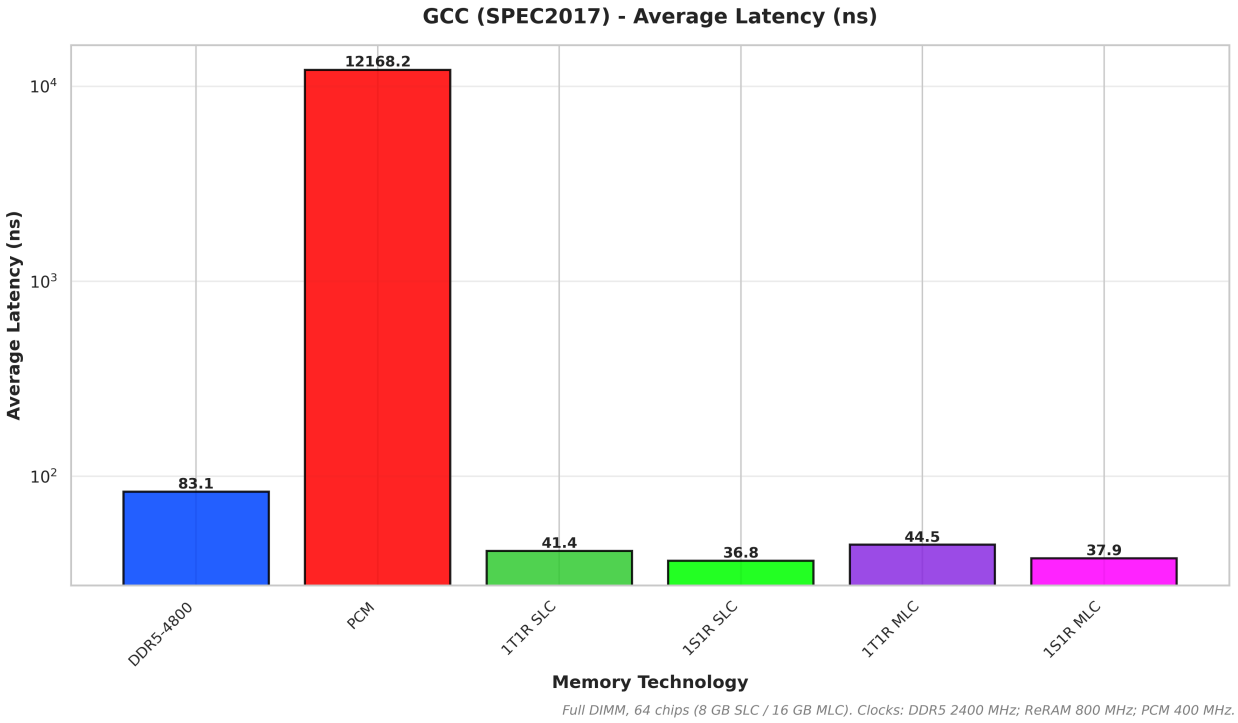


Figure 1: Average Total Latency under the compute-bound GCC (SPEC2017) workload.

Regenerated from the primary dataset of this revision and in agreement with Table 2. Primary figures carry the six primary configurations only; the two published-silicon rows and the 64 B DDR5 cross-check are deliberately excluded from every bar chart and must be read from Table 2 and from Section 2.3.

Under standard compute-bound workloads with small, infrequent memory bursts like gcc (Figure 1), the modeled 22nm ReRAM DIMM is *faster than DDR5-4800*, and the reversal is straightforward arithmetic rather than a surprise. 1T1R SLC averages 41.4 ns and 1S1R SLC 36.8 ns against DDR5's 83.1 ns, an average that includes DDR5's faithfully modeled refresh blocking (Section 3.1.6, item 7), its datasheet-sourced 40-39-39 timing (item 12), its JEDEC write-path and power-down timings (Section 2.3) and the power-down transition cost of the restored idle-gating mechanism (item 5). A DDR5 read that misses an open row pays tRCD plus CAS latency before its first beat - 16.25 plus 16.67 ns at 2400 MHz - and the array behind it is a capacitor that must be sensed and restored. The 2048 x 2048 ReRAM subarray NVSim characterizes reads in 10.12 ns (1T1R) or 4.70 ns (1S1R) end to end, and has no row to precharge.

That array time is the smaller half of the story, and the honest decomposition has to be given before the gap is interpreted. Of the 41.40 ns 1T1R SLC average, 11.25 ns is NVSim device time charged as tRCD + tCAS; the other 30.2 ns is NVMain's default protocol cycle counts (tCWD, tRTP, tXP, tPD, tWTR, tBURST, tCCD) evaluated at this book's assumed 800 MHz interface, which Section 2.3 states has no ReRAM citation behind it. So roughly three quarters of the ReRAM figure is an interface assumption rather than a device measurement, and the same is true

in the other direction of the DDR5 figure, whose protocol timings are JEDEC's. **The direction of the comparison is robust**: ReRAM is below DDR5 at all three interface clocks this book measures (Section 2.3), and it stays below the corrected, slower DDR5 baseline. **Its magnitude is model-dependent**, and a reader should treat 41.4 against 83.1 ns as the outcome of one clock choice, not as a device ratio. On sparse, compute-bound traffic where almost every request is a row miss, the projected array is on a shorter path; how much shorter depends on the interface it is given.

This result must be read with its condition attached, and the condition is large. **It holds at NVSim-projected device timings only.** No fabricated ReRAM part reads in nanoseconds. Replaying the identical trace through the identical module geometry at published fabricated-chip timings gives 11,888.5 ns for the Micron/Sony 16 Gb 1T1R part **[49]** and 378,763.1 ns for the SanDisk/Toshiba 32 Gb 1S1R cross-point **[48]** - 11.9 us and 379 us, three and four orders of magnitude above the projection - and neither completes the LBM window (18.2% and 1.0%). These two rows sit in Table 2 beside the projected ones deliberately. The honest statement of this section's latency result is that *a 22nm ReRAM array core, if it can be built at NVSim's projected timings, beats DDR5 on compute-bound traffic, and nothing yet built comes close*. The gap between those two statements is the engineering problem, and it is far larger than any difference between 1T1R and 1S1R.

Between the two projected cell topologies, the selector is now the *faster* one on reads, not the slower one: NVSim prices a 1S1R read at 4.70 ns against 10.12 ns for 1T1R at the matched organization, and a write at 24.59 against 15.26 ns. Earlier versions of this book reported the opposite ordering on reads and attributed it to sensing through the non-linear selector at the high 10^5 Ω calibration; that ordering was a consequence of characterizing the two cells at different column-mux ratios and does not survive a matched organization (Section 3.1.2). Whether the read advantage translates into system efficiency is a power question, answered in Sections 3.1.2 and 3.1.3.

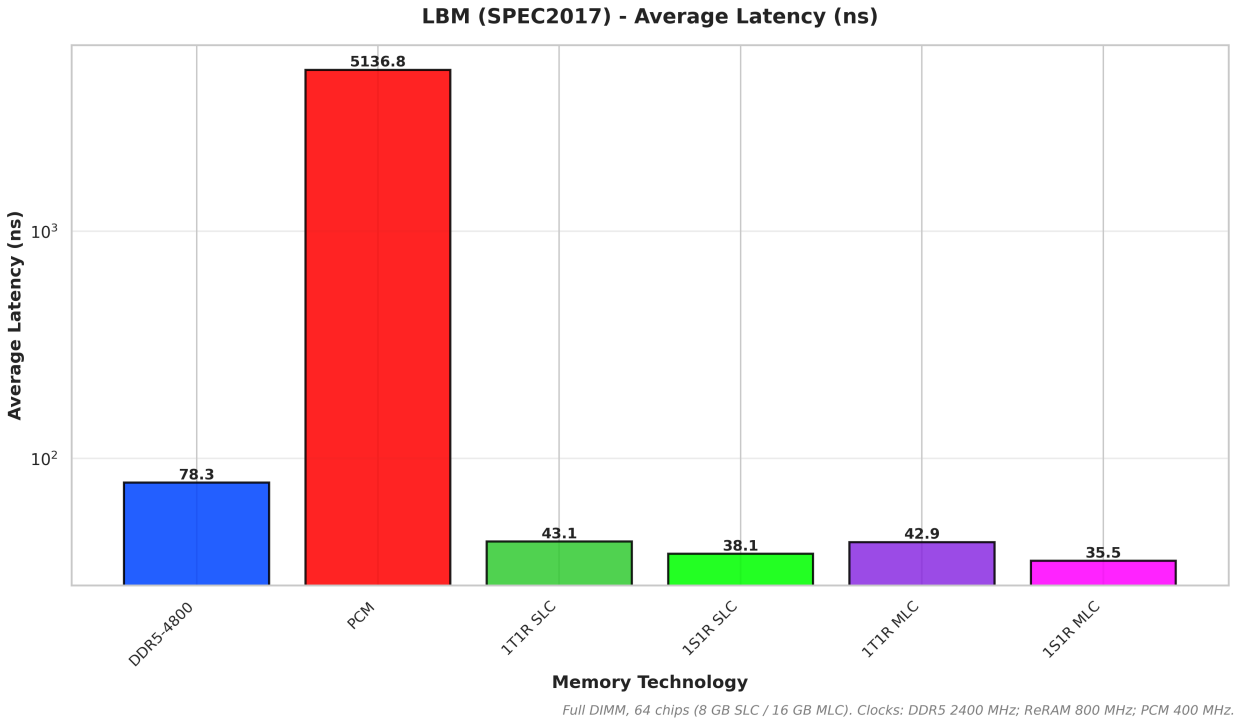


Figure 2: Average Total Latency under the continuous memory-streaming LBM (SPEC2017) workload.

Regenerated from the primary dataset of this revision and in agreement with Table 2. Primary figures carry the six primary configurations only; the two published-silicon rows and the 64 B DDR5 cross-check are deliberately excluded from every bar chart and must be read from Table 2 and from Section 2.3.

To contrast the compute-bound nature of GCC, I evaluated the memory subsystem under the lbm trace (Figure 2), a fluid dynamics workload characterized by heavy, continuous memory streaming at roughly eleven times GCC's request density. The ordering does not change: 1T1R SLC averages 43.1 ns and 1S1R SLC 38.1 ns against DDR5's 78.3 ns, and *every* ReRAM track completes the entire 250 ms window, 5,564,704 of 5,564,704 requests, exactly as DDR5 does. The dramatic completion shortfalls this book previously reported under LBM (Appendix D) were an artifact of the old trace: it duplicated every record fourfold and its window sat inside LBM's start-up burst, offering the device roughly four times the request rate the benchmark actually sustains. Against the regenerated trace, at 9.54 million writes per second sustained, none of the four ReRAM tracks saturates. What this workload does still expose is where the resistive technologies fail: the legacy PCM baseline completes 50.2% and the two published-silicon configurations 18.2% and 1.0%, and their end-to-end latencies (62.2 ms, 102.3 ms and 123.6 ms) show the backlog rather than the device.

The MLC verdict also changes, and it changes because of a *modeling choice made in this project's configuration generator*, not because of new device physics and not because of anything NVMain does on its own. Under LBM the MLC tracks are not slower than their SLC

siblings at all (1T1R MLC 42.9 ns against SLC 43.1; 1S1R MLC 35.5 against 38.1), and under GCC they are slower by only about 3 ns and 1 ns respectively. The reason is that this project's generator books NVSim's write latency as *tWR*, the write recovery time, and leaves NVMain's write-pulse parameter tWP at its default of zero. **Booked as tWR, the cell write time reaches the request path only when a write closes its row** (Section 2.3), and which requests it delays depends on the branch the write takes. Under the close-page policy these configurations use, a write that is the last queued request to its row becomes a write-with-precharge and NVMain adds tWR to that subarray's next activation, maxing the subarray's next precharge, next read and next write against it, so the next access to the subarray waits for the cell. A write followed by another queued request to the same open row charges nothing beyond the burst and the write-to-read turnaround. In neither branch does the cell write time delay the write's own completion event, and in both the unit that waits is the write's own subarray, not the whole bank. (The precondition is NVMain's `UniformWrites` setting, which defaults to true and which no generated configuration overrides: on that branch the write pulse is tWP itself. The other branch would charge tWP0 and tWP1, whose defaults are 40 and 60 cycles, and those *would* be on the request path.) With tWP zero, a write retires after tCWD + tBURST = 13.75 ns on *every* track, so the cell write pulse that distinguishes MLC from SLC (49.79 against 15.26 ns for 1T1R, 80.23 against 24.59 ns for 1S1R, Table 1) never reaches the write's own completion, and reaches the next request only through the row-closing branch. **The MLC latencies in Table 2 are therefore lower bounds**, and “MLC is barely slower than SLC” is a consequence of this book's own mapping rather than a device finding: with the write pulse on the request path a 1T1R MLC write would cost about 63 ns instead of 13.75, and a 1T1R SLC write about 29 ns. The subsection below measures that bound instead of asserting it, by re-running the whole primary matrix with the pulse moved onto the write's own completion. What the tWR mapping does capture is subarray occupancy, and at the request rates five of these six traces offer, the subarrays have enough idle time to absorb it. The MLC write penalty surfaces instead as endurance and as energy per write, which is where Sections 3.1.4 and 3.1.2 pick it up. The one place it surfaces as latency even under this mapping is the write-dominated AlexNet OFMAP trace below, where the offered write rate is high enough to leave the banks busy.

The write pulse on the request path: a bounding run

Because the lower bound above is a modeling choice and not a device fact, the whole primary matrix was re-run with that choice inverted, so that the size of the bound is measured rather than argued. The bounding run is invoked as `mbmm_master.py --write-pulse-on-path`. The flag changes exactly one thing in every generated ReRAM and published-silicon configuration: the NVSim cell write latency, which the default configurations book as tWR, is written to *tWP* instead, and tWR is held at one cycle. tWP is what NVMain's `WriteCellData` returns and therefore becomes the write timer, so in bounding mode *every* write's own completion carries the full cell pulse, and an open-row write charges that pulse to its subarray's next read and next write as well; after a row-closing write the subarray's next activation is unchanged to within one cycle. Everything else is identical to the primary matrix: the same six traces, the same 250 ms matched window, the same 32-entry queue, the same two channels, the same forced 2048 x 2048

organization at mux 64 and the same 800 MHz interface. The flag reaches the generated configurations only, so the DDR5 and PCM baselines are bit-identical runs in the two datasets. The bounding dataset is `results/system_rev2026-09_writepulse/`; the default dataset is the primary one used throughout this book. In interface cycles at 800 MHz the pulse carried by tWP is 13 cycles (16.25 ns) for 1T1R SLC, 40 (50.00 ns) for 1T1R MLC, 20 (25.00 ns) for 1S1R SLC, 65 (81.25 ns) for 1S1R MLC, 9,360 (11,700.00 ns) for the Micron 1T1R silicon track and 184,000 (230,000.00 ns) for the SanDisk 1S1R silicon track.

The two all-read traces are the control. GPT-2 IFMAP and AlexNet layer 1 IFMAP contain 65,536 and 1,269,600 reads and zero writes, and every one of their rows is byte-identical between the two datasets on every column, as is every row-buffer counter behind them. A flag on the write path changed nothing on a read-only path, which is the check that the two datasets differ only where they should.

Track	Trace	Latency, default (ns)	Latency, bounding (ns)	End-to-end, default (ns)	End-to-end, bounding (ns)
1T1R SLC	GCC	41.4014	48.3061	40.8826	47.7876
1T1R SLC	LBM	43.1218	50.0481	42.6584	49.5914
1T1R SLC	STREAM	42.2586	47.6758	41.7817	47.1989
1T1R SLC	AlexNet OFMAP	353.1069	696.6112	486,102.2	967,674.3
1T1R MLC	GCC	44.5279	65.7738	44.0151	65.2613
1T1R MLC	LBM	42.9154	64.3551	42.4542	63.8937
1T1R MLC	STREAM	46.5913	63.2562	46.1372	62.7921
1T1R MLC	AlexNet OFMAP	512.0013	1,649.9	780,165.4	2,448,063.9
1S1R SLC	GCC	36.7661	47.3896	36.2411	46.8652
1S1R SLC	LBM	38.0776	48.7699	37.5936	48.2929
1S1R SLC	STREAM	37.3001	45.6301	36.7865	45.1264
1S1R SLC	AlexNet OFMAP	391.4819	916.5738	543,242.2	1,280,376.4
1S1R MLC	GCC	37.8691	72.3981	37.3473	71.8756
1S1R MLC	LBM	35.4753	70.3857	35.0007	69.9167
1S1R MLC	STREAM	36.9376	63.9827	36.4709	63.5069
1S1R MLC	AlexNet OFMAP	652.9494	2,498.7	1,010,676.2	3,715,858.3
1T1R silicon	GCC	11,888.5	36,434.9	140,649.9	13,110,313.5
1T1R silicon	LBM	17,349.5	63,192.1	102,280,812.5	118,391,062.5
1T1R silicon	STREAM	27,802.3	131,153.8	111,315,375.0	121,740,062.5
1T1R silicon	AlexNet OFMAP	80,066.6	280,795.6	119,923,821.4	129,116,498.7
1S1R silicon	GCC	378,763.1	679,018.1	111,037,147.3	112,450,122.5
1S1R silicon	LBM	324,174.4	1,223,948.8	123,635,437.1	123,145,871.9
1S1R silicon	STREAM	530,595.6	2,487,218.8	123,697,181.0	123,911,319.6
1S1R silicon	AlexNet OFMAP	1,092,759.4	3,891,687.5	131,664,759.2	125,236,250

Table 9: The write-pulse bracket - average total latency and end-to-end latency, full-DIMM configurations, default mode against bounding mode. *Default values are the `Latency_ns` and `E2E_Latency_ns` columns of results/rev2026-09_primary_csv/processed_bar_chart_metrics.csv; bounding values are the same two columns of results/system_rev2026-09_writepulse/_csv/processed_bar_chart_metrics.csv, in both cases the `full_dimm` rows. The two all-read traces are*

omitted because they are identical in the two datasets. These are media latencies at an assumed 800 MHz interface, not device latencies of any shipped part.

Three derived readings follow, and all three are *derived* from the two columns of Table 9 rather than read from a column of their own. *MLC against SLC on GCC:* the ratio of the MLC track's average to its SLC sibling's is 1.076 for 1T1R and 1.030 for 1S1R in default mode, and 1.362 and 1.528 in bounding mode. On the AlexNet output map the same pairs give 1.450 and 1.668 against 2.368 and 2.726. The corrected statement is therefore that **MLC costs up to 1.53x SLC on CPU traffic and up to 2.73x on a write burst**, where the lower end of each range is this book's default mapping and the upper end is the bounding run. *ReRAM against DDR5 on the write burst:* the single ratios quoted below become brackets, taken against the same DDR5 average of 234.2819 ns (identical in the two datasets, because the flag never reaches the static DDR5 configuration) - **1.507x to 2.973x** against 1T1R SLC, **1.671x to 3.912x** against 1S1R SLC, **2.185x to 7.042x** against 1T1R MLC and **2.787x to 10.665x** against 1S1R MLC. *The gap on GCC:* the bounding run adds 6.90 ns to 1T1R SLC (+16.7%), 21.25 ns to 1T1R MLC (+47.7%), 10.62 ns to 1S1R SLC (+28.9%) and 34.53 ns to 1S1R MLC (+91.2%).

What produced the bracket is not the same thing on the CPU traces as on the write burst, and the difference is the whole interpretation. On GCC, LBM and STREAM the rise is the pure write-completion term and essentially nothing else: the write fraction of the completed requests times the cell pulse predicts the measured rise to within 1% on every one of those twelve track-trace pairs. That is what NVMain's own row-buffer counters predict, because those traces have almost no row hits at all - 0.026% under GCC for 1T1R SLC, 0.00002% under LBM (one hit in 5,564,704 accesses) and exactly zero under STREAM - so nearly every write closed its row and default mode had already charged the cell to the subarray's next activation. On the AlexNet output map the picture inverts. That trace is pure writes, and **76% of its accesses hit an open row**, the branch on which default mode charged the subarray nothing at all; the measured rise there is **21 to 23 times** the per-write pulse (21.1x for 1T1R SLC, 22.8x for 1T1R MLC, 21.0x for 1S1R SLC and 22.7x for 1S1R MLC). The headline write-burst bracket is therefore mostly *subarray blocking that the default mode never charged*, not the write's own completion latency, and a reader who takes it as “the cell pulse added to every write” will be wrong by a factor of about 22.

Four statements bound what this run does and does not establish. **First, the bracket is an upper end of the write-path cost inside this model, not a prediction of a device.** Bounding mode charges the full NVSim cell pulse to every write's completion with no write-combining, no buffering and no verify-loop structure, which is the pessimistic extreme of the same mapping question, just as the default is the optimistic one. **Second, “below DDR5 on the CPU traces” survives at the upper end.** In bounding mode all four projected tracks remain faster than the DDR5-4800 baseline on GCC, LBM and STREAM; the worst case in the whole matrix is 1S1R MLC under GCC at 72.398 ns against DDR5's 83.111 ns, which is 0.871x of DDR5 (derived). The latency headline of this book is therefore not a consequence of where the write pulse was booked. **Third, no endurance number moves.** `Wear_Max_Writes` is identical between the two

datasets on every trace for all four projected tracks, as is `Completed_Requests`: the same traces present the same writes, and the mode changes only when they are served. Power is likewise unchanged to six decimals on GCC, LBM, STREAM and the two read traces, and falls 0.9% to 2.4% on the output map because the same requests are spread over more time; the Power-Delay Product, being the product of the two, becomes a bracket wherever latency does. **Fourth, the two published-silicon tracks are exempt from the third statement, and their end-to-end column must be read with the window in mind.** At tWP of 9,360 and 184,000 cycles the fixed 250 ms window drains far fewer requests in bounding mode, so their `Completed_Requests` and `Wear_Max_Writes` both fall (1T1R silicon on the output map, 135,423 requests to 41,586; 1S1R silicon, 7,510 to 1,106). That is the service limitation Table 4 already warns about, not an endurance finding, and it is also why the 1S1R silicon end-to-end average on the output map *falls* from 131.7 ms to 125.2 ms in bounding mode: the surviving population is a seventh of the size and the requests that never completed are the ones that would have carried the long tail. That single cell is a survivorship artifact of the fixed window and not a speed-up.

The literature the bracket should be read against. Published program-and-verify data on 1T1R RRAM reports an average of 49 pulses per level move, median 36 **[55]**, and a research-setting self-terminating single-pulse MLC write technique is reported to cut that count substantially, though it is used in no shipped part **[56]**. The closest published two-bit *array* result is MLC phase-change memory: more than 70 program and verify iterations for a total write of 9.7 us, or under 1.6 us only if the verify step is skipped **[57]**. Against that, this book's own MLC write multiplier of x3.263 on the NVSim cell time (Appendix A) understates the media-level MLC write cost by one to two orders of magnitude, and the bounding run does not close that gap either: it moves this book's own pulse onto the request path, it does not replace the pulse with a verify loop. The single-level end of the same literature is much faster - an 8 Mb multi-layered cross-point ReRAM macro reports a 17.2 ns write cycle at 443 MB/s **[58]**, and a 128 Gb two-deck cross-point part reports a set time at or below 300 ns and a read time at or below 100 ns in a 16 Mb array **[59]** - which is the spread a single multiplier cannot represent. On the read side the honest statement is an absence: **no primary source pairing SLC and MLC array-level read times in the same study was found.** MLC read is directionally slower because it needs multi-reference sensing, but this book's x1.5 read multiplier has no array-level pair behind it, only the device-level sources of Appendix A. **TLC and QLC carry no defensible number at all.** TLC has been demonstrated only at the single-cell level and never in an array memory product, and QLC appears only in non-memory contexts such as analog neuromorphic weights or high-resistance-state-only readout. The only supportable statement is a qualitative trend - verify iterations grow, the per-level sensing window shrinks and retention worsens as more levels are packed into one cell - and no number should be attached to it. That is why this book models SLC and MLC only, and why the density-per-bit dividend of more levels is quoted in Section 3.3 as area and never as latency.

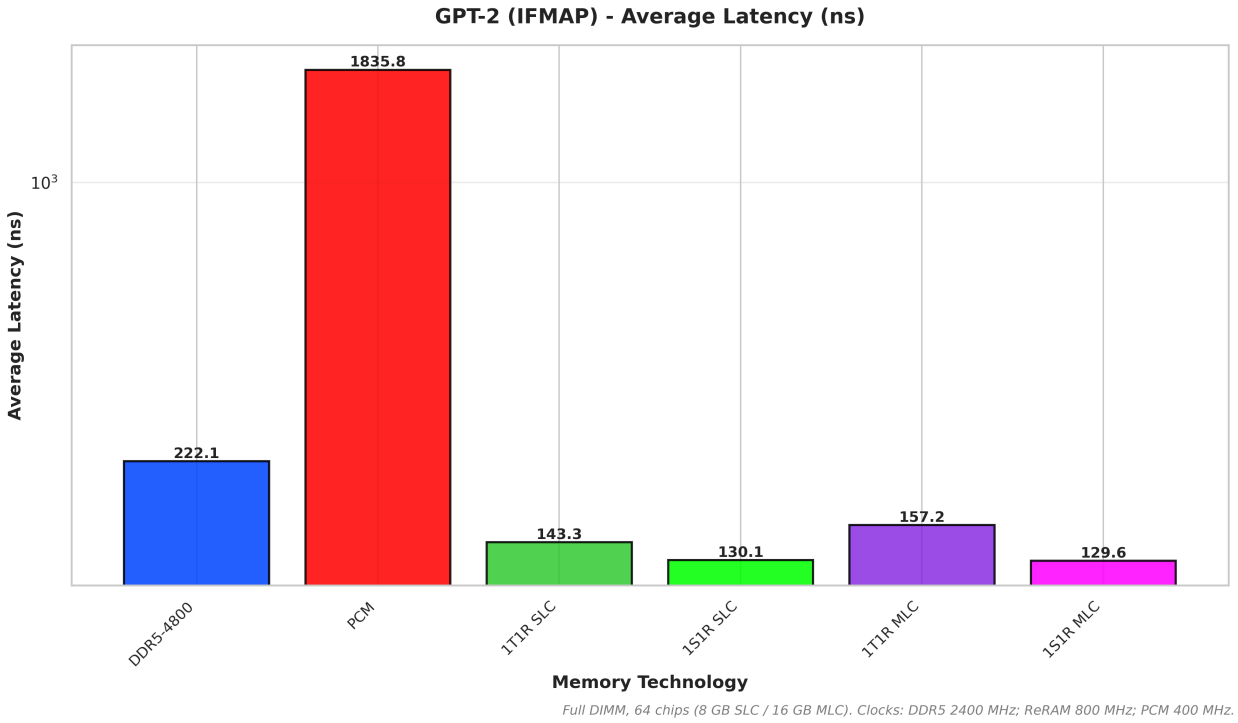


Figure 3: Average Total Latency under the parallel read-storm of GPT-2 Inference (IFMAP).

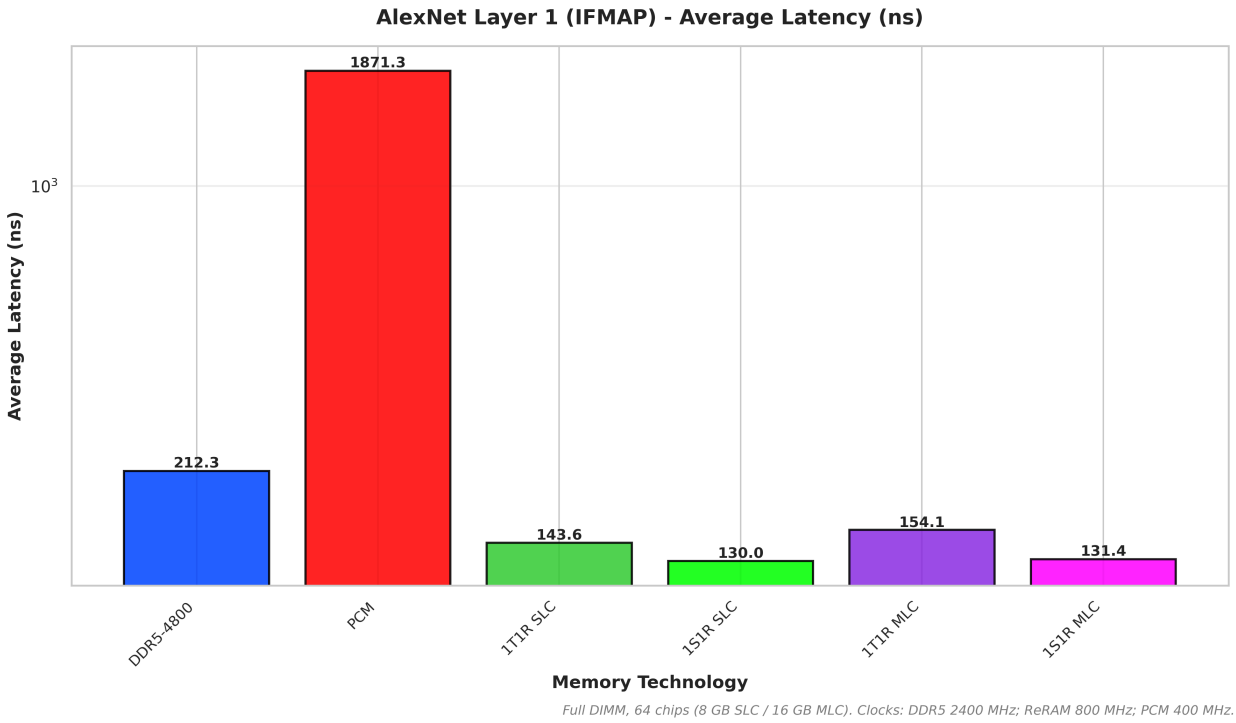
Regenerated from the primary dataset of this revision and in agreement with Table 2. Primary figures carry the six primary configurations only; the two published-silicon rows and the 64 B DDR5 cross-check are deliberately excluded from every bar chart and must be read from Table 2 and from Section 2.3.

When the system is subjected to the parallel read-storms typical of Large Language Model inference, the dynamics shift, but not in the direction this book previously reported. In the GPT-2 trace (Figure 3), 1T1R SLC averages 143.3 ns and 1S1R SLC 130.1 ns against DDR5's 222.1 ns, and all three complete the trace in full. The 4.6x AI-inference deficit reported in earlier versions of this book is gone, and its two causes are both now understood: the read double-count inflated every ReRAM latency, and the old AI traces undercounted addresses tenfold, which changed the shape of the offered burst.

What this trace does expose, for every technology alike, is that it is a *queueing* measurement and not a device measurement. GPT-2 IFMAP is a 2.2 us burst of 65,536 reads presented to a memory system with a 32-entry controller queue, so what dominates is how fast the queue drains, not how fast the array reads. The end-to-end statistic makes that explicit: against total latencies of 130 to 222 ns, end-to-end latencies are 97.9 us (1S1R SLC), 106.3 us (1T1R SLC) and 139.1 us (DDR5). Every one of these configurations spends three orders of magnitude more time waiting for admission than being served. The ordering among them is real, and it favors ReRAM here because the higher per-request service rate drains the burst sooner, but the honest reading is that no conclusion about sustained AI-inference serving can be drawn from a

microsecond single-layer capture. A representative full-model trace remains future work (Section 4.1).

That the queue is the binding term in this regime is not an inference; it was swept. The primary matrix runs a 32-entry controller queue, and the depth was re-run at 8 and at 128 on GCC, LBM and the AlexNet output-map burst. **On the two CPU traces the depth makes no difference at all**: 1T1R SLC averages 41.4 ns under GCC and 43.1 ns under LBM at depth 8, 32 and 128 alike, to the last digit the CSV carries, because at 2.1 and 22.3 million requests per second (519,568 and 5,564,704 admitted over the 250 ms window) the queue never fills. **On the write-dominated burst the two latency measures move in opposite directions.** For 1T1R SLC the in-controller average goes 124.9 ns (depth 8), 353.1 ns (32), 1,049.2 ns (128) while the end-to-end average goes 680.1 µs, 486.1 µs, 432.5 µs; 1S1R SLC follows the same shape (135.7 / 391.5 / 1,161.6 ns against 753.0 / 543.2 / 479.0 µs), as do both MLC tracks. A deeper queue therefore *raises* the classic latency metric and *lowers* the time the workload actually waits, because a short queue reports a fast average over the few requests it admits while the rest of the burst backs up outside the controller where the total-latency counter cannot see it. This is the argument for end-to-end latency as the saturation measure, stated in the one experiment that separates the two cleanly: the metric that improves with a shallower queue is the metric that is measuring the wrong thing. Two scoping notes. The depth flag reaches the ReRAM configurations only, not the DDR5 or the PCM baseline, whose queues stay at their own configured depths throughout the sweep; and every configuration completes the identical request population at every depth, so none of this is a completion effect.



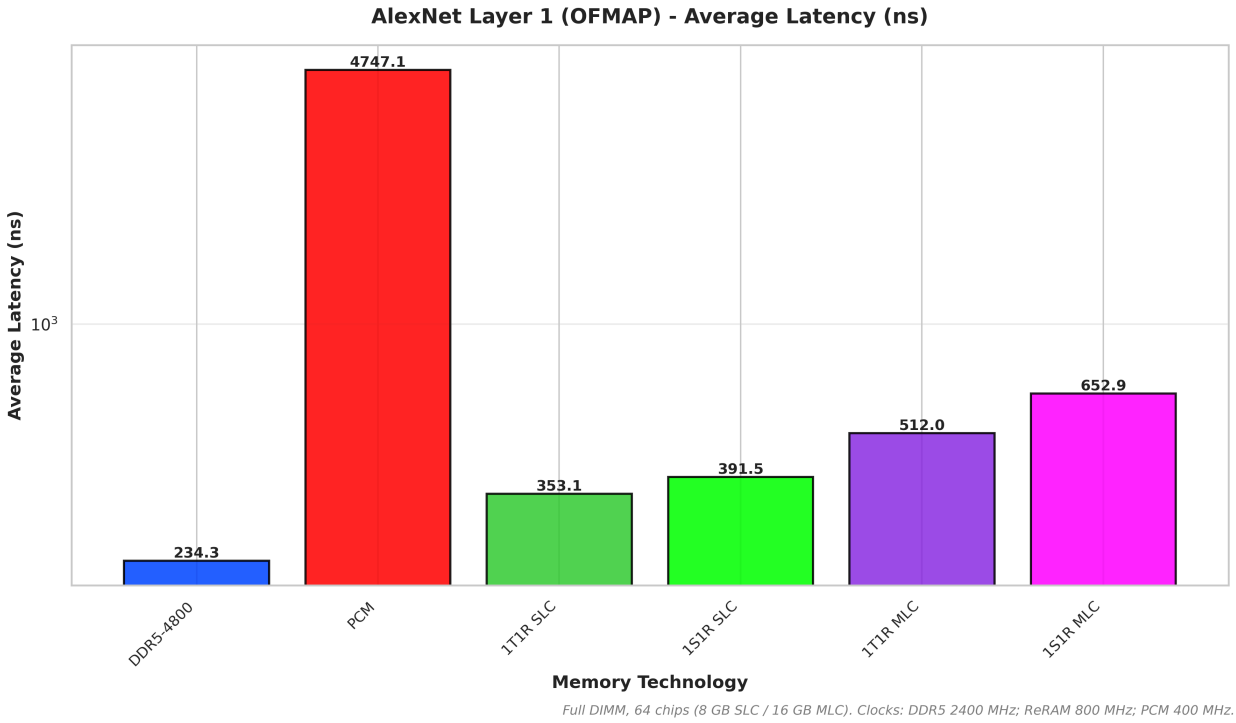


Figure 4 & 5: Average Total Latency under AlexNet Layer 1 IFMAP (pure-read) and OFMAP (pure-write) workloads, highlighting the MLC write-latency penalty.

Regenerated from the primary dataset of this revision and in agreement with Table 2. Primary figures carry the six primary configurations only; the two published-silicon rows and the 64 B DDR5 cross-check are deliberately excluded from every bar chart and must be read from Table 2 and from Section 2.3.

To explicitly isolate the write penalty, I compared the Input Feature Map (IFMAP) and Output Feature Map (OFMAP) traces of the same AlexNet convolution layer **(Figures 4 and 5)**. This is the one regime in the suite where ReRAM is *slower than DDR5*, and it is the regime the technology has always been expected to lose. Under the read-heavy IFMAP burst, 1T1R SLC averages 143.6 ns and 1S1R SLC 130.0 ns against DDR5's 212.3 ns. Under the write-heavy OFMAP burst the ordering inverts: DDR5 averages 234.3 ns while 1T1R SLC pays 353.1 ns, 1S1R SLC 391.5 ns, 1T1R MLC 512.0 ns and 1S1R MLC 652.9 ns. ReRAM loses this regime, and it loses it by a margin that is itself a bracket rather than a single number, because the write pulse can be booked at either end of the mapping question above: DDR5 is **1.507x to 2.973x** faster than 1T1R SLC here and **1.671x to 3.912x** faster than 1S1R SLC, rising to **2.185x to 7.042x** and **2.787x to 10.665x** against the two MLC tracks, the lower end of each range being this book's default mapping and the upper end the bounding run of the subsection above. DRAM writes and reads at nearly the same cost; a resistive cell does not, and under a pure write stream dense enough to keep the banks busy there is nothing left to hide the difference behind. Two bounds go with that statement and they point in opposite directions. The ReRAM side is a *lower* bound on the deficit, because in this model the cell write time is booked as write recovery and so

reaches the request path only when a write closes its row, never delaying the write's own completion (above); the bounding run of the subsection above supplies the matching upper end, which is where the second figure in each bracket comes from. The DDR5 side now carries JEDEC write recovery and write latency rather than the DDR3-template cycle counts earlier versions of this configuration used, which is why the deficit reported here is smaller than the one this book quoted before the correction of Section 2.3.

The pair also yields the suite's cleanest read-versus-write comparison at the system level, because the two traces isolate pure operation streams - IFMAP issues only reads (1,269,600 requests), OFMAP only writes (135,424) - and every technology replays the identical pair. The write-side degradation factor (OFMAP over IFMAP average total latency) ranks write tolerance directly: DDR5 degrades 1.10x, that is, it is marginally *slower* on the write stream once its JEDEC write recovery and write latency are modeled (Section 2.3), while 1T1R SLC degrades 2.5x, 1S1R SLC 3.0x, 1T1R MLC 3.3x and 1S1R MLC 5.0x. Two device-level facts from Table 1 sit beneath that ranking. The selector prices writes intrinsically: a 1S1R write costs 5.2x its own read at the device (24.59 against 4.70 ns) before Iterative Step-and-Verify programming stretches it to 80.23 ns for MLC. The access transistor makes the 1T1R device far more symmetric (15.26 against 10.12 ns), which is why 1T1R SLC is the most write-tolerant ReRAM configuration in the suite even though it is not the fastest reader. This ordering, not the LBM completion figures of earlier versions, is what confines MLC to read-dominant roles, and it is the asymmetry that controller-level write buffering and write combining (Section 4.1) would attack.

The STREAM benchmark (Figure 6) applies real vector-kernel bandwidth pressure at two reads per write. 1T1R SLC averages 42.3 ns and 1S1R SLC 37.3 ns against DDR5's 79.4 ns, all three completing the whole window at the 1,534.6 MB/s the trace offers. Together with the LBM result, sustained streaming is no longer a latency-deficit regime for projected ReRAM at all; it is the regime where the array's shorter, precharge-free read path shows up most consistently. It remains the regime that decides endurance, which is where its real constraint lies (Section 3.1.4).

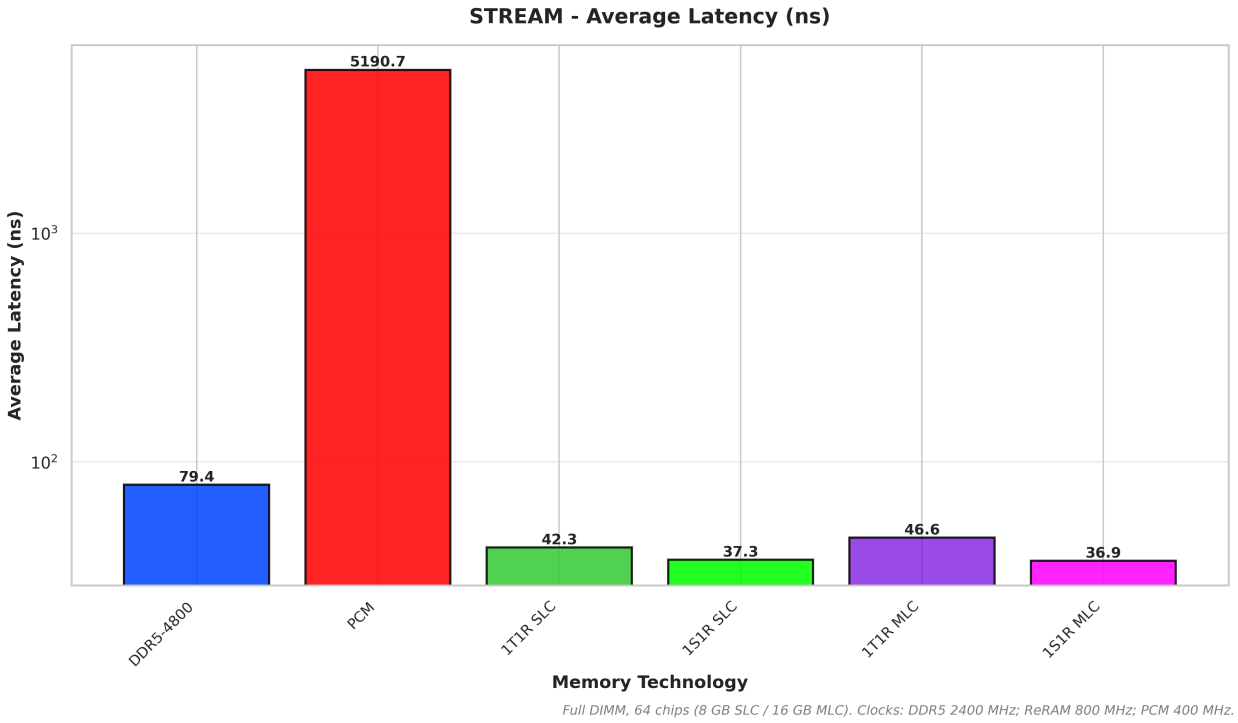


Figure 6: Average Total Latency under the STREAM memory bandwidth benchmark.

Regenerated from the primary dataset of this revision and in agreement with Table 2. Primary figures carry the six primary configurations only; the two published-silicon rows and the 64 B DDR5 cross-check are deliberately excluded from every bar chart and must be read from Table 2 and from Section 2.3.

Table 2 collects the average total latencies across the full workload suite, complementing the per-workload bar charts. The two published-silicon rows sit in the same table deliberately: they are the counterweight to every projected row above them, and no reading of this table is complete without them.

Table 2: Average total latency (ns), full-DIMM configurations, 250 ms matched window.

Technology	GCC	LBM	STREAM	GPT-2	AlexNet IFMAP	AlexNet OFMAP
DDR5-4800	83.1	78.3	79.4	222.1	212.3	234.3
1T1R SLC	41.4	43.1	42.3	143.3	143.6	353.1
1S1R SLC	36.8	38.1	37.3	130.1	130.0	391.5
1T1R MLC	44.5	42.9	46.6	157.2	154.1	512.0
1S1R MLC	37.9	35.5	36.9	129.6	131.4	652.9
PCM (legacy)	12,168.2	5,136.8	5,190.7	1,835.8	1,871.3	4,747.1
1T1R SILICON	11,888.5	17,349.5	27,802.3	10,934.1	10,758.9	80,066.6

Technology	GCC	LBM	STREAM	GPT-2	AlexNet IFMAP	AlexNet OFMAP
1S1R SILICON	378,763.1	324,174.4	530,595.6	193,995.8	193,699.1	1,092,759.4

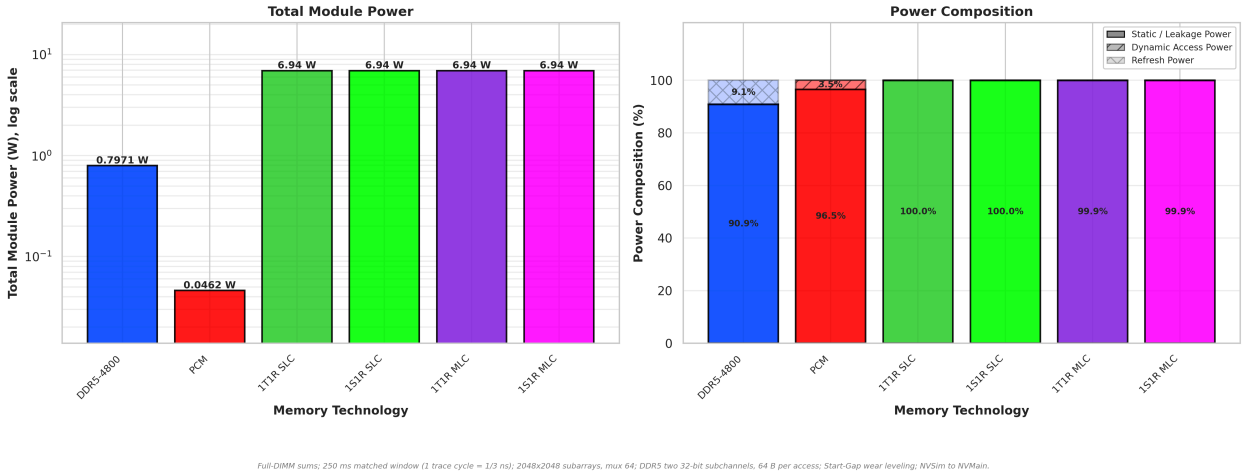
Average total latency, wall-clock nanoseconds: queue entry to completion, the like-for-like device comparison (clock domains: ReRAM 800 MHz, PCM 400 MHz, DDR5 2400 MHz). The first five rows are NVSim-projected device timings at the matched 2048 x 2048 organization; ***1T1R SILICON*** *and* ***1S1R SILICON*** *replay the identical traces through the identical module geometry at published fabricated-chip timings (Micron/Sony 27nm 16 Gb 1T1R, 2.3 us read / 11.7 us write [49]; SanDisk/Toshiba 24nm 32 Gb 1S1R, 40 us / 230 us [48]).* ***Completion:*** *DDR5 and all four projected ReRAM tracks complete 100% of every workload. PCM completes 50.2% of LBM and 46.1% of STREAM; 1T1R SILICON completes 18.2% of LBM, 9.2% of STREAM and 76.7% of AlexNet IFMAP, and the rest in full; 1S1R SILICON completes 9.2% of GCC, 1.0% of LBM, 0.5% of STREAM, 4.5% of AlexNet IFMAP, 5.5% of AlexNet OFMAP and 89.1% of GPT-2. A latency average over a service-limited run covers only that run's completed prefix, which is why end-to-end latency (Table note below) and not total latency is the saturation measure. Source: results/rev2026-09_primary_csv/processed_bar_chart_metrics.csv, column Latency_ns.*

The same matrix read through the end-to-end statistic separates the saturated configurations cleanly. Under GCC, where nothing saturates, end-to-end and total latency agree to a fraction of a memory cycle for every technology (DDR5 83.0 against 83.1 ns; 1T1R SLC 40.9 against 41.4). Under LBM, where DDR5 and all four ReRAM tracks still keep up, they continue to agree (DDR5 78.1 against 78.3; 1S1R SLC 37.6 against 38.1) while the three service-limited families separate by six orders of magnitude: PCM 62.2 ms, 1T1R SILICON 102.3 ms and 1S1R SILICON 123.6 ms end-to-end, against total latencies of 5.1, 17.3 and 324.2 us. Under the three AI bursts every technology is queue-bound, so every end-to-end figure is large (GPT-2: DDR5 139.1 us, 1T1R SLC 106.3 us, 1S1R SLC 97.9 us) and the ranking reflects how quickly each drains a burst it was never going to absorb at line rate.

3.1.2 Power Analysis: Static Leakage Dominance

While ReRAM exhibits latency penalties under write-heavy and massive parallel read workloads, its candidate advantage has always been the energy profile: traditional DRAM requires continuous, power-intensive capacitive refresh cycles to maintain data integrity - a permanent power tax paid regardless of workload activity - whereas non-volatile retention is free. Whether ReRAM actually realizes that advantage, however, is decided by peripheral leakage, and quantifying leakage correctly required repairing the simulation toolchain first. All power figures in this section therefore come from the repaired model documented in Section 3.1.6: NVSim's per-technology leakage is wired into the standby-energy parameters NVMain actually reads (validated against device-level anchors to 0.0% error), power is accounted as the whole-module sum for every technology, and NVMain's restored idle-gating mechanism is live

for every technology (Section 3.1.6, item 5). Its effect is not symmetric, and that asymmetry is restated wherever cross-technology totals are compared: DDR5 realizes real, JEDEC-datasheet-backed power-down savings, ReRAM's power-down energy is a disclosed placeholder equal to its standby energy (so no ReRAM figure claims a gating benefit), and PCM shows no power-down activity at all.



Full-DIMM sums; 250 ms matched window (1 trace cycle = 1/3 ns); 2048x2048 subarrays, mux 64; DDR5 two 32-bit subchannels, 64 B per access; Start-Gap wear leveling; NVSim to NVMain.

Figure 7: System Power Breakdown (Static / Dynamic / Refresh) under the compute-bound GCC (SPEC2017) workload.

Regenerated from the primary dataset of this revision and in agreement with Table 3: all four ReRAM tracks sit on one shared leakage floor near 6.94 W for 8 GiB (SLC) or 16 GiB (MLC), against a 16 GiB DDR5 module. The chart is a module-level view; the per-gigabyte normalization, which is the only form in which these modules are comparable, exists only in Table 3.

Power is the axis on which this revision reverses the book's previous conclusion outright, so the correction is stated first and the numbers follow. **There is no transistor-versus-selector leakage gap.** The two-orders-of-magnitude device-level separation that the 3 September version of this book made its deciding architectural fact (superseded figures in Appendix D) was an artifact of simulating the two cells at different array organizations, not a property of the access device. NVSim assigns memristor cells zero leakage for every access type; chip leakage is entirely peripheral circuitry, so it scales with the number of mats, and the two configurations had been given hand-set sense-amp column muxes of 32 and 256, which drove NVSim to 256 mats for 1T1R against 2 for 1S1R. At the matched 2048 x 2048 organization at mux 64 used throughout this revision, NVSim reports **108.384 mW per 1 Gb chip for both cell types**, identical to three decimal places, and every figure below follows from that.

In instruction-heavy, compute-bound workloads like gcc (Figure 7), the main memory subsystem remains idle for the vast majority of the execution time, making GCC the cleanest probe of each technology's standing power floor. The non-volatile opportunity shows immediately in one component: ReRAM's refresh power is identically zero, because retention is free, while 9.1% of

DDR5's 0.797 W module draw under GCC is refresh, energy spent merely retaining data, on top of a 90.9% standby/power-down floor, with actual access activity accounting for 0.02%. Across the whole suite DDR5 spends 6.5-9.1% of its module power on refresh it can never shed. But the component comparison does not decide the module comparison, and here it does not even come close. The full ReRAM DIMM draws **6.94 W under GCC, 99.96% of it static** - 64 chips at 108.384 mW is 6.937 W of leakage before a single request is served - against 0.797 W for the DDR5 baseline.

Those two numbers describe modules of different capacity, so the comparison that means anything is per gigabyte. The ReRAM DIMM is 8 GiB and the DDR5 module 16 GiB (Section 2.3), which makes the honest figures 0.867 W/GiB against 0.0498 W/GiB under GCC: **ReRAM draws about 17.4x DDR5's power per gigabyte**, and 12.7x to 17.4x across the six workloads for the SLC tracks, 6.4x to 8.7x for MLC.

Both of those module figures are memory-device-only. Neither side carries a register clock driver, a power-management IC, a PHY or its termination, an SPD hub, or - on the ReRAM side - the on-DIMM controller such a module would need; NVMain models memory devices and nothing around them. A real DDR5 RDIMM draws watts rather than the 0.797 W counted here, and any fixed per-module overhead added to both sides compresses the per-gigabyte ratio rather than widening it. That is a statement about a ratio of totals, and it does not carry over to the break-even gating fraction below, which the same overhead moves the other way (Section 3.3). The device-only scope is restated wherever a per-module ratio is quoted in this book, because the ratio is the headline and the scope is what bounds it.

That ratio is measured against a DDR5 baseline which realizes its JEDEC-datasheet power-down credit, while ReRAM's power-down energy is an explicit no-savings placeholder (Section 3.1.6, item 5), so it is also an **upper bound** on the gap a ReRAM module that gates its periphery would show. Zero refresh does not begin to pay for it on its own, and the gating the model would need is substantial - but it is no longer out of range. Gating the whole static component for an idle fraction f leaves $(1 - f) \times 6.937 + 0.003$ W, so reaching DDR5's 0.399 W for the same 8 GiB requires $f = 94.3\%$. **That threshold sits inside, not above, the idleness band this book cites**: Malladi et al. report 2-6% memory-bandwidth utilization for web-search and data-analytics servers [21], which is 94-98% idle (Section 3.3). At the idle end of that band, $f = 0.98$, the gated module would draw 0.142 W against DDR5's 0.399 W for the same 8 GiB - *below* DDR5, at 0.36x - and at the busy end, $f = 0.94$, it would draw 0.419 W, 1.05x above. The band brackets break-even. What this model cannot do is price a gated ReRAM module at all, because no citable ReRAM power-gating characterization exists and the placeholder assumes none. Closing the gap is therefore a measurement this project could not make, not a possibility it has excluded, and equally not one it has demonstrated. The legacy PCM baseline, for its part, draws 0.046 W for a 4 GiB module, 0.0115 W/GiB, which is *below* DDR5 per gigabyte: on the power axis alone, the old technology is still the floor. Real dynamic power is small for every ReRAM configuration under GCC (2.9 to 4.9 mW module-wide) and rises to 346-417 mW under the AI read bursts, which is still under 6% of the module total. ReRAM power is leakage, full stop - but at the

matched organization that is now a statement about both cell types equally, and it is the central negative result of this evaluation.

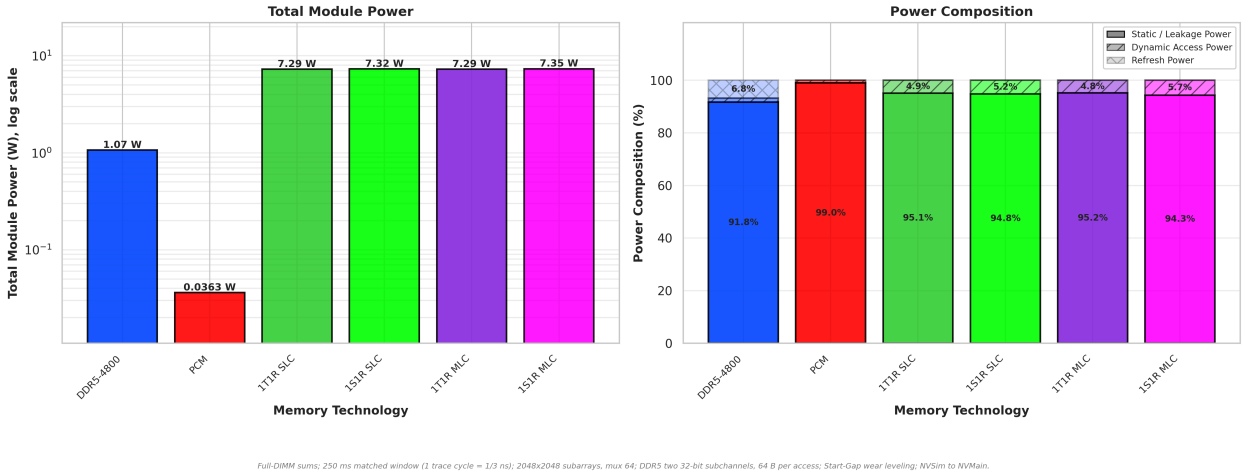


Figure 8: System Power Breakdown (Static / Dynamic / Refresh) under the highly active GPT-2 Inference (IFMAP) workload.

Regenerated from the primary dataset of this revision and in agreement with Table 3: all four ReRAM tracks sit on one shared leakage floor near 6.94 W for 8 GiB (SLC) or 16 GiB (MLC), against a 16 GiB DDR5 module. The chart is a module-level view; the per-gigabyte normalization, which is the only form in which these modules are comparable, exists only in Table 3.

The matched organization also settles an older question in this book. Under the simulator's generic standby defaults, total power once appeared to cluster at nearly the same level for every ReRAM configuration under any given workload - a convergence an earlier analysis treated as a real effect and named the “Standby Convergence”. The 3 September version retired that convergence in favor of two leakage classes. Both readings were consequences of the model rather than of the device, and the corrected result is that the four ReRAM full-DIMM configurations *do* share one static floor, 6.937 W, but for a substantive reason rather than a default: at a matched organization they have the same peripheral circuitry, and that circuitry, not the cell, is what leaks. Within that floor, SLC and MLC are nearly indistinguishable under GCC (6.9395 against 6.9411 W for the 1T1R pair), because dynamic energy contributes only milliwatts. The intra-family invariance that the earlier observation got right survives intact: write intensity never spikes module power, and MLC imposes no meaningful power penalty relative to SLC.

Under AI inference (GPT-2, Figure 8), the most parallel workload in the suite, accumulated dynamic read energy raises the 1S1R SLC module from 6.939 W to 7.318 W, a 5.5% excursion, while DDR5 spans 0.797-1.110 W across the entire suite; the standby floor decides the cross-technology ranking under every workload. One honest bound on the DDR5 side of that comparison: its supply currents are vendor *specification* limits (Section 2.3), not measured

typicals, and no typical figure exists to substitute - every Micron DDR5 datasheet checked, including [29], labels its only IDD column as worst-case current limits, no SK hynix typical figure was found, and JEDEC's own DDR5 IDD reporting template (a JC42.3 committee draft of [10]) defines only a maximum column. A typical-current module would sit *below* the figures used here, which widens the ReRAM deficit rather than narrowing it.

NVSim models no selector physics whatsoever, so on its own it cannot say whether a 2048-cell cross-point tile is even viable, let alone what a selector costs in sneak current. Because that omission sits directly under this book's flagship configuration, it is closed here by a standalone analytic layer built on the published read model of Zhou, Kim and Lu **[51]** and the handbook tile-leakage rule, evaluated at two selector qualities: an ovonic threshold switch representing present-day production practice (nonlinearity 10^4, I_{on} 100 uA, I_{leak} 10 nA at half the threshold) and a Crossbar-class FAST selector representing the best published selectivity (10^6, I_{leak} 0.1 nA) **[9]**. Three results follow, and all three are bounds rather than simulated values.

Sneak current adds nothing to standby power. Unselected, unbiased crossbar lines have no voltage across their cells and leak nothing through this mechanism, so the standby adder is exactly zero at both bounds. The 108.384 mW per chip above is not increased by the selector layer, and no figure in this book adds the two together.

During an access, the sneak budget is 7.56 mW per chip at the OTS bound and 0.076 mW at the FAST bound - counting the half-selected cells on the active lines of the sixteen subarrays a request activates. Paired with the single rank that actually serves a request, that is 60.5 mW (OTS) or 0.605 mW (FAST) of instantaneous access-time power. Even the OTS figure is a 0.87% instantaneous adder on a chip already leaking 108.384 mW, and it must be duty-cycled before being compared with anything.

Tile validity, not leakage, is what binds. Under the handbook rule that a tile side may not exceed $I_{on}/(6I_{leak})$, the OTS bound supports a maximum tile of 1666 cells a side - **so the 2048 x 2048 organization this book simulates is not valid at present-day selector quality**. The FAST bound supports 166,666 cells a side and is valid with three orders of magnitude to spare. Read margin is comfortable at both (24.0% and 23.2% against a 10% requirement), so the binding limit at the OTS bound is tile leakage and at the FAST bound is read margin. The consequence is stated plainly wherever 1S1R results appear: **every 1S1R number in this book is conditional on a selector better than today's production ovonic threshold switch.** That is a real, citable engineering requirement, and it replaces the “selector leakage discipline” framing of earlier versions, which attributed to the selector an advantage it does not have.

Bit errors and error correction

The selector layer above answers whether a 2048-cell tile can be *read* at all. It does not answer the question that sits one level above it: how often the bit that comes back is wrong, and what the correction of that bit costs. Neither NVSim nor NVMain models a bit error anywhere in this pipeline, so this subsection is a literature result set rather than a measurement, and it is written that way. Three categories are kept apart throughout, because literature that conflates them is

unusable here: *retention* errors (a cell that drifts out of its window while idle), *endurance* errors (a cell cycled past its rating, which hard-fails), and *read-disturb* errors (a cell perturbed by the reads of its neighbours or of itself). A single "bit error rate" that mixes the three cannot be compared against a DRAM figure, which is dominated by an entirely different mechanism.

Layer one: the raw device rate. For the cross-point track this book models as 1S1R, **no measured array bit error rate is published at all.** The only numeric figure is a channel model: a 1024 x 1024 array simulated at 10 ohm per junction with an *ideal* selector, so that the error is driven by line-resistance IR drop and device variability rather than by sneak-path current, gives a read bit error rate from 4.29×10^{-4} for the best-positioned cell to 7.33×10^{-2} for the worst-positioned one, and a write bit error rate from 3.35×10^{-4} to 1.75×10^{-2} **[60]**. A follow-on study adds BCH coding over 128- and 256-bit blocks with location-dependent interleaving and cuts the uncorrectable rate by about 45% **[61]**. Both carry the ideal-selector caveat, which is exactly the assumption the sneak-path analysis above exists to question, so **the 1S1R rate quoted here is a model output and must never be read as silicon.** The 1T1R track does have measured macros, and they span six orders of magnitude depending on the write scheme: HfO_2 arrays report a raw rate above 10^{-3} after a few million cycles and about 1.2% at low programming current, with a 2T2R variant reaching 0.2% and staying below 10^{-4} over 10^{10} cycles **[62]**; at the other end a 128 kb 1T1R macro with a smart-write scheme reports under 10^{-7} after 10^6 cycles at a 28.1 dB read margin **[63]**; and a 28 nm embedded RRAM study reports a raw rate around 10^{-6} corrected to an uncorrectable rate of 2×10^{-15} by a 2-bit-per-128-bit code **[64]**, with an endurance rating of 500,000 cycles at 2-bit correction extending to 1,000,000 cycles at 3-bit **[65]**. The usable statement is a band, not a number: **roughly 10^{-7} at the best published write scheme to 10^{-2} under adverse cycling**, and where a design lands inside that band is a write-algorithm and margin decision rather than a material property.

Layer two: what the selector itself contributes. A measured 1S1R selector stacked with STT-MRAM reports a per-read disturb rate of 10^{-9} and 10^9 read-disturb-free cycles **[66]**. An ovonic-threshold-switch phase-change cross-point cell reports a failure rate of 9% falling to 0.02% after read-endurance optimization **[67]**; an OxRAM-plus-OTS stack reports five decades of selectivity with more than 10^6 program cycles, 10^8 read cycles and 10^9 read-disturb-free cycles **[68]**; an OTS phase-change cell read below threshold reaches 10^{12} disturb-free reads **[69]**; and a 28 nm embedded RRAM read-disturb model puts the per-bit rate below about 10^{-18} **[70]**. The selector's own drift is a separate failure mode with a different signature: OTS threshold-voltage drift is reported at under 40 mV per decade in typical material stacks **[71]**, saturating after about 10^3 seconds **[72]**, and a drifted selector behaves as a *stuck bit* rather than as a noisy one, which is why it belongs with the hard failures and not with the soft-error rates above. **The selector's own contribution is therefore bounded well below the layer-one array rates**, which is the result that matters for this book: the 1S1R disadvantage established in this section is tile validity and sneak current, not error rate.

Layer three: the design point an NVM main memory actually uses. The standard design point in the persistent-memory architecture literature is a raw rate of 2×10^{-4} corrected with

BCH(2312,2048,22), which costs about **27% coding overhead [74]**. DRAM's on-die SECDED, by contrast, tolerates only about 10^{-6} raw **[75]**, and the only shipped NVM main memory corrects over a 256-byte block it calls an XPLine **[76]**, with its SSD sibling rated by datasheet at under one uncorrectable sector per 10^{17} bits **[77]**. Placing the three layers side by side gives the answer this subsection exists for. **A main memory built on published raw ReRAM error rates is possible, but it needs SSD-class rather than DRAM-class correction, at roughly a 27% cost in bits.** The stronger claim sometimes made in discussion - that the error rate rules out a memory system altogether - is not supportable against **[63]**, **[64]** or **[74]**. Two references are worth naming for the work that follows rather than for this model: ECP, which targets wear-out hard failures directly instead of treating them as soft errors **[78]**, and FREE-p, which tolerates up to four hard errors per line through operating-system-level remapping **[79]**. Both are the right shape for a resistive main memory and neither is modeled here.

Every latency in this book excludes ECC decode. The generated configurations carry no coding block, no syndrome computation and no decoder, so the read path this book times is the array plus the protocol and nothing else. A BCH(2312,2048,22) decoder sits on that path in a real part, and its latency is not a rounding error against an 11.25 ns device read. The latencies of Table 2 are therefore optimistic by exactly the decode time a 27%-overhead code would add, on top of being media rather than device numbers, and closing that gap is named as future work in Section 4.2 rather than estimated here from a code this project has not modeled.

Finally, the selector layer above is consistent with the same literature, which is worth stating because it was built independently of it. That layer applies a **10% minimum read margin [80]** and a half-bias nonlinearity of about 10^4, a value the OTS-only memory literature puts in the low thousands up to about 10^4 **[73]**. The analytic tradition that treats line resistance, rather than sneak current, as the binding constraint on read margin in a crossbar **[81]**, **[82]** finds that the 10% criterion needs a nonlinearity around 8,000 and an on/off window of 5 **[83]** - both of which this book's parameters satisfy. That is why, for the 2048 x 2048 tile, the limit that binds is the line-resistance and tile-leakage mechanism computed above (a ceiling of 1666 cells a side at the OTS bound, at a read margin of 0.24 against the 0.10 minimum) and *not* a sneak-path bit error rate. The two analyses agree on which constraint is active, and they were derived from different places.

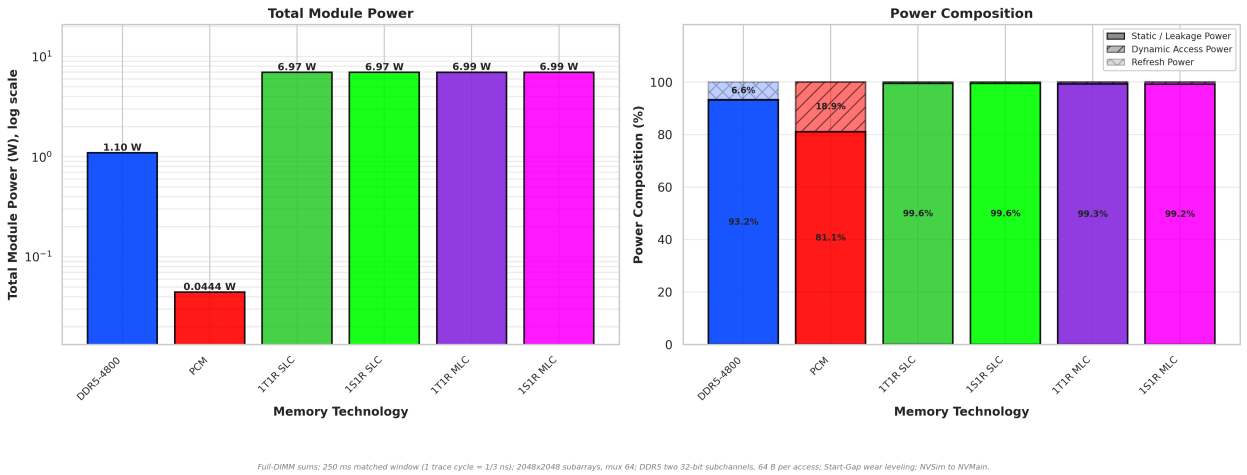


Figure 9: System Power Breakdown (Static / Dynamic / Refresh) under the continuous memory-streaming LBM (SPEC2017) workload.

Regenerated from the primary dataset of this revision and in agreement with Table 3: all four ReRAM tracks sit on one shared leakage floor near 6.94 W for 8 GiB (SLC) or 16 GiB (MLC), against a 16 GiB DDR5 module. The chart is a module-level view; the per-gigabyte normalization, which is the only form in which these modules are comparable, exists only in Table 3.

The dynamic term is best read under a workload like lbm, read-majority by request count (57/43, Section 2.4.1) but write-heavy in service-time impact (Figure 9; components this small cannot be resolved visually against the 6.94 W static floor - the values come directly from the NVMain energy counters). With the repaired per-technology access energies (Section 3.1.6, items 10 and 13), each MLC write costs 3.0x its SLC counterpart, and because every ReRAM track now completes the same request population, that energy difference is no longer offset by any throughput difference: 1T1R MLC dynamic power reaches 49.1 mW against SLC's 30.8 mW, and 1S1R MLC 52.6 mW against 31.3 mW. At module scale both variants remain pinned to the same 6.937 W static floor - a 0.3% perturbation - reconfirming that ReRAM DIMMs are bounded by static peripheral leakage rather than by dynamic cell activity.

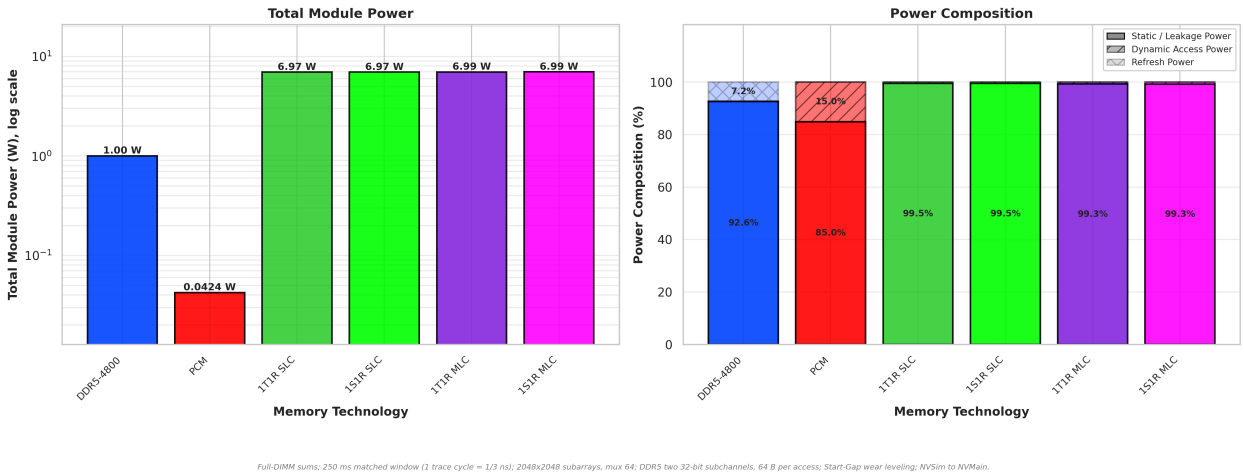
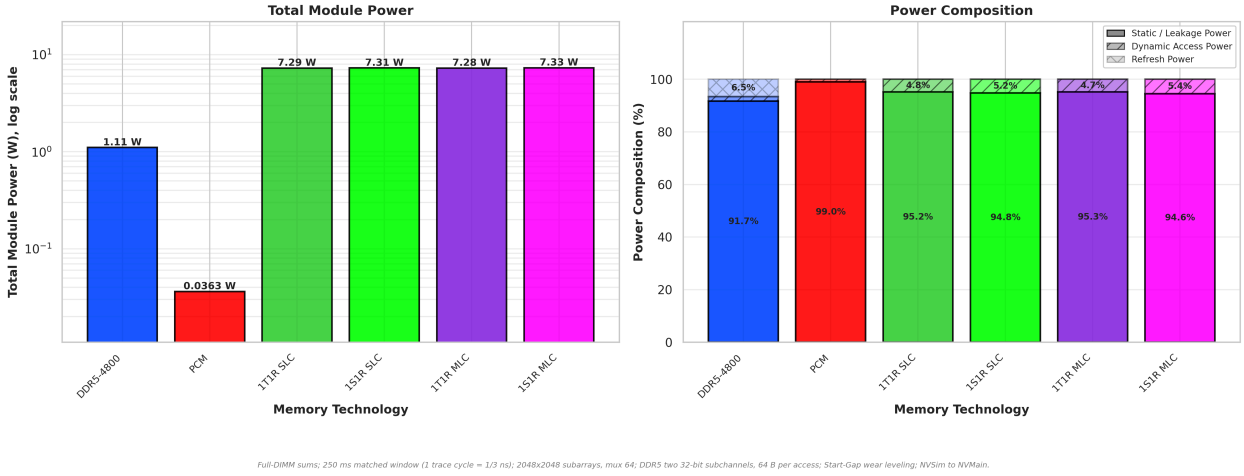


Figure 10: System Power Breakdown (Static / Dynamic / Refresh) under the STREAM benchmark.

Regenerated from the primary dataset of this revision and in agreement with Table 3: all four ReRAM tracks sit on one shared leakage floor near 6.94 W for 8 GiB (SLC) or 16 GiB (MLC), against a 16 GiB DDR5 module. The chart is a module-level view; the per-gigabyte normalization, which is the only form in which these modules are comparable, exists only in Table 3.

The real STREAM benchmark (Figure 10) follows the same direction as lbm: 1T1R SLC dynamic power reaches 33.6 mW while MLC draws 49.1 mW, the write-energy penalty showing through undiluted now that both complete the identical request population. On read-dominated traffic the ordering reverses, because MLC's corrected read-energy penalty is only 1.1x against the write side's 3.0x: under AlexNet IFMAP, 1T1R MLC draws 345.8 mW of dynamic power against SLC's 350.0 mW. In every case the module total remains anchored to the same 6.937 W static floor.



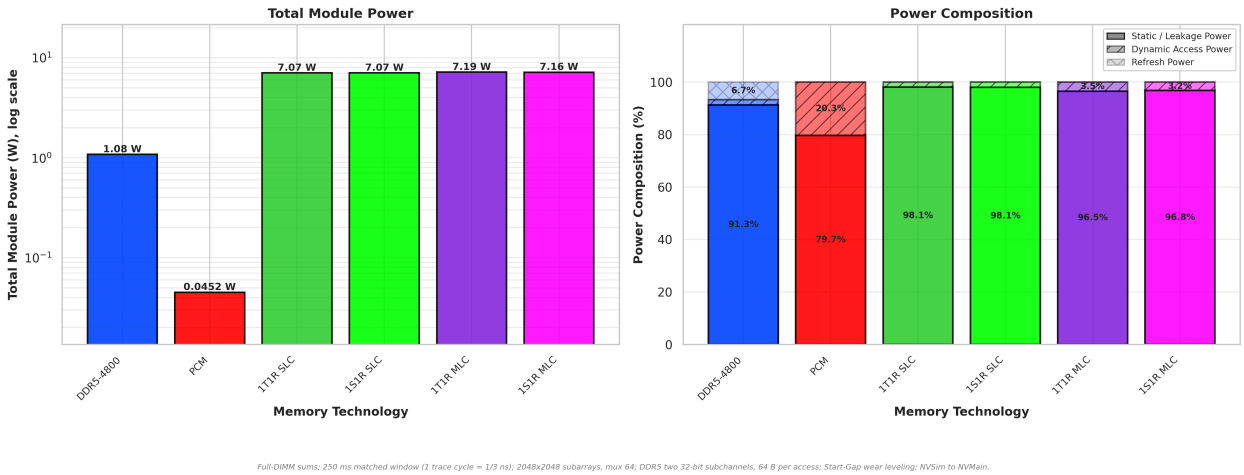


Figure 11 & 12: System Power Breakdown (Static / Dynamic / Refresh) under AlexNet Layer 1 IFMAP (Read-Heavy) and OFMAP (Write-Heavy) workloads.

Regenerated from the primary dataset of this revision and in agreement with Table 3: all four ReRAM tracks sit on one shared leakage floor near 6.94 W for 8 GiB (SLC) or 16 GiB (MLC), against a 16 GiB DDR5 module. The chart is a module-level view; the per-gigabyte normalization, which is the only form in which these modules are comparable, exists only in Table 3.

Finally, to ensure the Static Leakage Dominance principle holds across diverse neural network operations, I evaluated both the read-intensive AlexNet IFMAP and the write-intensive AlexNet OFMAP workloads (Figures 11 and 12). A ReRAM write is individually an expensive operation: NVSim prices a 1T1R SLC write at 0.913 nJ against 0.376 nJ for a read, and an MLC write at 2.740 nJ - so write-heavy traffic is precisely where one would expect ReRAM power to spike. It does not. Despite the severe latency penalty incurred during OFMAP writes (Section 3.1.1), OFMAP dynamic power is *lower* than IFMAP dynamic power for every configuration (1T1R SLC 132.8 against 350.0 mW; 1S1R SLC 137.0 against 378.1 mW; 1T1R MLC 248.7 against 345.8 mW), because the OFMAP burst is an order of magnitude smaller than the IFMAP burst (135,424 writes against 1,269,600 reads) and its slower service spreads that energy over a longer stretch of the window. Total power tracks the static floor in every case, perturbed only marginally by request throughput: the whole dynamic range across all six workloads and all four ReRAM tracks is 6.939 to 7.353 W, a 6% spread on a 6.937 W floor.

Table 3 collects the whole-module power totals and, in the column that actually supports a cross-technology comparison, the same figures per gigabyte.

Table 3: Total module power (W) and power per gigabyte (W/GiB), full-DIMM, module-sum semantics.

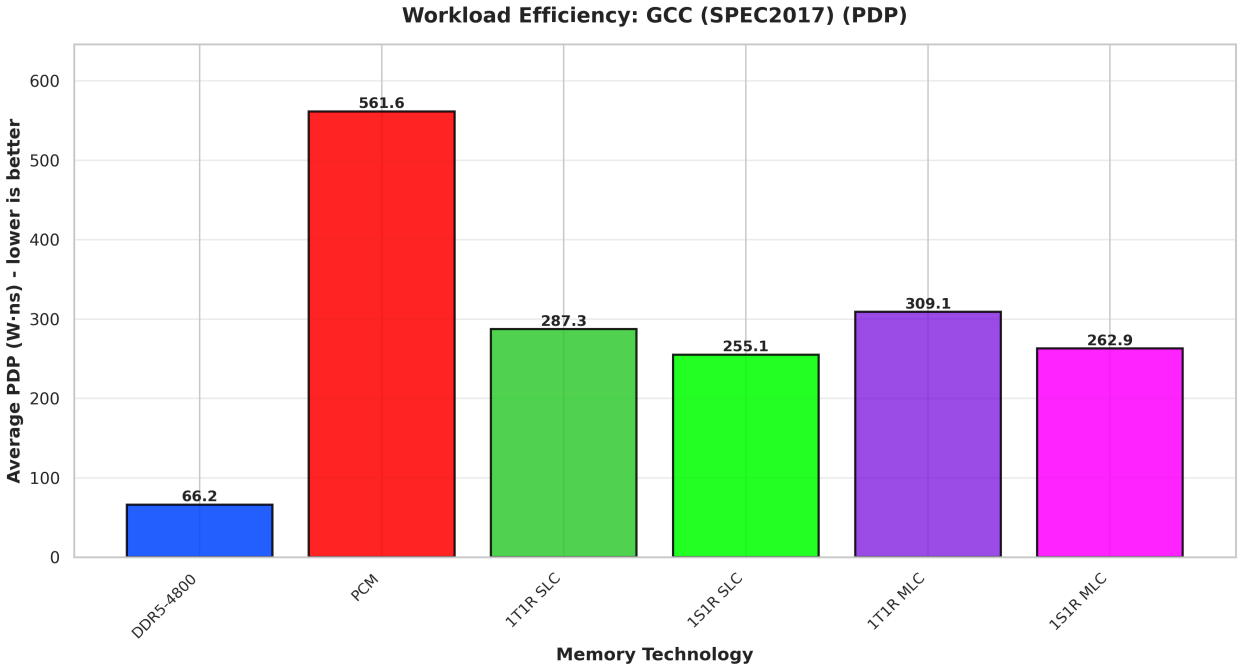
Technology	GiB	GCC	LBM	STREAM	GPT-2	AlexNet IFMAP	AlexNet OFMAP
DDR5-4800	16	0.797	1.095	1.003	1.067	1.110	1.080
PCM (legacy)	4	0.046	0.044	0.042	0.036	0.036	0.045
1T1R SLC	8	6.939	6.967	6.970	7.291	7.287	7.069
1S1R SLC	8	6.940	6.968	6.970	7.318	7.315	7.074
1T1R MLC	16	6.941	6.986	6.986	7.287	7.282	7.185
1S1R MLC	16	6.941	6.989	6.988	7.353	7.335	7.162
per GiB: DDR5	16	0.0498	0.0685	0.0627	0.0667	0.0694	0.0675
per GiB: PCM	4	0.0115	0.0111	0.0106	0.0091	0.0091	0.0113
per GiB: ReRAM SLC	8	0.8674	0.8709	0.8713	0.9113	0.9108	0.8837
per GiB: ReRAM MLC	16	0.4338	0.4366	0.4366	0.4554	0.4552	0.4491

Whole-module sums for every technology. The lower block divides each module total by its own capacity, which is the only form in which these technologies are comparable: the ReRAM SLC DIMM is 8 GiB, the MLC DIMM 16 GiB of the same 64 chips, the DDR5 module 16 GiB and the legacy PCM configuration 4 GiB. The per-GiB rows are shown for 1T1R; the 1S1R values differ by at most 0.4% for SLC and 0.9% for MLC. Read that way, ReRAM SLC costs ***12.7x to 17.4x*** *DDR5's power per gigabyte and ReRAM MLC* ***6.4x to 8.7x*** *- MLC halves the penalty precisely because leakage is per chip and MLC puts twice the capacity behind the same chips. PCM sits* below *DDR5 per gigabyte (0.13x to 0.23x), a figure that reflects the leakage constant of the NVMain configuration it inherits (Section 2.3) rather than a characterized device.* ***Every figure in this table counts memory devices only*** *- no register clock driver, power-management IC, PHY, termination or ReRAM-side controller, on either side (Section 3.1.2) - so a fixed per-module overhead would compress the per-GiB ratios, while moving the break-even gating fraction of Section 3.3 the other way. DDR5 figures include the restored idle-gating mechanism (Section 3.1.6, item 5) and the corrected current-mode background accounting of Appendix A; ReRAM's power-down mechanism is mechanically live but its power-down energy is an explicit, disclosed placeholder equal to existing standby energy, so no ReRAM figure claims a gating benefit and every per-GiB ratio here is an upper bound on the gap a gated ReRAM module would show; break-even against DDR5 needs 94.3% of the static component gated (Section 3.1.2). The analytic selector layer adds* ***zero*** *to any standby figure in this table. Source: results/ rev2026-09_primary_csv/processed_bar_chart_metrics.csv; results/selector_layer.json.*

3.1.3 Power-Delay Product (PDP): The Architectural Sweet Spot

While isolated latency and power metrics define physical boundaries, true architectural viability is measured by the balance between them. I utilized the Power-Delay Product (PDP = Total System Power [W] × Average Request Latency [ns]; units W·ns = nJ) to rank configurations. All PDP values in this section are computed from the repaired, module-sum power model of Section

3.1.6 with its idle-gating mechanism restored: DDR5's values include real power-down savings, ReRAM's include only the small power-down transition latency (its power-down energy is a no-savings placeholder), and PCM shows no power-down activity. This asymmetry must accompany any reading of the cross-technology ranking.



Full-DIMM sums; 250 ms matched window (1 trace cycle = 1/3 ns); 2048x2048 subarrays, mux 64; DDR5 two 32-bit subchannels, 64 B per access; Start-Gap wear leveling; NVSim to NVMain.

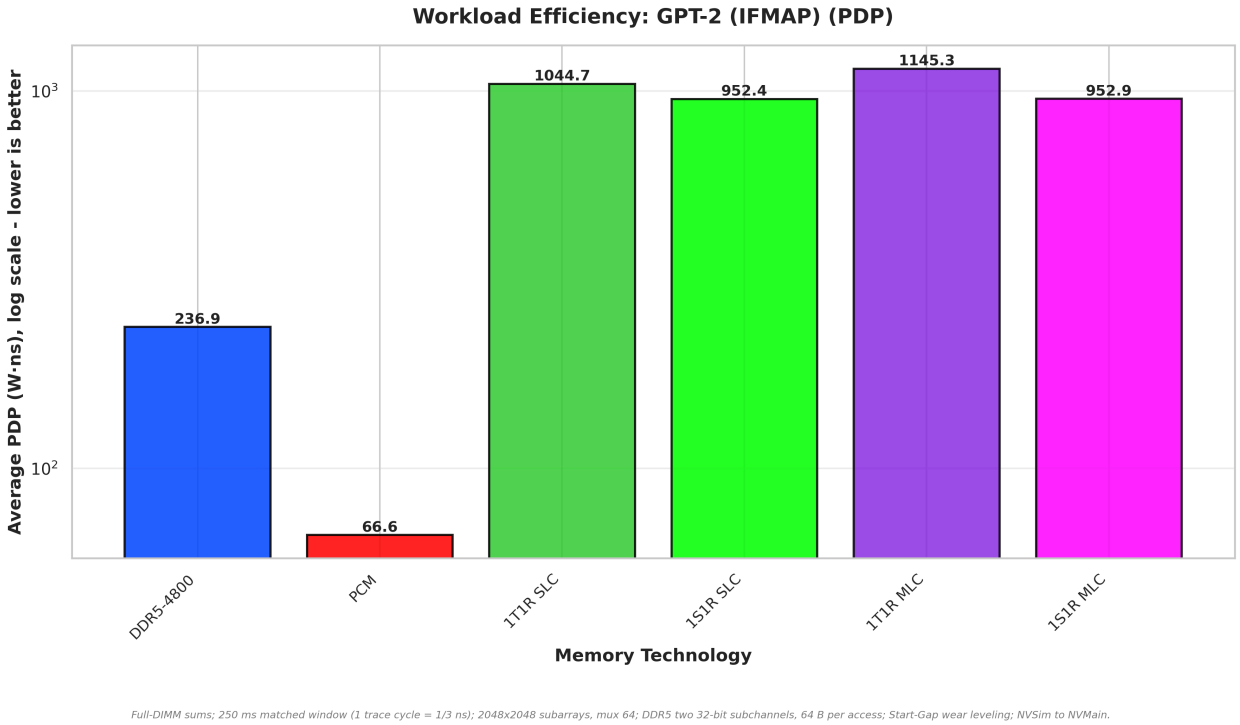


Figure 13 & 14: Power-Delay Product under single-threaded compute (GCC SPEC2017) and massively parallel AI inference (GPT-2 IFMAP) workloads.

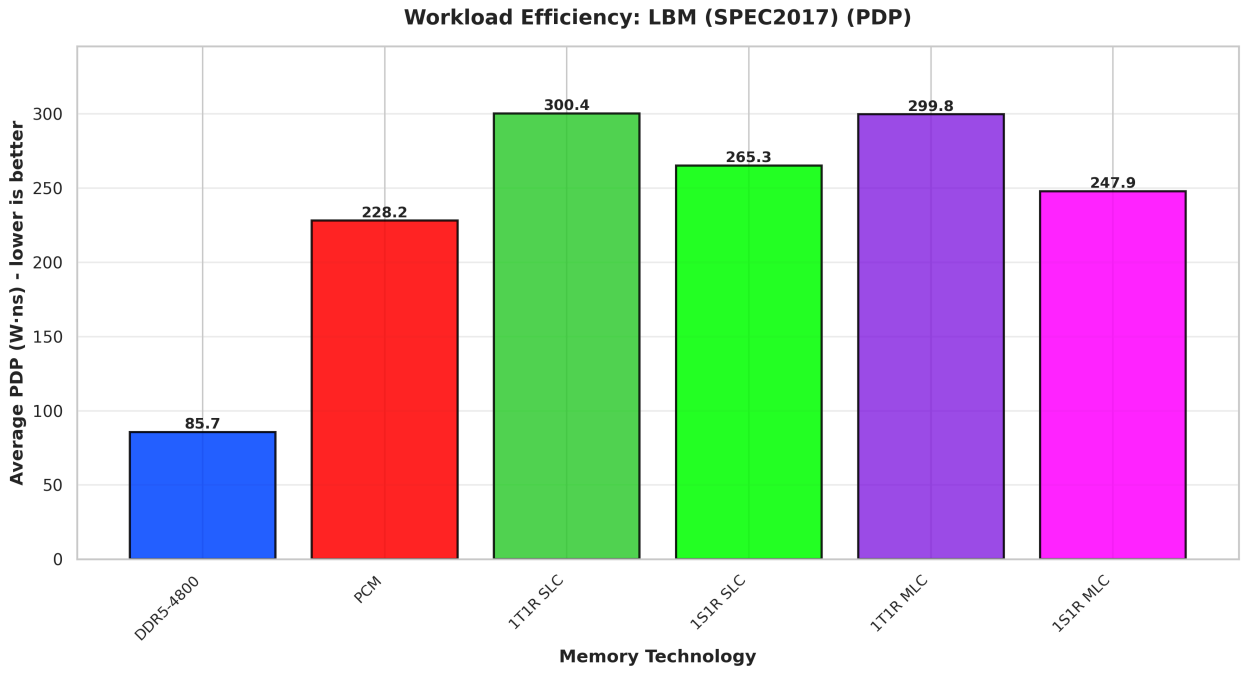
Regenerated from the primary dataset of this revision and in agreement with Table 4. The chart is a module-level view; the per-gigabyte normalization of Table 4's note, under which the ordering between PCM and MLC ReRAM reverses, is not plotted.

With leakage identical between the two cell types, the intra-ReRAM PDP hierarchy that the 3 September version reported collapses to a near tie. Under GCC (Figure 13), 1S1R SLC posts 255.1 W·ns against 1T1R SLC's 287.3, an 11% advantage that comes entirely from the selector's shorter read path, not from any power difference; across the six-workload geometric mean the two are 594.5 and 641.0 W·ns, 1.08x apart. The 29x intra-ReRAM advantage this book previously reported (Appendix D) was the withdrawn leakage artifact propagated into the power term, and it does not exist. Against the baselines the ordering is unambiguous and unfavorable: DDR5 posts 66.2 W·ns under GCC and 236.9 under GPT-2 (Figure 14), so the best ReRAM configuration in the suite trails DDR5 by 3.9x under GCC and by 4.4x on the geometric mean. Both terms of that ratio moved in this revision - DDR5's power rose with the corrected background accounting and its latency rose with the corrected JEDEC write path (Appendix A) - so the gap is roughly a quarter of the one this book reported before the correction, and it is still an order-of-magnitude statement only in the per-gigabyte form below.

At module level the legacy PCM baseline lands between them at 173.6 W·ns geometric mean, ahead of every ReRAM track by 3.4x to 4.2x. Three caveats bound that comparison and none of them is small. First, its latency averages on LBM and STREAM cover only the 50.2% and 46.1% of requests it completed, so two of the six terms in its geometric mean are *lower bounds* - the

same service-limitation caveat Table 4 applies to the two SILICON rows applies to PCM. Second, its module is 4 GiB against ReRAM SLC's 8 GiB and MLC's 16 GiB, and *per gigabyte the ordering reverses*: PCM is 5.1x DDR5 against 4.7x for 1S1R MLC (Table 4 note). Third, its static floor is a constant inherited from NVMain's bundled example configuration on a single-channel, single-rank, 8-device module under a different memory controller (Section 2.3), not a characterized device. The defensible statement is therefore narrow and is about neither phase-change physics nor a technology ranking: nothing in this study's ReRAM matrix improves on a legacy PCM configuration's efficiency *at module level*, while MLC ReRAM does improve on it *per gigabyte*. Both halves are statements about the 22nm peripheral-leakage floor.

Cell-level density partly repays this, and it is the one lever that does. PDP as tabulated is a module quantity, and the ReRAM SLC module holds 8 GiB against DDR5's 16 GiB, so the capacity-normalized comparison is harsher still for SLC (8.7x on the geometric mean) and markedly better for MLC, which puts 16 GiB behind the same 64 leaking chips: 4.7x for 1S1R MLC. Within a family, MLC costs efficiency at the module level under write pressure (4,676.7 against 2,769.2 W·ns for 1S1R under AlexNet OFMAP) and is essentially free elsewhere, so the efficiency case for MLC rests on capacity, not on speed.



Full-DIMM sums; 250 ms matched window (1 trace cycle = 1/3 ns); 2048x2048 subarrays, mux 64; DDR5 two 32-bit subchannels, 64 B per access; Start-Gap wear leveling; NVSim to NVMain.

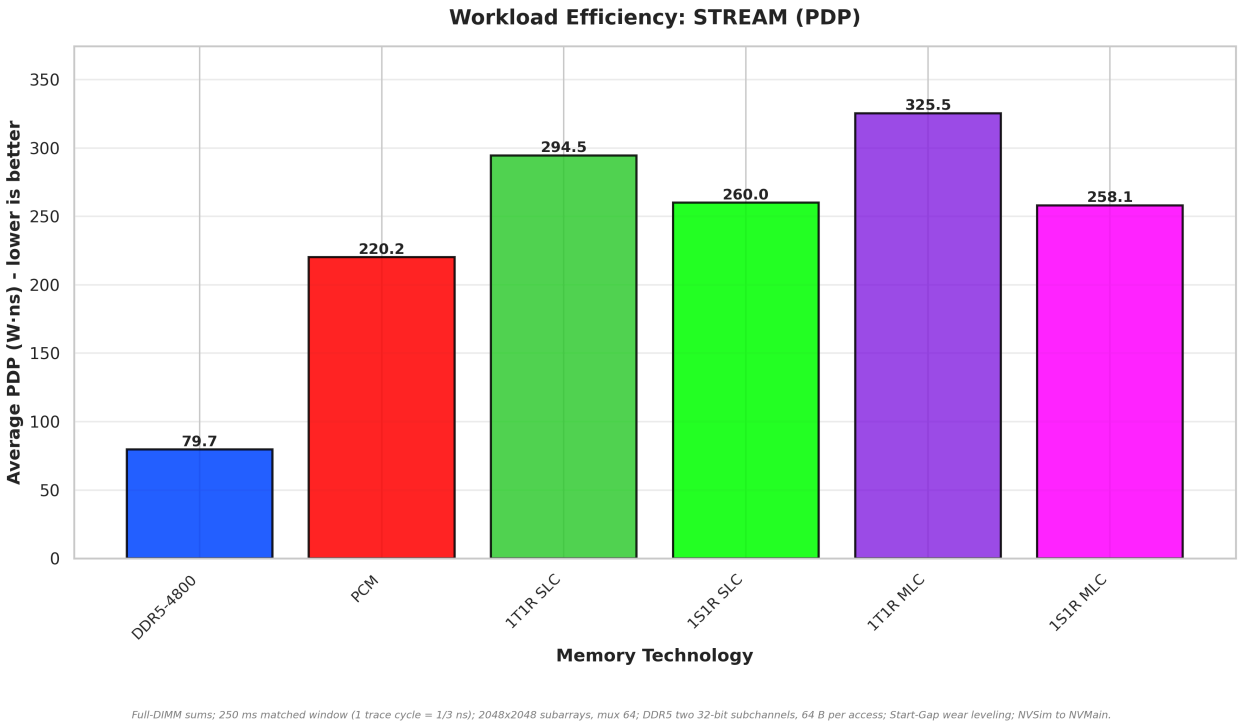
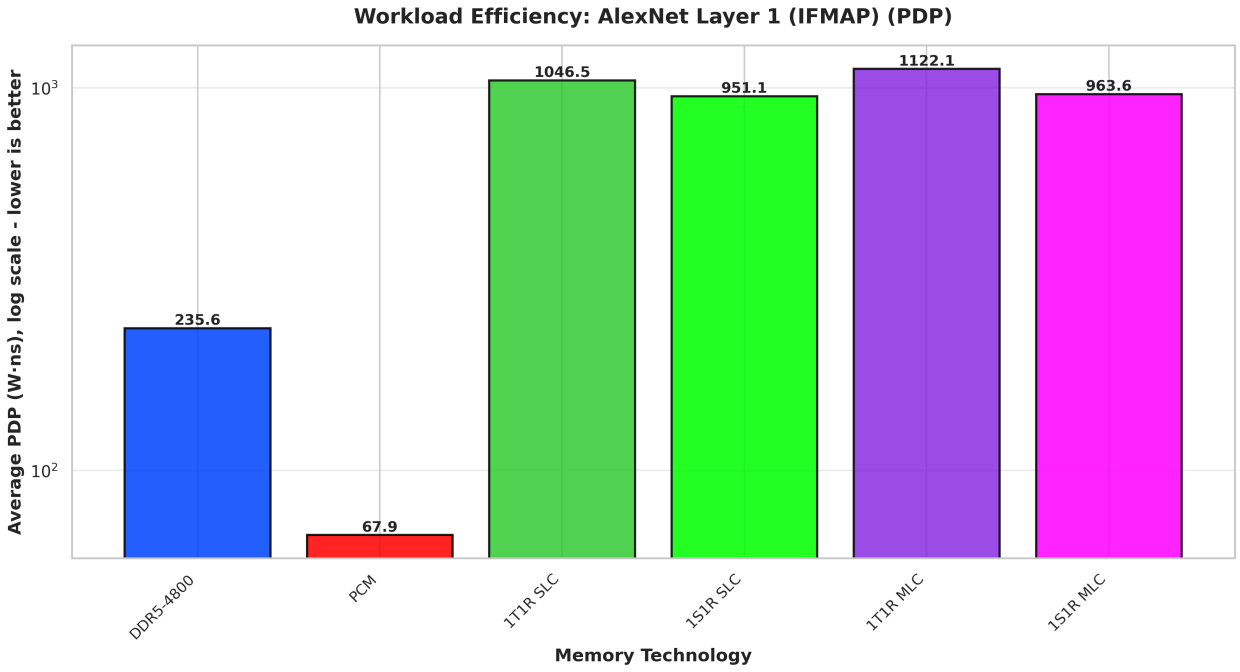


Figure 15 & 16: Power-Delay Product under heavy continuous memory-streaming workloads (LBM SPEC2017 and STREAM).

Regenerated from the primary dataset of this revision and in agreement with Table 4. The chart is a module-level view; the per-gigabyte normalization of Table 4's note, under which the ordering between PCM and MLC ReRAM reverses, is not plotted.

Under maximum bandwidth pressure (Figures 15 and 16) the ordering persists and the margins narrow within the ReRAM family: 1S1R SLC posts 265.3 W·ns under lbm and 260.0 under stream, 1T1R SLC 300.4 and 294.5, and the MLC siblings are indistinguishable from their SLC counterparts (247.9 and 258.1 for 1S1R MLC), because at these request rates the extended write recovery is booked as subarray-busy time after a row-closing write rather than as the write's own request latency (Section 3.1.1; the bounding run there brackets this). DDR5 holds a 3.1x and 3.3x edge over 1S1R SLC in the streaming regime (85.7 and 79.7 W·ns). Streaming is therefore not where ReRAM loses efficiency ground relative to its own compute-bound result; leakage is flat, so the efficiency ratio tracks latency, and latency is the one axis on which the projected device is competitive.



Full-DIMM sums; 250 ms matched window (1 trace cycle = 1/3 ns); 2048x2048 subarrays, mux 64; DDR5 two 32-bit subchannels, 64 B per access; Start-Gap wear leveling; NVSim to NVMain.

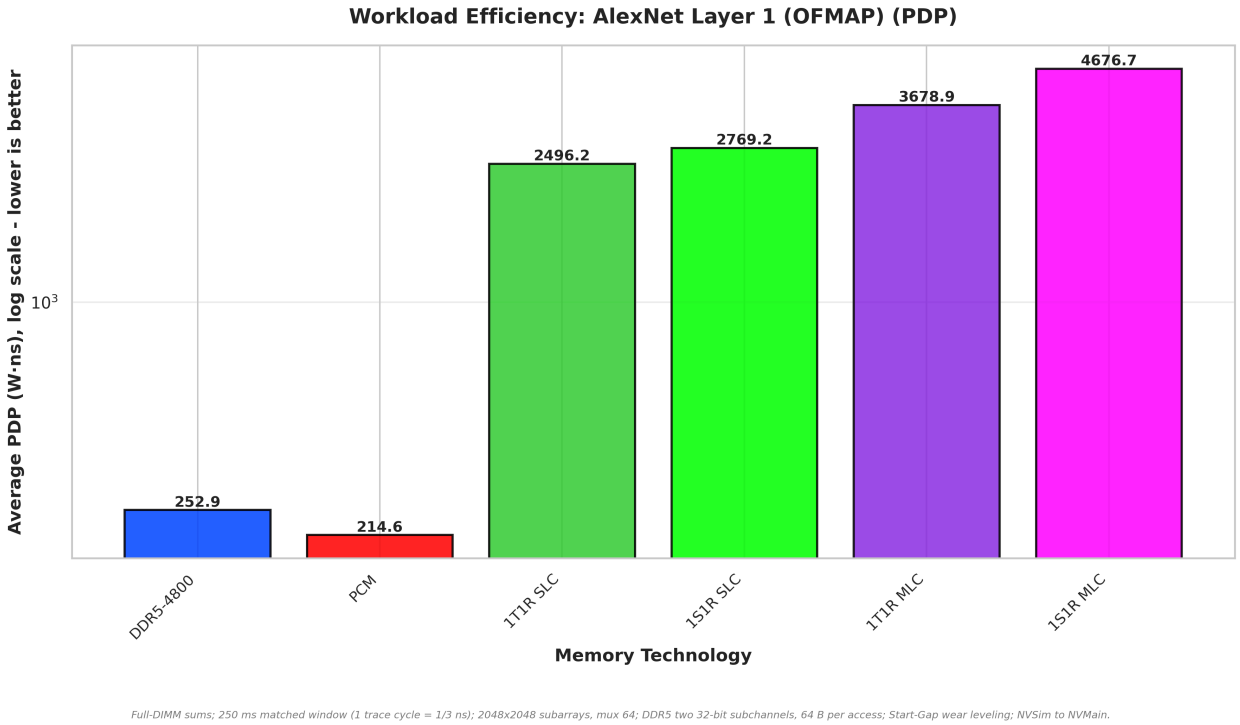


Figure 17 & 18: Power-Delay Product under AlexNet Layer 1 IFMAP (Read-Heavy) and OFMAP (Write-Heavy) workloads, highlighting the MLC write-latency penalty on system efficiency.

Regenerated from the primary dataset of this revision and in agreement with Table 4. The chart is a module-level view; the per-gigabyte normalization of Table 4's note, under which the ordering between PCM and MLC ReRAM reverses, is not plotted.

Finally, the PDP diagnostics map the efficiency limits of density scaling. Under the read-heavy AlexNet IFMAP (Figure 17), moving from 1S1R SLC to the ultra-dense 1S1R MLC topology costs only 1.3% at the module level (951.1 to 963.6 W·ns), and once capacity is accounted for MLC is twice as efficient per gigabyte. Under the write-heavy AlexNet OFMAP trace (Figure 18), the 1S1R MLC PDP rises to 4,676.7 W·ns, 1.7x its SLC sibling's 2,769.2, which even after capacity normalization leaves MLC slightly ahead. The efficiency tax for density is therefore real but confined to sustained write pressure, converging with the latency argument of Section 3.1.1 and the endurance argument of Section 3.1.4 on the same deployment rule: MLC belongs in read-dominant tiers.

Table 4 collects the Power-Delay Products with the suite geometric means, the tabular form of Figures 13-18 and 27.

Table 4: Power-Delay Product (W·ns = nJ), full-DIMM, and six-workload geometric means.

Technology	GCC	LBM	STREAM	GPT-2	AlexNet IFMAP	AlexNet OFMAP	Geo. mean
DDR5-4800	66.2	85.7	79.7	236.9	235.6	252.9	136.21
1T1R SLC	287.3	300.4	294.5	1,044.7	1,046.5	2,496.2	641.0
1S1R SLC	255.1	265.3	260.0	952.4	951.1	2,769.2	594.5
1T1R MLC	309.1	299.8	325.5	1,145.3	1,122.1	3,678.9	722.8
1S1R MLC	262.9	247.9	258.1	952.9	963.6	4,676.7	645.4
PCM (legacy)	561.6	228.2	220.2	66.6	67.9	214.6	173.6
1T1R SILICON	82,498	120,432	192,929	75,899	74,678	555,432	134,930
1S1R SILICON	2.63e6	2.25e6	3.68e6	1.35e6	1.34e6	7.58e6	2.58e6

*PDP = total module power x average request latency, so it is a **module** quantity and the modules differ in capacity: the ReRAM SLC DIMM is 8 GiB, the MLC DIMM 16 GiB, DDR5 16 GiB, the legacy PCM configuration 4 GiB (Table 3). Normalized per gigabyte, the geometric-mean ratios against DDR5 are 9.4x (1T1R SLC), 8.7x (1S1R SLC), 5.3x (1T1R MLC), 4.7x (1S1R MLC) and 5.1x (PCM), so per gigabyte MLC ReRAM is ahead of the legacy PCM configuration even though it is behind at module level. At module level the same ratios are 4.7x, 4.4x, 5.3x, 4.7x and 1.3x.* ***Service-limited rows:*** *the two SILICON rows* and the PCM row *carry the service-limitation caveat of Table 2. PCM completes 50.2% of LBM and 46.1% of STREAM, so two of its six terms, like every SILICON term, average a completed prefix only, and its PDP is a lower bound on the full-window cost. DDR5's idle power-down mechanism is restored and live (Section 3.1.6, item 5); ReRAM's is mechanically live but carries a no-savings placeholder energy, so no ReRAM figure here claims a gating benefit. Source: results/rev2026-09_primary_csv/ processed_bar_chart_metrics.csv and processed_geometric_means.csv.*

3.1.4 Endurance Viability Analysis

A key viability question for ReRAM as a main memory replacement is write endurance: how many write cycles before cell degradation? Everything in this section is a *projection* from a write distribution measured in simulation. **Every projection here assumes that distribution is stationary**: the spatial pattern of writes measured over one 250 ms window is taken to repeat, unchanged, for the module's entire life - no footprint drift, no operating-system page remapping, no allocator churn, no change of phase inside the application. That assumption is what converts a quarter-second measurement into a figure in years, and it bears hardest on the columns that are set by a single hottest location: the per-row maxima measured in the window are 1,059 writes under GCC, 3,072 under LBM and 1,024 under STREAM, spread over the 1,024 line locations a row spans, so the hottest individual line carries only a handful of writes in the sample being extrapolated. For the no-leveling and single-region Start-Gap columns the assumption is therefore pessimistic - a footprint that moves at all relieves the hot line - while for the ideal

column it is close to neutral. No lifetime here was measured, and none should be read as though it were.

The rating basis has been corrected. Earlier versions of this book rated SLC cells at 10^7 cycles and MLC at 10^6, citing Wong et al. [14]. That paper gives no SLC or MLC rating at all: its endurance table lists single laboratory devices spanning 10^2 to 10^{12} cycles, and its only multilevel figure is a tungsten-oxide cell below 100 cycles without verify. The citation has been withdrawn for this purpose. The best-sourced planning value for a gigabit-class storage-class memory is **10^6 cycles**, which Chen's 2020 review states as the typical specification for that application [47], and 10^6 is the primary basis used throughout this section. Two bounds are carried alongside it: 10^4, the floor that qualified array-level parts actually ship at (TSMC's 22nm macro is qualified to 10K SET+RESET cycles [50], and the EMBER macro reports 10K for 1 to 2 bits per cell [31]), and 10^7 as an optimistic ceiling. Laboratory single-device results reach 10^{10} to 10^{12} [45], but [15] finds that most claims above 10^6 rest on very few measured points from one device, and [46] places shipped filamentary parts at 10^4 to 10^7. Every lifetime below scales linearly with this choice: reading Table 5 at 10^4 divides every figure by 100, and at 10^7 multiplies it by 10.

Wear is now measured per location rather than assumed uniform. A per-location write counter was added to NVMain's endurance model for this revision, so the simulation reports where its writes actually landed instead of only how many there were. Two accounting rules govern how that output is read. First, the endurance model in use keys wear by *row*, and a row in this configuration spans 1024 sixty-four-byte line locations, so the simulator's `wearMaxWrites` statistic (3,072 under LBM, for instance) is a per-row total and **overstates per-cell wear by up to 1024x**. It is used in this book only for comparing one configuration against another, never as a cell-wear figure; per-line wear is taken from the trace-level analysis instead. Second, lifetime is computed over the *physical* cell population: the 8 GiB SLC module comprises 134.2 million 64-byte line locations, and the configuration generator's old 512 GB decoded address space, which made that population ambiguous, has been corrected (Section 2.3).

The write rates themselves were wrong, and are now right. The 3 September version reported LBM writing at 39.1 million writes per second. That figure came from a window that sat inside LBM's start-up burst, on a trace carrying fourfold duplicate records. On the regenerated trace, over a 250 ms window starting 10 ms after the CPU switch, LBM writes at **9.54 million writes per second** sustained (2,384,804 writes in window), GCC at 0.88 M/s (220,776) and STREAM at 7.99 M/s (1,998,124). The AlexNet OFMAP figure, 132.5 M/s, is a *burst rate* and not a sustained requirement: that trace is a 4.5 us burst of 135,423 writes which the 1T1R SLC full DIMM takes 1.022 ms to absorb, the rate is the burst divided by that drain time, and every row derived from it is flagged accordingly.

Table 5 collects the projection at a 64 GiB module, the capacity at which ReRAM's density advantage is realized, across four wear-leveling policies.

Table 5: Projected lifetime and required endurance (1T1R SLC full DIMM, 64 GiB, 10^6 cycles/cell, 250 ms window, one write per line per request).

Workload	Write rate	No leveling	Start-Gap	Rand. Start-Gap	Ideal	Req. endur. (ideal)	Req. endur. (rand. SG)
GCC	0.88 M/s	7.7 h	7.7-15.4 h	7.7-15.4 h	38.6 yr	2.6×10^5	6.3×10^6
LBM	9.54 M/s	23.2 h	23.2 h	0.20 yr	3.6 yr	2.8×10^6	6.5×10^6
STREAM	7.99 M/s	69.5 h	69.5 h	0.96 yr	4.3 yr	2.3×10^6	4.3×10^6
AlexNet OFMAP†	132.5 M/s†	0.27 min	0.27-0.53 min	0.27-0.53 min	0.26 yr	3.9×10^7	4.1×10^8

*†**Burst-derived row: not a sustained requirement.** AlexNet OFMAP is a 4.5 us burst of 135,423 writes; its "rate" is that burst divided by the 1.022 ms the module takes to drain it (the simulation ends when the trace is exhausted), as if bursts arrived back to back without pause, and its lifetimes and required-endurance figures must not be read as a steady-state duty cycle. The three gem5 rows are sustained rates from traces that run longer than the window. Ranges in the Start-Gap columns mark cases where the hottest line fails before the gap completes its first rotation, so the true lifetime lies between the no-leveling value and twice it. **Required endurance** is the per-cell cycle count a ten-year life would demand at this write rate and capacity, which is the same equation solved for endurance rather than for time. **Every row assumes a stationary write distribution:** the spatial pattern of writes measured inside the 250 ms window is taken to repeat unchanged for the module's whole life (Section 3.1.4). Lifetimes scale linearly with the endurance rating throughout, but **capacity scaling applies only to the Ideal column** (8 GiB divides by 8, 128 GiB multiplies by 2). With no leveling, lifetime is set by the hottest line and is* independent *of module size. Under either Start-Gap column a larger module can be* worse, *because the rotation period grows with the region while the gap still advances one line per hundred writes: see the paragraph below. Source: results/endurance_table.csv, rows capacity_gib = 64, endurance = 1e6, write_reduction = 1.0, rate_basis = admitted.*

The table's dominant feature is the distance between its columns, and that distance is the finding. **Endurance viability is decided almost entirely by the wear-leveling mechanism, not by the cell.** With no leveling at all, a 64 GiB SLC module survives *hours* under every sustained workload in the suite, because the traces are sparse: GCC touches 577 row locations, LBM 826 and STREAM 1,954, out of the 131,072 rows of the simulated 8 GiB module, and out of 1,048,576 at the 64 GiB capacity Table 5 projects. A workload that concentrates its entire write stream on a few hundred locations exhausts them regardless of how much unused capacity sits beside them. With ideal uniform leveling the same module reaches 38.6 years under GCC and 3.6 under LBM. The previous version of this book quoted only the ideal-leveling column, described it as a capacity law, and never measured the alternative; the gap between the two columns is four to five orders of magnitude.

Deterministic Start-Gap does not close that gap for these footprints. A faithful implementation of Qureshi et al.'s Start-Gap remapper [52] was added to the simulator for this

revision: one gap line, all lines rotating by one every 100 writes. The decoder's own relocation traffic, one line read plus one line write per 100 host writes and so about a 1% write overhead, is charged as *wear* in the projection of Table 5 but is *not* issued as traffic in the simulation, so no latency or power figure anywhere in this book carries the cost of running Start-Gap. Projected over a single whole-module region the mechanism recovers essentially nothing: 7.7 hours under GCC against 7.7 without leveling, 23.2 under LBM against 23.2. The mechanism is sound and the implementation is exact; the reason it fails here is structural. Start-Gap moves the gap one line per 100 writes, so rotating it once through a 64 GiB module of 1.07 billion lines takes about 10^{11} writes, while the hottest line in these sparse footprints accumulates its 10^6-cycle budget after about 10^6 writes land on it. The hot line dies long before relief arrives. Where the rotation *does* complete within the lifetime, the projection is exact (LBM completes 8 rotations, STREAM 19); where it does not, the model reports a range between the no-leveling value and twice it, because the outcome depends on where in the rotation the hot line sat. Qureshi's reported 97% of ideal endurance (for *randomized* Start-Gap; his plain Start-Gap reaches 53% and no leveling 5%) **does not transfer here, and the write distribution is only half the reason**. The other half is the endurance basis: his projection assumes cells rated at 2^{25} cycles, about 3.4 x 10^7, roughly 34 times the 10^6 planning value used throughout this section, and a rotation only pays off if it completes before the hottest line exhausts its budget, so 34x more budget buys 34x more rotations per lifetime. Denser, less sparse footprints are the second factor: his workloads spread their writes over far more of the module than the few hundred row locations these six traces touch. Randomizing the region assignment, which breaks the fixed correspondence between an address and its position in the rotation, recovers a substantial part of the ideal in the two streaming cases (0.20 years under LBM, 0.96 under STREAM, against ideals of 3.6 and 4.3) and still nothing under GCC, whose footprint is small enough that randomization has almost nothing to randomize over.

A consequence of that mechanism is worth stating plainly, because it runs the other way from intuition: **under a single whole-module region, a larger module is worse, not better**. The rotation period grows with the region while the gap still advances one line per hundred writes, so relief arrives later on a bigger module even though there is more of it to spread wear across. Projected under randomized Start-Gap, GCC survives 0.69 years at 8 GiB and only 7.7 hours at 64 GiB, because at 64 GiB the hottest line exhausts its budget inside the first rotation; LBM falls from 0.22 years at 8 GiB to 0.20 at 64 and 0.05 at 128, and STREAM is non-monotonic (0.36, 0.96, 0.87). The three capacity behaviors therefore differ by scheme, and Table 5’s note states each: ideal-leveling lifetime scales *with* capacity, no-leveling lifetime is *independent* of it, and single-region Start-Gap lifetime can fall *against* it. Sub-dividing the module into fixed regions, which is what Qureshi et al. actually propose [52], shortens the rotation period and decouples it from total capacity - but it does not, on its own, recover the ideal, because wear stays confined to the regions the workload actually touches, and no scheme confined to the touched footprint can outlast the endurance budget of that footprint divided by the write rate. Qureshi pairs regions with address randomization, and it is that pair, not regions alone, which produces his 97%. The corresponding experiment here has *not* been run: the projection tool supports regions, and what

can honestly be said in advance is only that regions shorten the rotation, not by how much they would move any lifetime in Table 5.

Read in the form a system architect actually needs - Shahar Kvatinsky's formulation, which is the same equation solved for endurance rather than for time - the requirement is explicit. For a ten-year life on a 64 GiB module, a cell must survive **2.6×10^5 cycles under GCC, 2.8×10^6 under LBM and 2.3×10^6 under STREAM with ideal leveling**, and 6.3, 6.5 and 4.3×10^6 under randomized Start-Gap. Against the 10^6 planning value from [47], GCC clears the bar under ideal leveling by a factor of four and everything else fails it by factors of 2.3 to 6.5. Against a 10^4 array-qualified part, nothing clears it. Against an optimistic 10^7, everything clears it under ideal leveling and the two streaming workloads clear it under randomized Start-Gap. The honest summary is that **a ten-year ReRAM main memory requires a cell rated at least in the high 10^6 range and a wear-leveling scheme substantially better than a single-region Start-Gap** - and that the second requirement is the one this project can state with measured support, because it measured where the writes went.

One negative result belongs here, because it bounds what the simulator can contribute to this section at all. **The simulation cannot show wear leveling at work.** With one Start-Gap region of 67,108,864 lines (4 GiB: NVMain's default region is ROWS x COLS x BANKS x RANKS x 64 B, the per-channel geometry without the channel count, and the two channels share one gap register, so the move counts are totals across both) and one gap move per 100 writes, a 250 ms window makes 2,207 gap moves under GCC and 23,848 under LBM, roughly 3×10^{-5} and 4×10^{-4} of a single rotation, and the gap starts at the top of the region while these workloads occupy low addresses. Running the matrix with the decoder off therefore reproduces the primary run's wear statistics and latencies *identically*: row-level maximum writes 1,059, 3,072 and 40,704 and hot-spot factors 2.77, 1.06 and 1.20 on GCC, LBM and the AlexNet output map, the same with the decoder on and off (`results/system_rev2026-09_decoder_default`). At the measured move rates, 8,828 per second under GCC and 95,392 under LBM, one rotation of the 67,108,864-line region takes about 7,600 s of simulated time under GCC (2.1 hours) and about 700 s under LBM (11.7 minutes), that is 30,000 and 2,800 windows of 250 ms. That is the expected outcome rather than a defect, and it is why this section shows no before-and-after hot-spot figure: inside any window this pipeline can simulate, leveling has not yet had time to do anything. The effect of leveling is carried entirely by the projection behind Table 5, whose model was independently validated against a write-by-write simulator to within 0.3%.

Three qualifications complete the picture. MLC does not "live longer" than SLC, and the claim in earlier versions that the selector variant outlives the transistor variant has been withdrawn: it rested entirely on the slower configuration completing fewer writes inside a service-limited window, and now that every ReRAM track completes the identical request population, all four wear at the same rate. MLC's real endurance cost is its programming: the multipliers this book applies (3.263x write, 1.5x read) are *per-bit throughput ratios* measured on the EMBER macro [6], [31], and per fixed 48-cell word the same data gives 6.5x for writes, which is carried as a sensitivity rather than as the headline. Finally, the projection charges one full line write per

request; real systems reduce that with write coalescing, data-comparison write and error correction, and a 4.5x write-reduction axis is carried in the underlying table for that reason. It moves every lifetime in Table 5 upward by that factor and does not change any of the orderings above.

3.1.5 Design Robustness: ReadVoltage Sensitivity

A persistent concern with resistive memories is parameter sensitivity: do the efficiency conclusions hold if the actual fabricated device deviates from the target resistance? To quantify this, I ran a three-point sweep of the NVSim ReadVoltage parameter at −20%, nominal, and +20% of the 1.4V design point ({1.12V, 1.40V, 1.68V}) for the 1T1R SLC Full DIMM configuration under LBM and GCC, the two workloads that bracket the bandwidth-bound and compute-bound extremes.

The results, shown in Figure 19, reveal a key robustness property: read latency is completely invariant to a +/-20% ReadVoltage perturbation. The current-sensing amplifier operates within its saturation margin at all three voltages, producing identical read latency (32.13 ns at the pre-revision organization) regardless of supply. Only read energy scales with applied voltage: read energy is 22.4% lower at 1.12V and 27.5% higher at 1.68V relative to the 1.4V baseline. At the full-DIMM system level, where static peripheral leakage dominates over per-operation read energy, the per-read energy variation is negligible: total PDP changed by less than 4% across the full +/-20% sweep. One limitation applies, and it is now a double one: this sweep was executed before the power-model repair of Section 3.1.6 *and* before the matched 2048 x 2048 organization of this revision, and it has not been re-run under either. Its absolute numbers therefore belong to the pre-revision dataset and are not comparable with Tables 2-5. Its qualitative conclusion survives both changes a fortiori: the static share of the swept module's power is 99.96% at the corrected organization (Section 3.1.2), so per-operation read energy is an even smaller fraction of the total than it was, and read-latency invariance is a sense-amplifier property unaffected by either change.

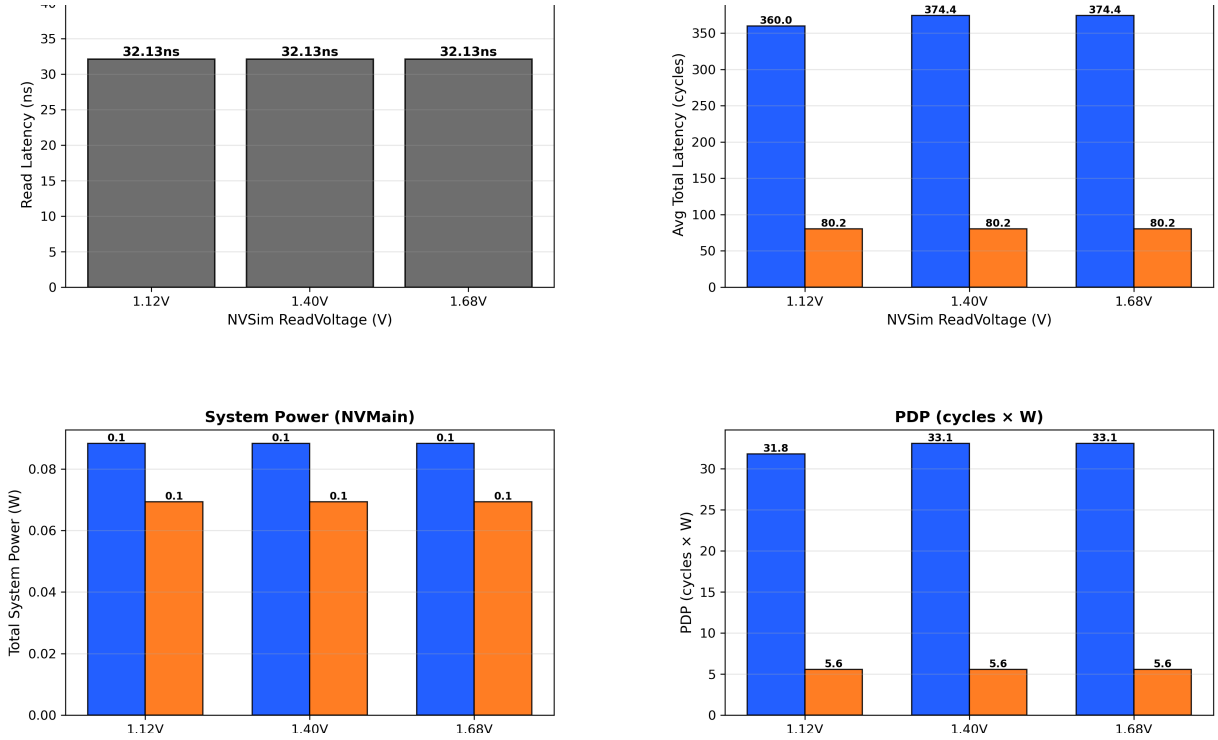


Figure 19: ReadVoltage Sensitivity Analysis - 1T1R SLC Full DIMM (LBM and GCC, +/-20% sweep), a 2 x 2 grid. Top left: NVSim circuit read latency, invariant at 32.13 ns across all three voltages. Top right: system average total latency. Bottom left: system power. Bottom right: Power-Delay Product, whose largest movement across the full sweep is 3.8% (LBM, 31.80 against 33.07 cycles x W at the 1.40 V design point), so efficiency conclusions are robust to process variation in the ReadVoltage operating point.

Two things to read carefully here. First, the invariance claim is a ***circuit*** *claim, and only the top-left panel makes it: the system average total latency in the top-right panel is* not *invariant, because LBM moves 360.0 cycles at 1.12 V against 374.4 at 1.40 V and 1.68 V. That is a queueing difference at the memory controller, not a device one, and it is the reason the PDP moves at all. Second, this is the one figure in this book that is* ***not*** *rendered from the primary dataset of this revision, and it is retained deliberately. The ReadVoltage sweep is a separate NVSim experiment (*`results/sweep_voltage_results.json`*) rather than a view of the matched-window matrix, and it was not re-run at the 2048 x 2048 organization; regenerating it would require re-running the sweep, which this revision did not do. What the chart shows is therefore the pre-revision organization, as the paragraph above states, and it is kept because the property it demonstrates - that circuit read latency does not move with ReadVoltage and that system PDP moves under 4% - is a sense-amplifier property that the organization change can only strengthen, since the static share of module power rose to 99.96%. No absolute value in this figure should be read against Tables 2 to 5.*

3.1.6 Simulation-Fidelity Audit: Where the Toolchain Could Not Be Trusted - and How It Was Repaired

A systematic audit of the NVSim-to-NVMain flow, conducted against the raw simulator sources and statistics files, found fourteen silent failure modes that bounded which conclusions this evaluation could honestly draw - across the ReRAM configurations, the metrics pipeline, and both non-ReRAM baselines. Thirteen of the fourteen have since been repaired in this project's toolchain, with the affected simulations re-run; every power and PDP figure in this book derives from the resulting repaired dataset. (1) Found and fixed: the pipeline's original efficiency metric multiplied Watts by cycle counts taken from different clock domains (800 MHz ReRAM vs. 2400 MHz DDR5), biasing the comparison 3x against DDR5; all PDP values are computed in physical W·ns. (2) Found and fixed: the StandbyPower parameter written by the configuration generator was dead configuration - NVMain's source contains no reference to it, so the device-characterized leakage never reached the simulation. The repair maps NVSim's per-technology leakage onto the standby-energy parameters NVMain actually reads. (3) Found and fixed: in place of real leakage, NVMain applied generic standby-energy defaults (Eactstdby/Eprestdby), giving every non-volatile configuration the same ~68 mW-per-rank static floor, so the original power results were technology-blind in their static component. Under the repaired wiring the simulated full-DIMM static floor reproduces the NVSim characterization exactly (0.0% anchor error): 6.937 W for both ReRAM cell types at the matched organization, against 64 chips at 108.384 mW. A second defect sat behind this one and was found later: the two cells were being characterized at different array organizations, which made their NVSim leakage figures differ by two orders of magnitude. That is corrected by the forced matched organization of this revision, and the superseded figures are tabulated in Appendix D. (4) Found and fixed: the reported “Total System Power” was the single most-loaded rank, not the module sum; because the ReRAM configurations span 8 ranks while DDR5 spans 2 ranks across 2 sub-channels, per-rank accounting distorted every cross-technology comparison. All power figures in this book are whole-module sums for every technology. (5) Found and fixed: NVMain's power-down state machine was disabled in source, so no idle-gating policy could be simulated for any technology - every ReRAM power figure in earlier drafts of this book was worst-case ungated, while the DRAM and PCM baselines modeled their standard idle behavior, an asymmetry this book previously disclosed but did not close. The dormant call site (`MemoryController::CycleCommandQueues()`'s `HandleLowPower()`) has been restored: it is a shared mechanism, so it activated for every technology at once, not ReRAM alone. DDR5 already carried real, JEDEC-datasheet-backed power-down currents, and now realizes a real, measured power reduction from them under GCC (Section 3.1.2). ReRAM has no equivalent real number: no NVSim datapoint decomposes its leakage into a gatable-periphery-vs-ungatable-crossbar split, and a dedicated literature search (Appendix A) found no citable ReRAM power-gating figure - so its power-down energy is set to an explicit, honest placeholder equal to its own existing standby energy (“assume no savings, pending real characterization”), which keeps the mechanism mechanically live (86-89% of full-DIMM cycle-slots are now power-gated, confirmed directly from the simulator's own counters) while changing no reported power number

for ReRAM, by design - the alternative (a fabricated non-zero savings figure, or the naive 0.0 J/cycle NVMain default) would have been worse than the disclosed ungated baseline this replaces. PCM shows no power-down activity at all in either run, most likely because its `FRFCFS-WQF` write-queue-flush controller keeps its request queue non-empty far more of the time than the plain `FRFCFS` controller ReRAM and DDR5 use, starving the power-down entry condition - flagged as an open follow-up, not yet root-caused to full confidence. (6) Found and *repaired in this revision*: trace provenance was weaker than the methodology narrative implied. The audit found that the gem5 traces carried no documented region of interest, no warm-up and a parser whose line-matching rule kept roughly four records for every real memory-controller request, and that their timestamps were being read on a time base the parser had never actually established. Every gem5 trace in this book has since been regenerated from scratch and revalidated. The runs are full detailed-mode O3 runs *with* L1 and L2 caches enabled, after 500 M fast-forwarded instructions, for 400 M (gcc) or 300 M (lbm, STREAM) detailed instructions; the first 10 ms of the detailed region is discarded as cache warm-up and the 250 ms replay window starts immediately after it. Timestamps are exact integer NVMain cycles on the CPUFreq 3000 MHz basis, and every trace ships a machine-readable provenance sidecar recording the gem5 command line, the raw-log hash, the region boundaries and a full accounting of every line read. Each regenerated trace reconciles *exactly* with gem5's own memory-controller request counters (kept plus skipped equals `readReqs` + `writeReqs`), so the earlier fourfold duplication is gone: gcc 545,566 records, lbm 10,779,425, STREAM 48,439,056. The three SCALE-Sim traces were regenerated with the same discipline, keeping every address and dropping SCALE-Sim's padding markers, which the old parser had written into the trace as real addresses. What remains unrepaired, and is therefore the one disclosed permanent limitation of this list, is the GPT-2 trace's generator configuration: the AlexNet traces are reproduced byte for byte from a known SCALE-Sim TPU-v1 256x256 output-stationary run, but the original GPT-2 run's configuration was never preserved, so that trace stands as a representative parallel-read pattern, not a validated GPT-2 capture. A fourth intended SPEC workload, 505.mcf_r, could not be traced at all: gem5 panics inside the benchmark's own input reader roughly 0.13 ms into the detailed region, identically on two independent attempts, so mcf is absent rather than approximated. (7) Found and fixed: the DDR5 configuration's refresh machinery was inherited from a DDR3-1333 template and never rescaled when the clock was raised to 2400 MHz - refresh operations fired 3.6x too often (derived tREFI 1.085 µs vs. JEDEC 3.906 µs) at 6.6x too little cost each (tRFC 44.6 vs. 295 ns). Recalibrating to JEDEC JESD79-5 raised DDR5's refresh share from 57.9% to 71.3% of module power under GCC (a share subsequently repriced by item 8's vendor-current calibration, by item 5's restored idle-gating, by this revision's corrected 16 GiB module and finally by the corrected background-power accounting of Appendix A, settling at the 6.5-9.1% band of Table 3) and raised its average GCC latency, because refresh blocking is now faithfully priced. (8) Found and partially fixed: every supply parameter in the DDR5 configuration proved to be NVMain's stock built-in default rather than a datasheet value. The supply voltage - 1.5 V against DDR5's JEDEC-specified 1.1 V - was corrected, scaling all DDR5 power linearly by 0.733 (verified exact); the IDD current magnitudes were subsequently calibrated to published

vendor datasheets (Micron and SK hynix 16 Gb DDR5-4800 tables) [29], [30], run as a two-vendor band - the conservative SK hynix floor is this book's headline, the Micron ceiling is reported alongside - with a documented caveat that datasheet IDD values are specification limits rather than typicals (vendors publish no typical DDR5 IDD figure to use instead, Section 3.1.2); calibrating with real currents also exposed and forced the repair of a sign artifact in NVMain's activate-energy formula (clamped at zero, worst case 2.8% of module power before the fix). (9) Found and fixed: the PCM baseline's cycle timings were derived for its cited 400 MHz basis, but a stray trailing clock override ran it at 800 MHz - twice as fast as Lee et al.'s parameters intend - and a second instance of the same bug sat in the reporting layer's hardcoded clock table; a syntax typo additionally zeroed its per-bit write energy. Restoring the 400 MHz basis roughly doubled PCM's latencies (e.g., LBM 3,732 to 7,465 ns) and corrected its power to 0.036-0.044 W. (10) Found and fixed: the configuration generator wrote ReRAM access energies under key names NVMain never reads (ReadEnergy/WriteEnergy vs. the live Erd/Ewr) - so every ReRAM configuration had silently used identical stock access-energy constants for the pipeline's entire history, despite NVSim characterizing a 5.7x read-energy difference between the families. Wiring the live keys and re-running the 96-run ReRAM matrix left every static floor, latency, and endurance counter identical while making dynamic power technology-differentiated for the first time (hand-reconciled against access counts to four significant figures). A definitive key-liveness audit of every generator-written key against the simulator's parameter readers closed this bug class permanently.

The repair was validated before its results were adopted. Pilot runs confirmed the leakage wiring end-to-end: at the matched organization the ReRAM full-DIMM configuration measures 6.937 W of static power against a 6.937 W device-derived target of 64 chips at 108.384 mW (0.0% error), with real dynamic power resolving to a few milliwatts under GCC, confirming that ungated ReRAM power is, to three significant figures, leakage. The full 120-run matrix (20 configurations × 6 workloads, including all 96 ReRAM runs) was then re-executed under the repaired model with anchor checks passing, and the endurance write counters were verified bit-identical to the pre-repair runs - the repair changes power accounting, not timing. The DDR5 baseline repairs were validated by the same discipline: the recalibrated refresh model produced exactly the predicted 21,333 refresh events per bank over the 200M-cycle window, the voltage correction reproduced the predicted linear scaling to within 4×10^{-6} relative deviation, and all non-DDR5 rows remained byte-identical across re-runs. The final calibration cycle added an independent, blind re-verification pass (a fresh session re-deriving every check from configs, sources, and raw statistics), which validated the DDR5 calibration end-to-end, refuted one suspected regression, and surfaced item (10). (11) Found and fixed: the configurations assumed heterogeneous host CPU frequencies - CPUFreq 800 MHz for every ReRAM configuration, 2 GHz for PCM, 3 GHz for DDR5 - and NVMain admits trace timestamps against a (CPUFreq/CLK)-scaled cutoff without rescaling them, so ReRAM was replayed under ~3.75x slower wall-clock request arrivals than DDR5 and each clock domain saw a different admitted trace population. Repaired by fixing CPUFreq = 3 GHz for every configuration and matching every cycle budget to the same and matching every cycle budget to the same wall-clock admission

window, so that every technology now processes the identical request population per workload, verified exactly. (The window length itself was corrected separately, once the traces' time base was established: it is 250 ms, and the figure this book previously quoted is tabulated in Appendix D.) The correction moved exactly what its direction analysis predicted: admission-sensitive compute-bound latency rose, trace-limited AI workloads did not move, static power did not move (anchors exact to the last digit), and endurance write pressure rose toward each configuration's service ceiling. The absolute figures that correction produced were later superseded in turn by the read-path and trace corrections of this revision, and are tabulated in Appendix D. Exact-config snapshots now ship with every results generation, closing a reproducibility gap in which on-disk configurations had drifted from the state that produced banked results. The residual limitations are item (6), the no-savings placeholder that item (5) leaves in place of a real ReRAM power-down energy, and the spec-limit character of vendor IDD tables: ReRAM totals claim no gating benefit, absolute queueing magnitudes inherit the open-loop-replay caveat - and, more broadly, the pipeline is open-loop trace replay with no CPU or accelerator feedback path, so every latency reported here is a memory-subsystem quantity: every AI-inference ratio in Section 3.1.1 is a memory-latency ratio, not a projected application slowdown, and latency-tolerant accelerators that overlap compute with memory access would experience a smaller end-to-end penalty; additionally, the completed-request and trace-player mechanism audit that exposed item (11) confirms that, after the matched-host correction, every configuration admits the identical request population per workload: GCC and the four lighter traces complete in full for every technology, so their latency averages compare identical populations, while LBM - the one genuinely service-limited workload - is completed to a technology-dependent depth (the sustained-throughput ordering of Section 3.1.1), so its latency averages cover each configuration's completed prefix, and DDR5 power is reported as a two-vendor calibration band. Within those bounds, the cross-technology power and PDP comparisons that earlier drafts of this work withheld are restored in Sections 3.1.2, 3.1.3, and 3.3.

(12) Found and fixed: the DDR5-4800 baseline's tCAS/tRCD/tRP timing parameters (34-34-34 cycles) were unsourced placeholders with no attached citation or datasheet reference. Cross-referencing SK hynix's public DDR5 SDRAM and DIMM part-number decoders (which enumerate per-speed-grade CAS-latency codes) against the standard DDR5-4800 (non-3DS) speed bin identifies the real value as 40-39-39, not 34-34-34 - a 15.7% increase in the CAS+RCD+RP component of DDR5's timing envelope. [10] (the primary JEDEC JESD79-5D standard) remains access-gated and could not be consulted directly; the SK hynix decoder is the best available public grounding. The DDR5-4800 configuration was corrected and the full 6-benchmark trace suite re-run; DDR5's total latency rose 2.3-8.6% across the suite (diluted from the 15.7% component-level change once blended with the timing parameters this fix did not touch - tRAS, tWR, tRFC, and refresh), while DDR5's power was unaffected, since this fix touches only timing, not the IDD/energy calibration of item (8). The parameters this item left untouched were themselves DDR3-1333 template cycle counts, and they have since been corrected to JESD79-5 as well, together with the power-down and activate-window timings; that

correction, which came after the audit list was closed, is recorded in Appendix A under “DDR5 timing and power corrections of this revision” and in Appendix D.

(13) Found and fixed: the MLC read/write latency and energy penalty multipliers - applied analytically on top of NVSim's SLC characterization, since NVSim itself has no multi-level-cell model - were unsourced 3x/4x placeholders that earlier drafts of this book attributed to "EMBER Macro analytical heuristics" not actually present in either EMBER publication: the cited conference paper (Upton et al. [6], ESSCIRC 2023) reports no write-verify data split by bits-per-cell (it does give an aggregate, non-split SET/RESET pulse-energy estimate, but nothing usable as a 1-vs-2-bit/cell multiplier). A literature search for the real multipliers - including two independent research-assistant passes, both of which initially overreached with unverifiable claims that had to be retracted under direct primary-source checking - located the actual measured figures on the same EMBER macro's full journal publication (Levy et al. [31], IEEE JSSC 2024, Section V): read energy 1.0/1.1 pJ/bit at 1/2 bits per cell (from the ESSCIRC precursor's own Table I), write-verify bandwidth 12.4/3.8 Mbps and write-verify energy 0.40/1.2 nJ/bit at 1/2 bits per cell (from the JSSC follow-up, which the ESSCIRC paper's page budget omitted entirely). At this stage the read-latency multiplier was set to 1.917x, derived by pairing EMBER's own 12 ns read time with a “23 ns” figure read off the same ESSCIRC Table I - a derivation later found to be itself in error (see item (14)). The write-latency, read-energy, and write-energy multipliers - 3.263x, 1.1x, 3.0x respectively - replace the unsourced 3x/3x/3x set and remain correct; write energy happens to be numerically unchanged (3.0x both times), while the read-energy correction is the largest single change of the three (3.0x to 1.1x). All 8 MLC configurations were re-simulated across all 6 benchmark traces under the multipliers as understood at this stage (Sections 3.1.2 and 3.1.3, in their original form, detailed the resulting per-workload comparisons against SLC; those sections now reflect item (14)'s further correction).

(14) Found and fixed: item (13)'s 1.917x read-latency multiplier was itself a data error, caught by an independent citation-verification pass that read Upton et al.'s [6] Table I directly rather than trusting the prior derivation. The table's “Read Time (1 b)” row lists five macros side by side - EMBER (This Work) at 12 ns, and four competing designs, including reference [9] (T. F. Wu et al., ISSCC 2019) at 23 ns - and every entry in that row, including EMBER's own, is explicitly scoped to *1-bit/cell* operation. The “23 ns” is [9]'s own 1-bit/cell read time, not EMBER's 2-bit/cell number; EMBER's ESSCIRC paper never reports a 2-bit/cell read latency at all. Re-deriving from EMBER's own data: the JSSC 2024 follow-up (Levy et al. [31], Section V.A and Abstract) states “1 b/cell read operation with 1.0 pJ/bit energy at 2.4 Gbps, and 2 b/cell read with 1.1 pJ/bit at 1.6 Gbps” - giving a clean 1.5x read-latency multiplier (2.4/1.6 Gbps) via the same bandwidth-ratio methodology already used for the write-latency multiplier (12.4/3.8 Mbps -> 3.263x). The corrected 1.5x is milder than the erroneous 1.917x, not harsher: MLC read latency was previously over-penalized. All 8 MLC configurations were re-simulated across all 6 benchmark traces under the corrected multiplier; MLC latency fell a further 5.5-18.6% and MLC PDP fell 0.7-18.6% relative to the item (13) dataset (Sections 3.1.2 and 3.1.3 reflect the corrected figures

throughout; Section 3.1.4's LBM endurance lifetimes were likewise recomputed, since faster reads let more total requests - including writes - complete within the fixed simulation window).

A dedicated NVSim sensitivity sweep, prompted by a related citation question over the HRS resistance target (Appendix A), separately confirmed that modeled leakage is insensitive to that target across the more than two orders of magnitude of HRS it spans. In hindsight that result was the first evidence that NVSim's leakage figure is not a cell property at all; see Appendix A for the sweep and for what it turned out to be telling us.

3.2. Architectural Scaling: Chip Count, Channels and Ranks

Whether a technology can replace main memory is partly a question about one module and partly a question about how that module scales. This section takes the second question directly: it replays all six traces through four points of the same ReRAM architecture - one chip, eight chips, sixteen chips and the 64-chip full DIMM - and asks what each added chip buys. The Pareto figures below plot module power against average total latency along that trajectory; Table 8 gives the same data as numbers.

The four points are not four sizes of one thing, and the differences matter more than the chip count does. **One chip** is a single 1 Gb device on a 64-bit bus, one rank, one channel. **Eight chips** is eight x8 devices forming the same 64-bit bus, still one rank and one channel, and it is the smallest of the four whose decoded capacity equals the silicon it contains: 1 GiB of SLC. **Sixteen chips** splits those ranks across *two* channels, one rank each, matching the two subchannels of the DDR5 baseline (Section 2.3). **Sixty-four chips** is four ranks on each of two channels. So the step from eight to sixteen chips is where the second channel arrives, and the step from sixteen to sixty-four is where rank depth arrives; no step in this matrix isolates chip count by itself. Module capacities are summed from the simulator's own per-channel capacity prints: 1 GiB at eight chips (one channel of 1 GiB), 2 GiB at sixteen (two channels of 1 GiB) and 8 GiB at the full DIMM (two channels of 4 GiB) for the SLC tracks, and twice each of those for MLC, the second bit per cell doubling the module without changing the chip count.

The one-chip point is reported for continuity with earlier versions of this book and should not be read as an architecture. Its decoded address space is a floor value the configuration generator imposes, not a physical module: a single 1 Gb chip is decoded as 32 GiB, 256 times the storage the chip actually holds, because the generator keeps a minimum row count for the one-device case (Appendix A, "Module Capacity and the Address Footprint"). Its row count, and therefore its row-conflict behavior, belongs to no real part. Every conclusion below is drawn from the three physical points.

Table 8: Chip-count scaling, 1T1R SLC, average total latency (ns) and Power-Delay Product (W·ns) at each architecture, 250 ms matched window.

Workload	1 chip lat.	8 chip lat.	16 chip lat.	64 chip lat.	1 chip PDP	8 chip PDP	16 chip PDP	64 chip PDP
GCC	44.7	45.4	44.8	41.4	5.0	39.5	77.8	287.3
LBM	43.3	48.1	44.9	43.1	6.0	43.2	79.2	300.4
STREAM	43.3	45.1	44.0	42.3	6.2	40.7	77.8	294.5
GPT-2 IFMAP	223.1	240.6	143.3	143.3	75.7	250.9	299.2	1,044.7
AlexNet IFMAP	206.3	241.2	143.6	143.6	68.6	251.6	299.3	1,046.5
AlexNet OFMAP	224.2	391.3	352.1	353.1	61.7	375.0	657.6	2,496.2

Source: `results/rev2026-09_primary_csv/processed_bar_chart_metrics.csv`, rows `Technology = 1T1R_SLC`. Module power at the four points is 0.111, 0.870, 1.737 and 6.939 W under GCC, of which the static component is exactly 1, 8, 16 and 64 times the 108.384 mW per chip of Table 1. The other three ReRAM tracks have the same shape to within a few percent and the same power, and are omitted for width; the one-chip column is a non-physical decode and is not used in any conclusion.

Static power scales exactly with chip count, with no exceptions and no sub-linearity to hope for. The static component of module power at the four points is 0.108384, 0.867072, 1.734144 and 6.936576 W, which is 1, 8, 16 and 64 times the per-chip leakage of Table 1 to six digits, and it is identical for all four ReRAM tracks. Dynamic power does not follow that law and does not need to: it moves by under 2 mW across the three CPU trajectories (2.84 to 2.88 mW under GCC, 30.58 to 30.82 under LBM) and by at most 0.18 W on the AI bursts (175.7 to 354.1 mW under GPT-2 IFMAP, 175.9 to 350.0 under AlexNet IFMAP, 91.2 to 166.9 on the output map), so it never displaces the static term: even under GPT-2 IFMAP the full DIMM is 6.94 W static against 0.35 W dynamic. Total module power is therefore *near*-linear rather than exactly linear (0.111, 0.870, 1.737, 6.939 W under GCC against an exact 1:8:16:64 on the static part alone). Capacity does scale exactly with chip count. So the dominant power cost and the capacity benefit of scaling are both linear, and every interesting question is about what happens to latency.

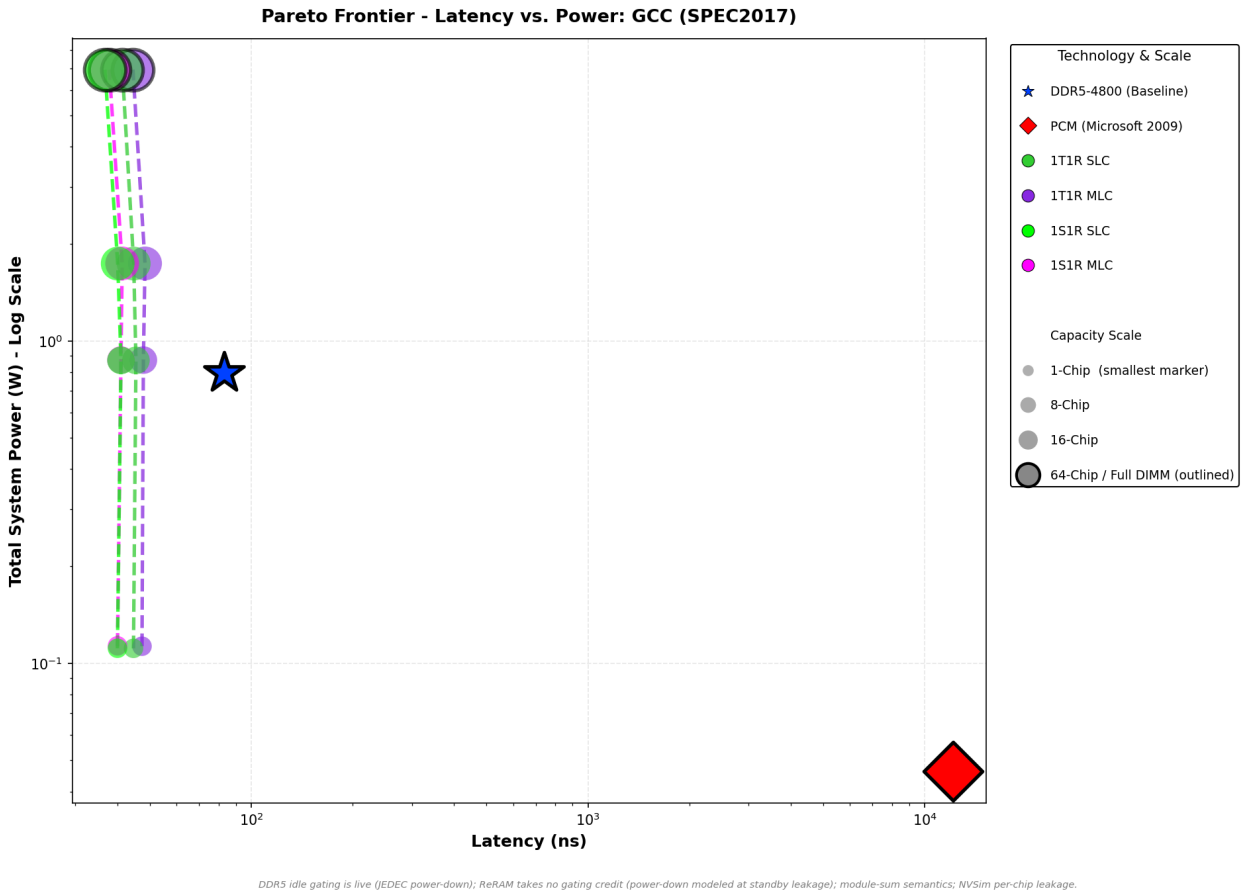
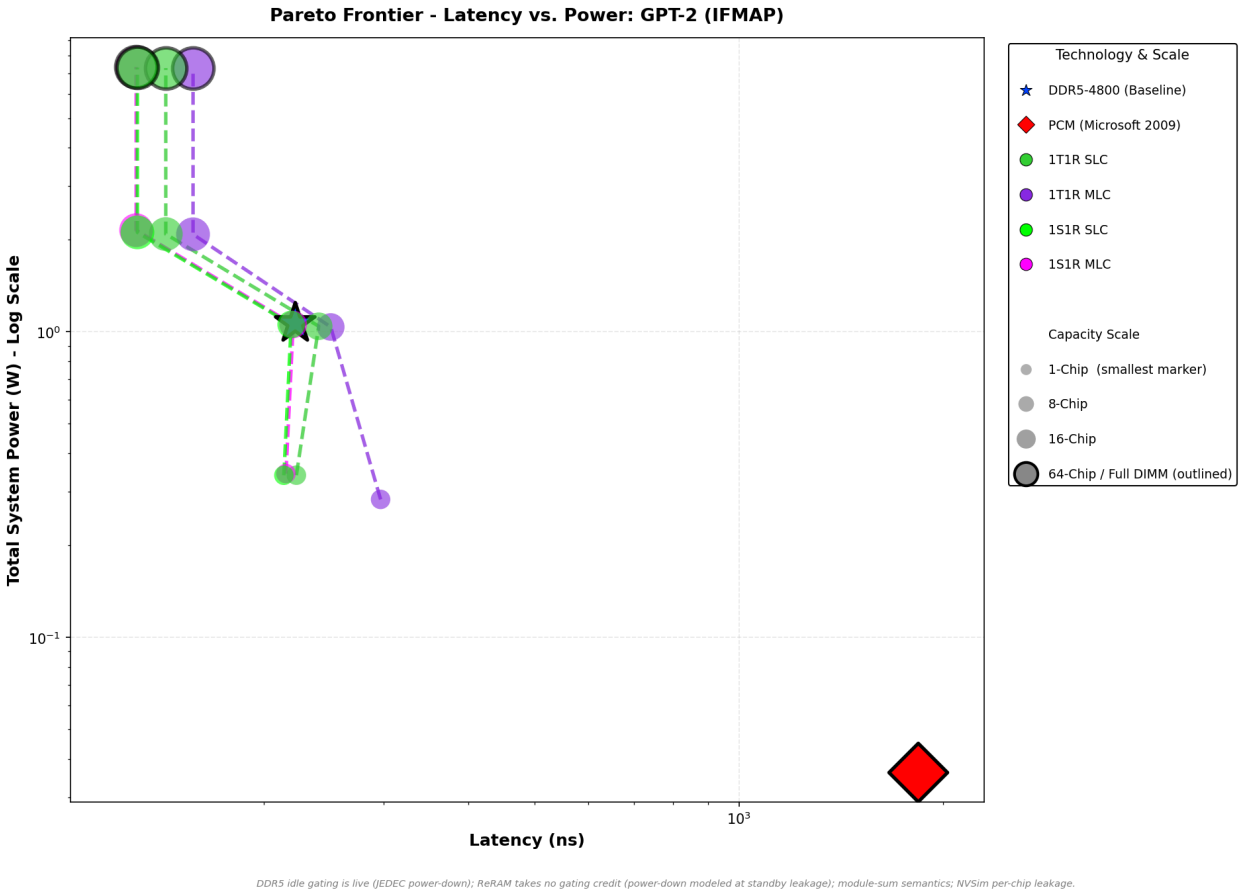


Figure 20: Pareto scaling trajectory under the single-threaded GCC workload.

Regenerated from the primary dataset of this revision and in agreement with Table 8, all four architectures plotted. The leftmost, lowest-power point of each trajectory is the single-chip architecture, whose decoded capacity is a non-physical floor the configuration generator imposes (32 GiB from one 1 Gb device, Table 8 note and Appendix A); it is plotted for continuity with earlier versions and enters no conclusion in this book.

For the three CPU-class traces, scaling chip count buys very little in latency. Across GCC, LBM and STREAM, 1T1R SLC moves from 45.4, 48.1 and 45.1 ns at eight chips to 41.4, 43.1 and 42.3 ns at the full DIMM, an improvement of 8.9%, 10.4% and 6.4% across the three physically realizable points. The widest spread across all four architectures is LBM's 11.6%, and the trend is not monotonic, because the eight-chip point is the slowest of the four on all three traces while the non-physical one-chip point is among the fastest. Either way the movement is smaller than the difference between the two access devices (41.4 against 36.8 ns under GCC). The reason is visible in the delivered-bandwidth column of the same runs, which is *identical* at all four architectures: 133.0 MB/s under GCC, 1,424.6 under LBM and 1,534.6 under STREAM. These traces are offered-rate limited, not device limited. The queue component of their latency is 1.3 to 4.6 ns out of 41 to 48 ns, so there is no backlog for extra ranks or an extra channel to relieve. What the modest full-DIMM improvement *is* is the device part of the latency falling as

rank depth spreads traffic over more banks, and that part is measurable: across the sixteen-to-sixty-four-chip step, which holds channel count at two and raises ranks per channel from one to four, `HW_Latency_ns` falls from 43.4 to 40.1 ns under GCC and total latency falls 7.5% (GCC), 3.9% (LBM) and 4.0% (STREAM). That is what rank depth is worth on this suite. Against it, the full DIMM costs 62 times the one-chip power and 8 times the eight-chip power. The Pareto trajectories for these three traces (Figures 20, 24 and 25) therefore run almost vertically: power climbs an order of magnitude while latency moves a few percent. Scaling a ReRAM module of this design buys capacity, pays leakage proportionally for it, and buys a single-digit percentage of speed.



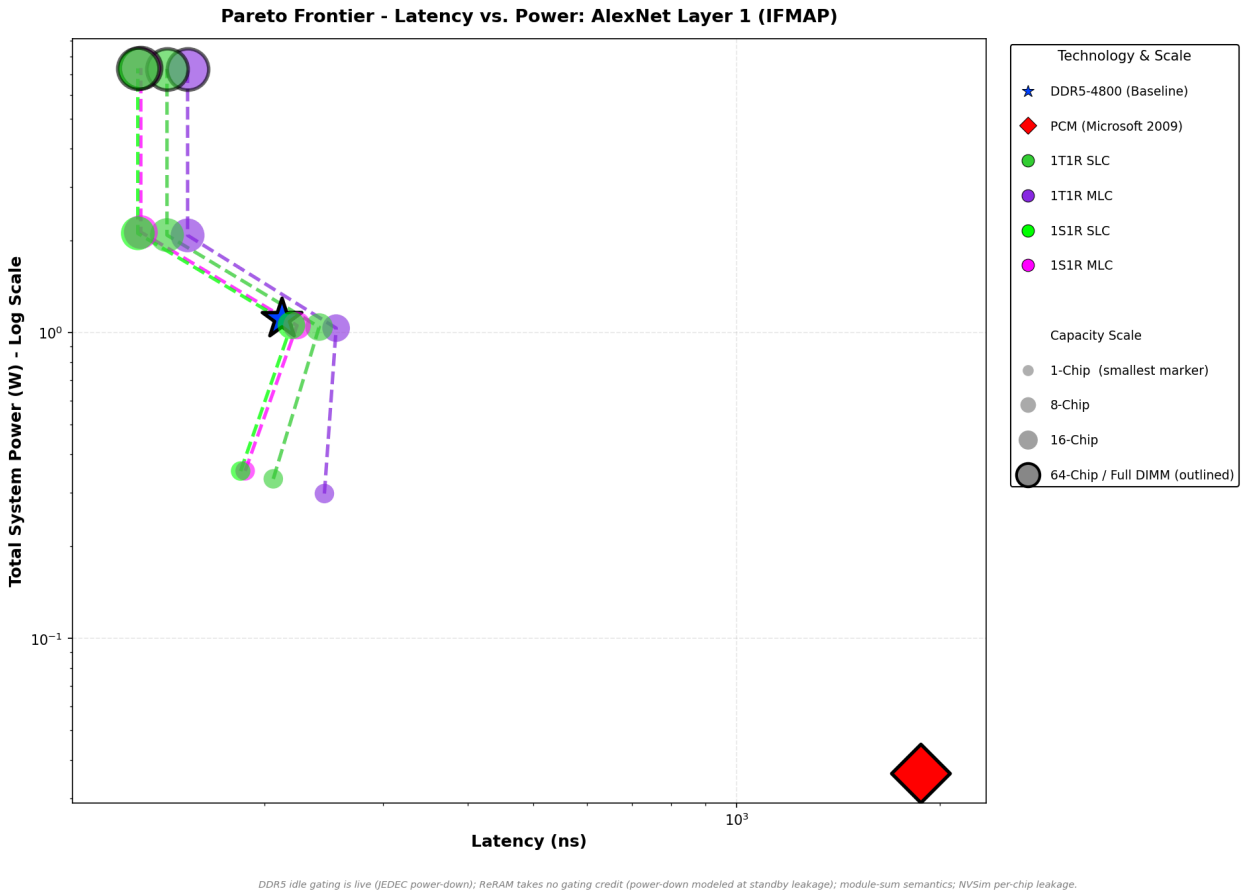


Figure 21 & 22: Pareto scaling trajectories under highly parallel AI inference workloads (GPT-2 IFMAP and AlexNet IFMAP).

Regenerated from the primary dataset of this revision and in agreement with Table 8, all four architectures plotted. The leftmost, lowest-power point of each trajectory is the single-chip architecture, whose decoded capacity is a non-physical floor the configuration generator imposes (32 GiB from one 1 Gb device, Table 8 note and Appendix A); it is plotted for continuity with earlier versions and enters no conclusion in this book.

The two AI read bursts do gain, by 1.68x, and a control run shows the gain is entirely the second channel. GPT-2 IFMAP goes 223.1, 240.6, 143.3, 143.3 ns across the four points and AlexNet IFMAP 206.3, 241.2, 143.6, 143.6. The eight-to-sixteen-chip step is worth 1.68x on both traces; the sixteen-to-sixty-four-chip step, which quadruples the rank count and the capacity at fixed channel count, is worth *exactly nothing* - the two figures are identical to every digit the CSV carries, as are their end-to-end latencies (106.31 µs for GPT-2, 2,096.09 µs for AlexNet) and their delivered bandwidths (19,565.5 and 19,249.5 MB/s). This is the corrected form of the "flatline" this book has argued about since the 3 September version, and it is a flatline in rank depth rather than in chip count.

Because the eight-to-sixteen-chip step doubles chips and channels together, that step was re-run with the channel count held at one. The control isolates it cleanly: at a single channel the eight-

chip, sixteen-chip and full-DIMM points collapse onto each other, 240.6 ns at all three under GPT-2 IFMAP and 241.2, 242.0, 242.0 under AlexNet IFMAP, with the queue term flat at 211.5 to 212.7 ns and the device term flat at 29.1 to 29.3 ns. Restore the second channel and the sixteen-chip and full-DIMM points drop to 143.3 and 143.6 ns, the queue term halving to 114.3 and 114.5 while the device term does not move at all (29.0 and 29.1 ns). **So the 1.68x is admission queueing at a second memory controller, not device parallelism, and rank count contributes nothing to it.** Doubling banks and ranks bought no service-time improvement whatever; halving the requests each controller must admit bought all of it.

The address footprint is why rank depth cannot help these two traces, and it is measured rather than assumed. Each rank of each channel covers 1,024 columns of 64 bytes, that is 64 KiB of consecutive lines, and because the channel field sits below the rank field in this configuration's address decode a two-channel module rolls into the next rank only after 128 KiB. Inside the 250 ms window GPT-2 IFMAP touches exactly 1,024 distinct 64-byte lines spanning 64.0 KiB and AlexNet IFMAP 1,094 lines spanning 68.4 KiB, both inside a single 128 KiB rank span; the three CPU traces touch 276,294, 1,648,067 and 3,248,128 lines over spans of 63.0, 345.4 and 229.5 MiB. (The three AI traces end inside the window, so for them the in-window figure and the whole-trace figure are the same; for the CPU traces they are not, and the values above are the in-window ones.) The simulator's own rank counters confirm the consequence exactly: at the full DIMM, GPT-2 IFMAP and AlexNet IFMAP put *every* request on rank 0 of each channel and leave ranks 1 to 3 with zero reads and zero writes, while GCC spreads across all eight ranks of the two channels within 12%, and LBM and STREAM within 0.3%. Rank depth is therefore unreachable for the two AI read bursts no matter how many chips the DIMM carries. The channel pair, by contrast, they do span: GPT-2's 65,535 admitted reads split 32,767 to 32,768 between the two channels at sixteen chips and again at sixty-four. The queueing term that split halves is 114.3 ns of the 143.3 ns total (Section 3.1.1), and nothing after that step touches it.

The write-dominated burst scales worst of all, and is the one place where added hardware makes the in-controller latency worse. AlexNet OFMAP goes 224.2, 391.3, 352.1, 353.1 ns. Between the three physical points the second channel is worth 1.11x and rank depth is worth nothing, or marginally less than nothing (353.1 against 352.1 ns, and 486.10 against 483.13 µs end to end). Its footprint, 2,116 distinct lines over 132.2 KiB, is the only AI footprint that crosses a 128 KiB rank boundary, and the rank counters show it accordingly: 27,008 writes on rank 0 of each channel and 40,703 and 40,704 on rank 1, with ranks 2 and 3 untouched. That is why it is the one AI trace that moves at all between sixteen and sixty-four chips, and it moves in the wrong direction. Meanwhile its PDP goes 61.7, 375.0, 657.6, 2,496.2 W·ns: a 40-fold efficiency loss between the one-chip and the 64-chip points, over which latency gets 57% worse, and a 3.8-fold loss between sixteen and sixty-four chips for a latency that gets 0.3% worse in the controller and 0.6% worse end to end. Write-intensive traffic on this design does not scale; it merely costs more.

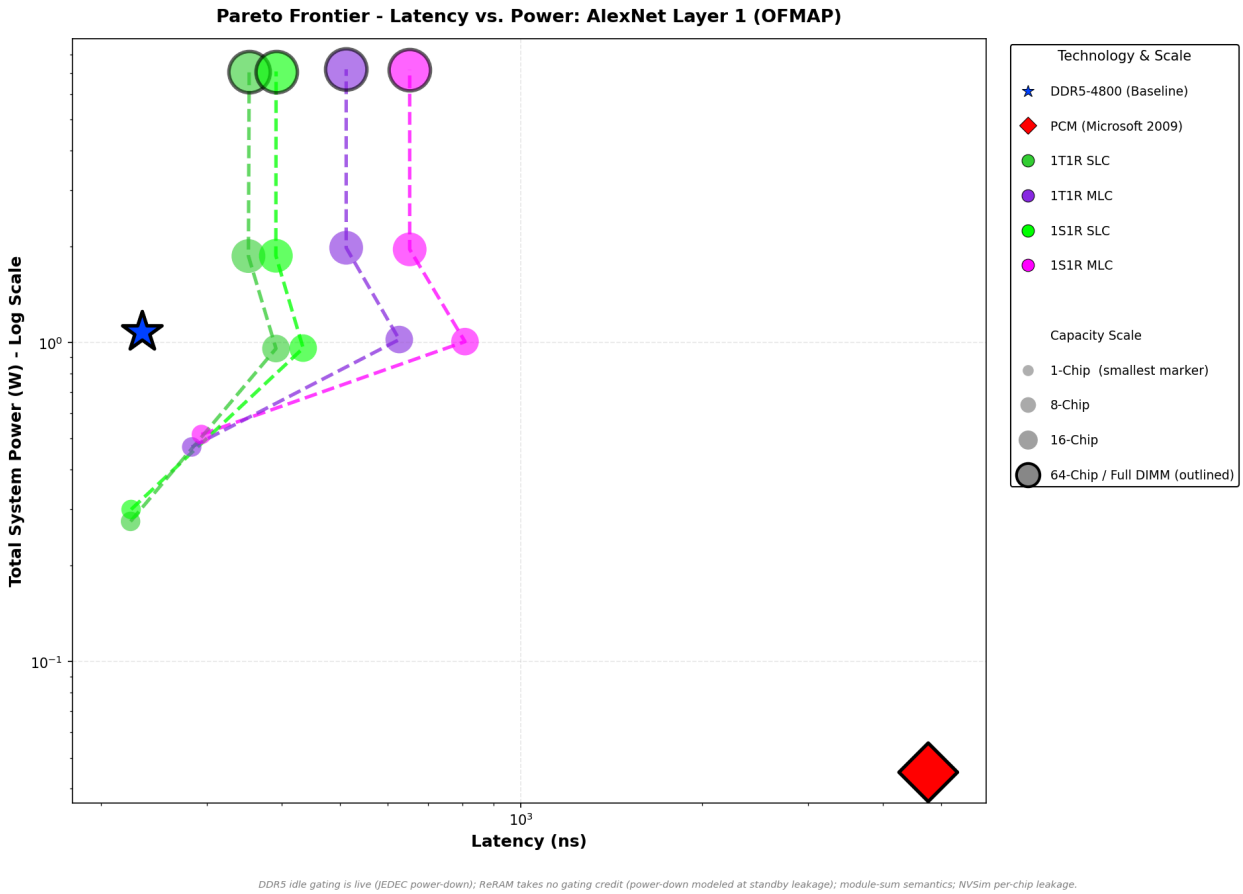
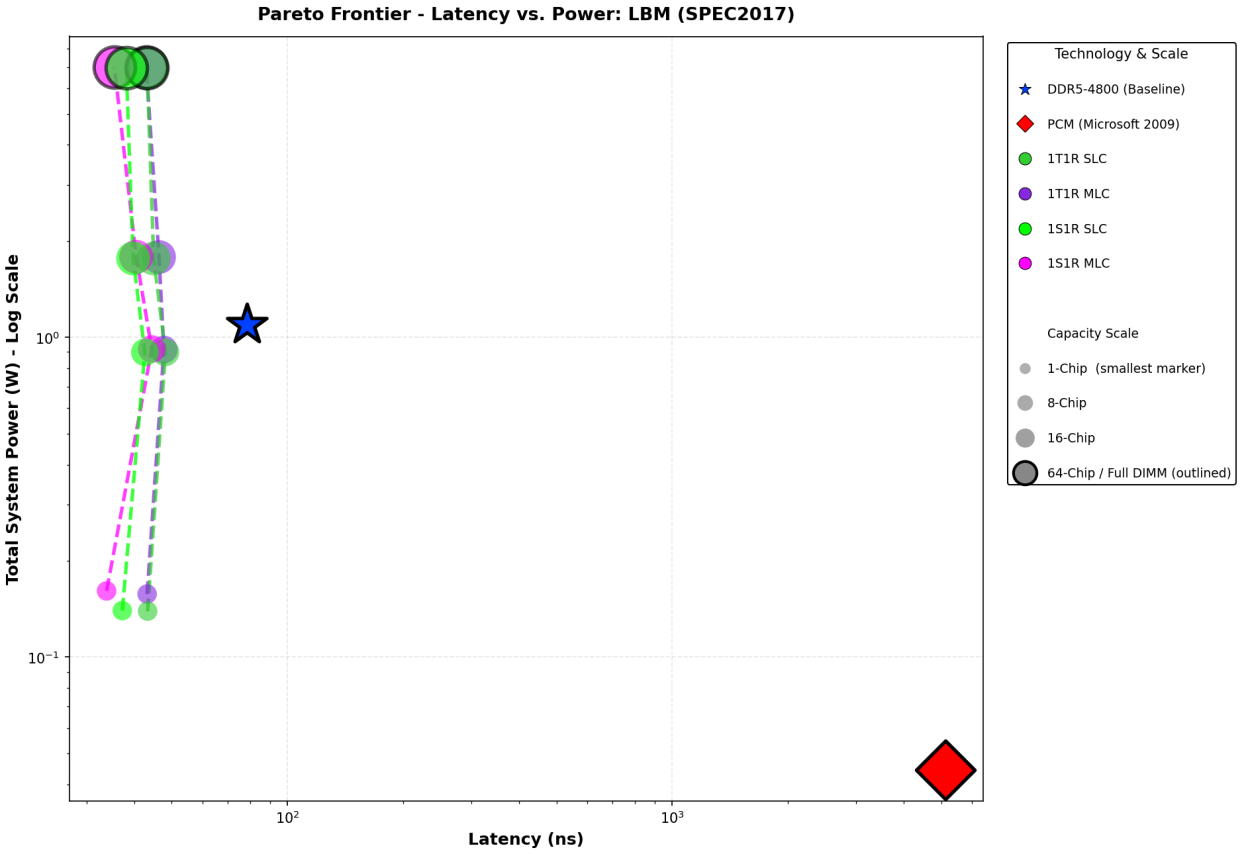


Figure 23: Pareto scaling trajectory under write-torture conditions (AlexNet OFMAP).

Regenerated from the primary dataset of this revision and in agreement with Table 8, all four architectures plotted. The leftmost, lowest-power point of each trajectory is the single-chip architecture, whose decoded capacity is a non-physical floor the configuration generator imposes (32 GiB from one 1 Gb device, Table 8 note and Appendix A); it is plotted for continuity with earlier versions and enters no conclusion in this book.



DDR5 idle gating is live (JEDEC power-down); ReRAM takes no gating credit (power-down modeled at standby leakage); module-sum semantics; NVSim per-chip leakage.

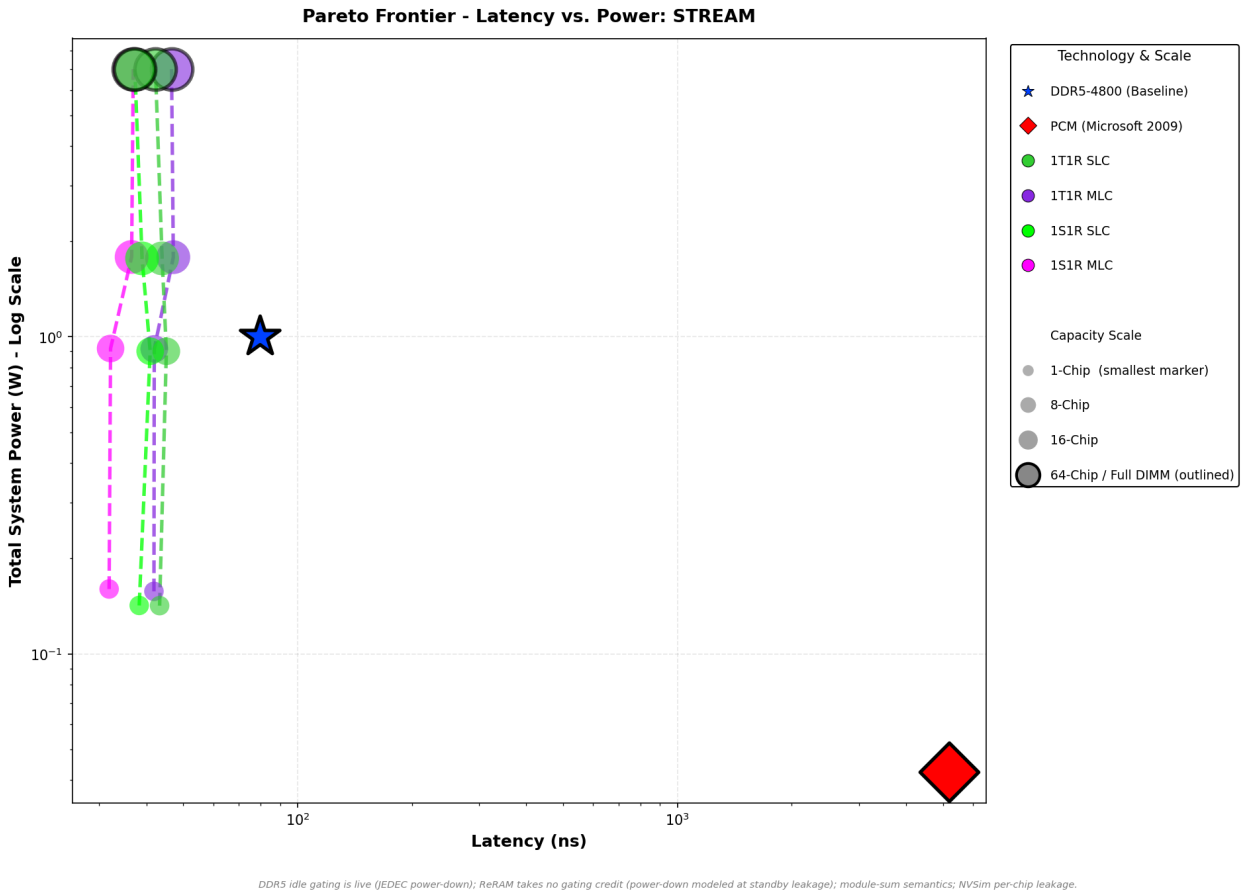


Figure 24 & 25: Pareto scaling trajectories under continuous memory-streaming workloads (LBM SPEC2017 and STREAM).

Regenerated from the primary dataset of this revision and in agreement with Table 8, all four architectures plotted. The leftmost, lowest-power point of each trajectory is the single-chip architecture, whose decoded capacity is a non-physical floor the configuration generator imposes (32 GiB from one 1 Gb device, Table 8 note and Appendix A); it is plotted for continuity with earlier versions and enters no conclusion in this book.

The channel attribution rests on two control runs rather than on the chip-count matrix alone, because within that matrix channel count and rank count move together. The first, reported above, re-runs the two AI input maps at one channel and collapses the eight-, sixteen- and 64-chip points onto a single latency. The second runs the full DIMM at one channel on GCC, LBM and the AlexNet output map, holding capacity and total rank count fixed and moving only channel count, and gives the same answer from the other side: on the CPU traces the second channel is worth nothing (1T1R SLC 41.5 ns at one channel against 41.4 at two under GCC, 42.6 against 43.1 under LBM), while on the write burst it is worth a clear amount (391.5 ns and 717.1 µs end to end at one channel against 353.1 ns and 486.1 µs at two, a 1.5x end-to-end improvement). Together they say the same thing from both directions: channels help exactly the traffic that is queue-bound, and only that traffic.

Array organization is a second-order latency choice and a first-order power choice. Every result above uses the 2048 x 2048 subarray organization. Re-running the pipeline end to end at 1024 x 1024 subarrays (the same 64:1 sense-amplifier multiplexing, 256 mats per chip instead of 64) moves the device in the expected direction and the module in an expensive one. NVSim reports a slightly faster array (read 9.306 ns against 10.120 ns for 1T1R, 4.453 against 4.702 ns for 1S1R), a larger die (13.545 against 12.008 mm^2 and 5.128 against 3.540 mm^2), and about twice the leakage: 224.138 and 220.224 mW per chip against 108.384 mW for both cells. Four times as many mats, each with half as many sense amplifiers, is twice the sense-amplifier count, and peripheral leakage follows it, which is the same mechanism that produced the retired leakage gap between the two cells (Appendix D). At the module the full DIMM gains 4 to 10% in latency (1T1R SLC 39.9 ns against 41.4 under GCC, 38.9 against 43.1 under LBM, 317.8 against 353.1 on the AlexNet output burst) and pays for it with 14.3 W against 6.9 W (1S1R SLC: 14.1 W against 6.9 W). The two cells still leak within 2% of each other at the finer organization, so the matched-organization conclusion of Section 3.1.2 does not depend on which organization is matched; the 2048 x 2048 choice is simply the lower-power one.

Three statements survive this section, and they are narrower than the ones it previously made. First, **the leakage and capacity cost of scaling is exactly linear in chip count** and is identical for the transistor-gated and the selector-gated families: at the corrected characterization both leak 108.384 mW per chip and both full DIMMs draw 6.94 W (Section 3.1.2), so a chip-count trajectory says nothing about the choice of access device. The two-leakage-tier picture earlier versions of this section plotted, and the claim that one family scaled into infeasibility while the other stayed inside a commodity envelope, are withdrawn and tabulated in Appendix D. Second, **the latency return on scaling is a queueing return, not a parallelism return**: it appears only where a workload arrives faster than one controller can admit it, and then it is bought by the second channel, not by rank depth. Among the module-organization choices this matrix varies - chips, ranks and channels - channel count is the one that moves burst latency. That is a statement about module organization only: the interface clock (2.2x on the CPU traces, Section 2.3) and the controller queue depth (1.6x end to end on the burst, Section 3.1.1) are separate levers of this book's own, and so is the choice of access device. Third, **rank depth bought at most 7.5% on any of the six traces**, and only on the three offered-rate-limited CPU traces, where it shows up as a fall in the device component of the latency (GCC 43.4 to 40.1 ns of hardware service time across the sixteen-to-sixty-four-chip step) rather than as relief of a backlog; on the three AI traces it bought exactly nothing, because their footprints never leave one rank. That last figure is a statement about this workload suite, not about ReRAM: a trace with a large footprint *and* a sustained arrival rate above one controller's service rate is exactly the case none of the six provides, and it is the case that would decide the question (Section 4.1).

3.3. Global Viability (Hero Graphs)

With the scaling dynamics established, the final evaluation must address the core question: does 22nm ReRAM present a globally viable alternative to DRAM?

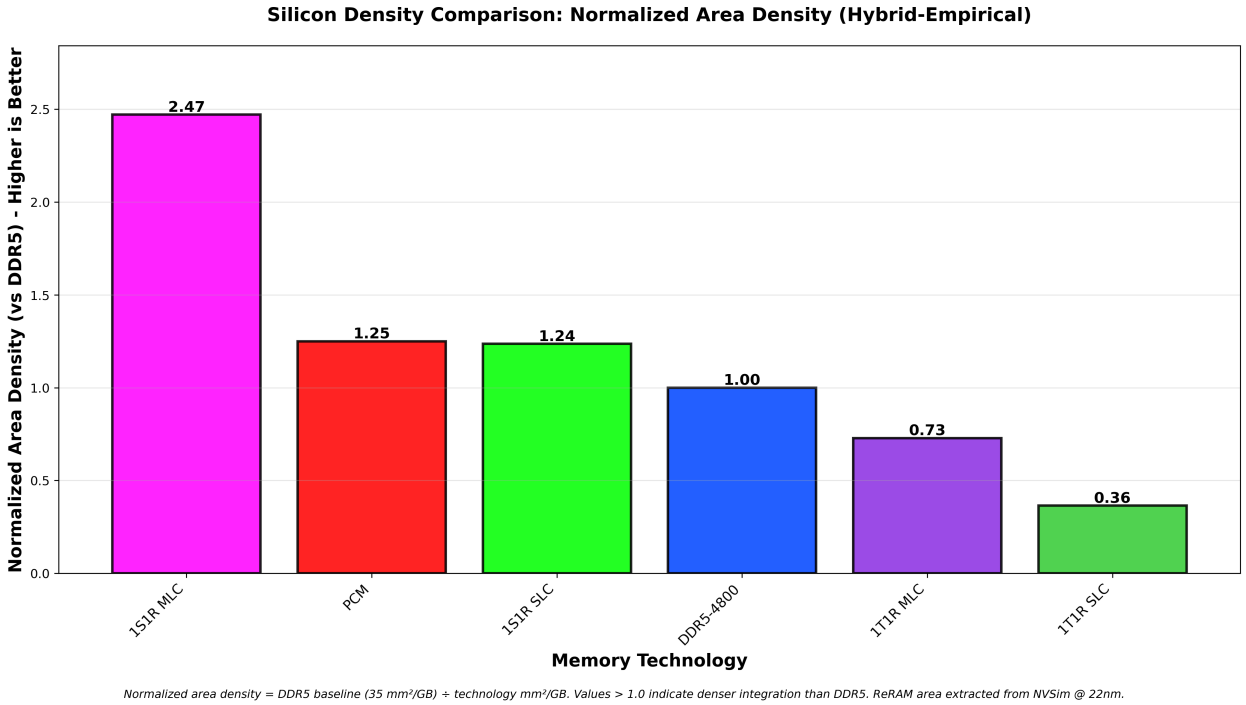


Figure 26: Die-Level Density Comparison, normalized to DDR5 = 1.00 (higher is better): NVSim-characterized die areas per GB versus a commodity DDR5 baseline of 35 mm² per GB [18].

Regenerated from the primary dataset of this revision and in agreement with Table 6: 1.24x (1S1R SLC), 2.47x (1S1R MLC), 0.36x (1T1R SLC) and 0.73x (1T1R MLC) at the matched 2048 x 2048 organization. The PCM bar, 1.25x, is ***not characterized****: it is a fixed constant in the metrics pipeline, inverted from an older 0.80 mm^2/GB placeholder and carrying no device or datasheet source, and it should be read as a placeholder rather than as a measured die-level density.*

To alleviate the severe memory supply-demand gap, MBMM must provide superior physical scaling, and density is best read at three levels. At the die level - what a module vendor can actually buy - Figure 26 normalizes NVSim's characterized die areas against a commodity DDR5 baseline of 35 mm² per GB (1.00) - a figure sitting inside the 33.1-37.6 mm²/GB band measured across Micron, Samsung, and SK hynix production 16 Gb DDR5 dies [18]: at the matched 2048 x 2048 organization the selector cross-point delivers **1.24x** DDR5's density as SLC and **2.47x** as MLC, while the transistor-gated 1T1R lands at 0.36x (0.73x as MLC). Both figures move substantially from the 3 September version (1.92x and 0.22x), for the same reason every other device-level number in this revision moves: those were measured at two different array organizations, which changed the periphery-to-array ratio on each die. A 20F² access-transistor cell fabricated at 22nm still cannot compete with 10 nm-class DRAM on area, but the margin against it is 2.7x rather than 4.5x. At the cell level, the comparison becomes node-independent and exact: DRAM stores one bit per 6F², 1T1R one per 20F² (Appendix A), and the selector cross-point one per 4F² (two per 4F² as MLC) - so at any matched process node, 1S1R MLC is

3.0x denser than DRAM and 1S1R SLC 1.5x denser, while 1T1R is 3.3x less dense under the planar-access-transistor assumption used throughout this book (a DRAM-process recessed-channel alternative closes this gap in principle but is not modeled here - Appendix A, Section 4.2). The characterized die-level advantage (1.24x) now falls *short* of the 1.5x cell-level bound rather than exceeding it, which is the expected direction and the more credible one: at a 2048-cell tile the cross-point die carries enough sense-amplifier, decoder and mux area that it gives back part of the cell-level margin. The earlier 1.92x figure, which exceeded the cell-level bound, came from a two-mat organization whose periphery was implausibly small. The third level is headroom: the die-level ratios are achieved across an asymmetric process comparison - DDR5 fabricated on 10 nm-class nodes (1γ [17]) with no remaining capacitor runway [5], the ReRAM figures from a mature 22nm logic-compatible process. A $4F^2$ cross-point cell scales quadratically with feature size: a port to a 12 nm-class node would multiply the density ratios by roughly $(22/12)^2 \approx 3.4$x before any stacking, and cross-point arrays additionally admit back-end-of-line 3D deck stacking [9], a lever planar DRAM does not possess. Node scaling compounds through the capacity law of Section 3.1.4, but only under *ideal* uniform leveling: a doubling of capacity doubles the ideal-leveling lifetime and does nothing at all for the no-leveling bound, so the density dividend converts into lifetime only to the extent that a wear-leveling scheme better than single-region Start-Gap exists.

Table 6 makes the third level concrete: the characterized die-level ratios of Figure 26, scaled quadratically to 16nm- and 12nm-class nodes, with the back-end-of-line deck-stacking lever that only the cross-point possesses.

Table 6: Projected die-level density versus DDR5 (= 1.00) under quadratic feature-size scaling and 3D deck stacking.

Config.	22nm (characterized)	16nm (projected)	12nm (projected)	12nm, 2 decks	12nm, 4 decks
1S1R SLC	1.24	2.34	4.15	8.31	16.6
1S1R MLC	2.47	4.67	8.31	16.6	33.2
1T1R SLC	0.36	0.69	1.22	N/A	N/A
1T1R MLC	0.73	1.38	2.45	N/A	N/A

Projection: measured 22nm die-level anchors (Figure 26) x $(22/F)^2$, with the DDR5 baseline held fixed at its 10 nm-class density per the end-of-roadmap capacitor argument [5][17]. Deck rows multiply the 12nm figure by 2 and 4; 1T1R cannot deck-stack - its access transistor is a front-end-of-line device.

These projections are bounded claims, not measurements: the 16nm and 12nm columns are geometry rather than NVSim characterizations, they optimistically assume peripheral circuits shrink with the cell array, and selector device physics below the 2x-nm regime is unvalidated. Every ratio in the table is also linear in the assumed cell area, and that sensitivity is worth stating

explicitly because the cell areas are assumptions, not measurements (Appendix A): demonstrated 1T1R endpoints span a 14nm FinFET embedded macro at 112F² [20] - which would drag 1T1R's 22nm ratio from 0.36x down to roughly 0.07x - and a DRAM-process recessed-channel part at 6F² [33] - which would lift it to roughly 1.2x; on the selector side, 4F² is the idealized floor, and a realized cell at twice that area would halve every 1S1R entry, dropping 22nm SLC *below* DDR5 parity (0.62x) while MLC retains 1.24x before node scaling restores the margin. That is a materially weaker position than the 3 September version described: at the corrected organization, the selector cross-point's SLC density advantage over DDR5 no longer survives a factor-of-two miss on cell area. The density verdict is therefore conditional on cell-area realization for *both* topologies, and MLC is what carries the density case. The counterpoint is equally real, and moving on three fronts: DRAM's escape from the planar capacitor is on the incumbents' public roadmap, with Samsung outlining vertical-channel-transistor 3D DRAM for the second half of the decade [19], and the challenger ecosystem is moving faster still - a monolithic 3D X-DRAM built on 3D NAND-style vertical processing passed proof-of-concept validation in April 2026, reporting sub-10 ns access and second-scale retention [25]. Notably, even that challenger's headline retention claim is a 15x longer refresh interval, not refresh elimination: charge-based storage refreshes by construction, so while 3D DRAM contests the density column of this comparison, the zero-refresh power asymmetry this work quantifies survives every DRAM roadmap, planar or vertical. The third front attacks the economics around the cell rather than the cell itself: Intel's July 2026 patent filing for Cross-Batch Memory (XBM) proposes moving the 1T1C cell into back-end-of-line thin-film transistors and replacing HBM's silicon interposer with direct die-to-die UCIe links, targeting HBM4's footprint at lower packaging cost - a response to the same supply-side crunch behind this project's premise (Section 1.1), though the design remains patent-stage, discloses no independent performance data, and is not targeted for commercialization before 2030 [34][35]. The window Table 6 quantifies is therefore real but not permanent - it is the interval in which a cross-point that ports to logic-compatible nodes today faces a DRAM that must first re-architect its cell to go vertical. Every density multiplier in the table also compounds through the capacity law of Section 3.1.4, and again only under *ideal* uniform leveling: each projected doubling of module capacity doubles the ideal-leveling lifetime, leaves the no-leveling bound unmoved, and under a single-region Start-Gap can shorten the projected life rather than extend it (Section 3.1.4). The density dividend is a lifetime dividend only for a leveling scheme this study has not yet demonstrated.

The same geometry extends beyond density to power, and this is where the corrected numbers are least kind to the technology. ReRAM's draw is 99.96% peripheral leakage, and periphery is standard CMOS that shrinks with the process: to first order, static power per gigabyte falls by the same $(22/F)^2$ factor that density rises. Holding the 8 GiB module and its characterized 6.937 W static anchor fixed, a 16nm-class port lands at roughly 3.67 W and a 12nm-class port at roughly 2.06 W. The DDR5 baseline it must beat is 0.797 W for 16 GiB, which is 0.399 W for the same 8 GiB of capacity. Two full shrinks therefore leave the ReRAM module **about 5.2x** above DDR5 per gigabyte, and the feature size at which this arithmetic reaches parity ungated is about **5 nm** - at the leading edge of logic CMOS today, far outside any credible ReRAM roadmap, and on an

area-proportional first order that the paragraph below argues is optimistic. Node scaling does not rescue the power result on its own; it improves it from 17.4x to about 5.2x under GCC. Gating would have to do the rest, and the fraction required is now inside the range real deployments report: 94.3% of the static component at 22nm, 80.8% at a 12nm-class port. The honest conclusion, which reverses the 3 September version's, is that **a 22nm peripheral-leakage floor of 108 mW per gigabit chip is a structural obstacle to ReRAM main memory, not a transient one**, and that attacking it means attacking the periphery - fewer, larger mats, lower-leakage sense circuitry, or a fundamentally different sensing scheme - rather than choosing between access devices. Latency, by contrast, neither improves nor collapses under scaling: filament switching is voltage- and material-limited rather than lithography-limited - HfOx devices demonstrated at 10 x 10 nm retain nanosecond-class switching [14] - so the latency profile of Section 3.1.1 carries to first order, with thinner-wire RC the main unmodeled risk. Endurance splits both ways: device variability - the field's named barrier to large arrays and multilevel operation, rooted in the stochastic filament process [14] - is unvalidated at scaled dimensions, while under ideal wear leveling, and only there, every density multiplier in Table 6 becomes an equal lifetime multiplier at fixed footprint (Section 3.1.4: without leveling lifetime does not move with capacity, and a single whole-module Start-Gap region can do worse as the module grows). The caveats of the preceding paragraph apply with full force - this is bounding geometry, not simulation, and per-transistor leakage densities worsen at advanced nodes, making area-proportional scaling an optimistic first order - but the direction is uniform: every process generation moves the comparison in ReRAM's favor, against a DDR5 baseline whose capacitor cannot follow [5][17].

A fair challenge to these projections is why no sub-22nm ReRAM exists to measure, given that logic CMOS ships below 5nm. The answer is primarily market structure, not device physics. ReRAM's commercial role today is embedded flash replacement on mature 40-22nm foundry processes [24] - the node range where embedded flash itself stops scaling - so no product pull exists toward leading nodes, even though devices have been demonstrated at 10 x 10 nm - with sub-10 nm filaments observed - in the laboratory [14] and a 14nm-FinFET megabit macro has been demonstrated [20]. The one scaled, selector-gated cross-point that did reach volume production - Intel and Micron's 20nm-class 3D XPoint - was discontinued in 2022 on cost-per-bit economics, squeezed between DRAM and NAND, not on device failure [23]: an instructive precedent that the barrier at scaled nodes has been the business model, not the physics. What scaling genuinely risks at the device level is variability - the cycle-to-cycle and device-to-device spread rooted in the stochastic motion of the oxygen vacancies that form the filament, which the field itself names the major remaining barrier to large arrays and multilevel operation [14] - which is exactly why Table 6's 16nm and 12nm columns are labeled projections rather than measurements, and why the memory supply-demand crisis that motivates this work (Section 1.1) may be the demand-side event that finally changes the economics.

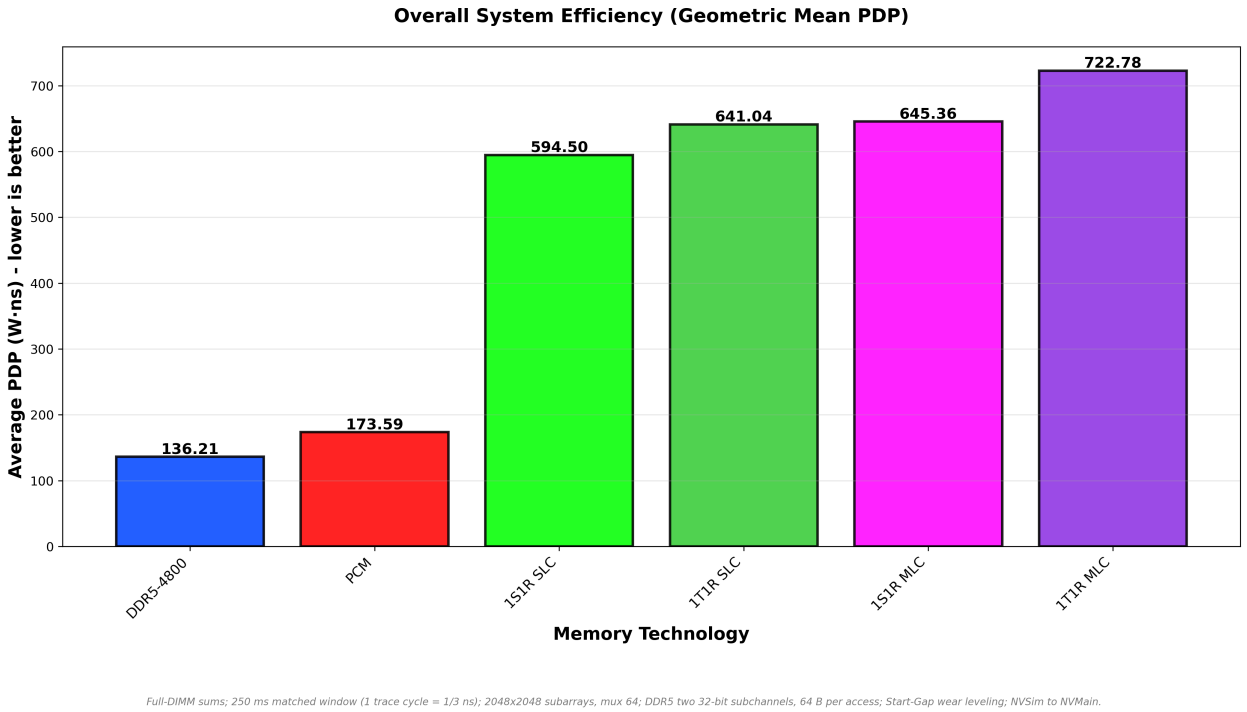


Figure 27: Overall System Efficiency (Geometric Mean PDP).

Regenerated from the primary dataset of this revision and in agreement with Table 4's geometric-mean row: 136.21 (DDR5), 173.59 (PCM), 594.50 (1S1R SLC), 641.04 (1T1R SLC), 645.36 (1S1R MLC), 722.78 (1T1R MLC) W· ns. Module-level; per gigabyte the PCM-versus-MLC ordering reverses (Table 4 note).

However, density is a liability if the system is too slow or burns too much power. The Geometric Mean PDP across all six workloads (Figure 27) ranks the architectures under the corrected, whole-module power model. The full ranking, matching Table 4: DDR5 136.21 W·ns, PCM 173.6, 1S1R SLC 594.5, 1T1R SLC 641.0, 1S1R MLC 645.4 and 1T1R MLC 722.8, with the two published-silicon configurations three and four orders of magnitude further out (134,930 and 2.58×10^6). Three things in that list differ from the 3 September version and all three matter. First, the four ReRAM configurations are now within 1.2x of one another rather than separated by the 29x geometric-mean gap tabulated in Appendix D: with identical leakage, cell type barely moves system efficiency at all. Second, DDR5's own figure moved from 103.3 to 136.21 W·ns - worse, not better, once the corrected background accounting and the corrected JEDEC write path are both in it (Appendix A) - so the gap it sets is narrower than the 3 September version claimed and narrower still than the figure this book carried yesterday. Third, the module-level ranking above is not the ranking that decides anything: normalized per gigabyte, which is the only form in which modules of 4, 8 and 16 GiB are comparable, the geometric-mean ratios against DDR5 are 4.7x (1S1R MLC), 5.1x (PCM), 5.3x (1T1R MLC), 8.7x (1S1R SLC) and 9.4x (1T1R SLC), and the legacy PCM configuration's module-level lead over ReRAM does not survive that normalization against MLC. PCM's own figure additionally rests on two service-limited

workloads and on an inherited configuration constant (Section 3.1.3), so it is a floor-power reference point rather than a competitor.

There is no reading of these numbers on which 22nm ReRAM is a power-competitive DDR5 replacement *as it is modeled here, ungated*. The gap is 4.4x on module geometric-mean PDP and 8.7x per gigabyte for the best SLC configuration, and it is a leakage gap, structurally identical for both access devices. It is a smaller gap than this book has reported at any earlier point, and the reason is that both terms of the DDR5 side were corrected in this revision rather than that ReRAM improved. What the technology does hold is *latency* (Section 3.1.1, at projected device timings and at an assumed interface clock), *density* at MLC (2.47x DDR5 die-level, compounding under node scaling per Table 6), and *zero refresh* - DDR5 spends 6.5-9.1% of its module power retaining data it can never stop retaining, and a ReRAM module spends none. Those are the three assets a system architecture would have to be built around, and the leakage floor is the obstacle it would have to be built against. The “selector leakage discipline” framing of earlier versions, which made the choice between access devices the deciding architectural requirement, has been withdrawn: at a matched organization that choice decides die area and write symmetry, and nothing else.

How much of that draw a gating policy could actually reclaim is bounded by how idle real memory is - a fraction that is strongly workload-dependent, so one measured example is quoted to make the point, not as a universal constant. Profiling Microsoft's Bing web-search and Cosmos data-analytics servers, Malladi et al. report 67-97% processor utilization against only 2-6% memory bandwidth utilization, with web search drawing under 6% of peak channel bandwidth [21]: in that deployment class - a large and commercially central one - the memory channel sits idle the overwhelming majority of the time. Memory-bound workloads occupy the other extreme, and it is a growing one: transformer token generation is dominated by matrix-vector products of such low arithmetic intensity that memory bandwidth, not compute, is its binding constraint - and with peak hardware FLOPS scaling at 3.0x per two years against DRAM bandwidth's 1.6x, the imbalance worsens by design [22]. This suite brackets that extreme deliberately: LBM and STREAM saturate the channel, and GPT-2 is precisely the decode-class workload Gholami et al. analyze. The bounding arithmetic for the 1S1R SLC module follows directly from its characterized composition (6.937 W static plus about 0.003 W dynamic under GCC): gating the static component for an idle fraction f leaves $(1 - f) \times 6.937 + 0.003$ W. Against DDR5's 0.797 W for twice the capacity - 0.399 W of DDR5 power for the same 8 GiB - break-even requires $f = 94.3\%$. At a 12nm-class port (Table 6's geometry, 2.06 W static) it requires $f = 80.8\%$. The 2-6% memory-bandwidth utilization Malladi et al. report for the web-serving class [21] corresponds to 94-98% idleness, and the 22nm threshold now sits *inside* that band rather than at its edge: at $f = 0.98$ the module draws 0.142 W against DDR5's 0.399 W for the same capacity, a ratio of **0.36x - below DDR5** - and at the more conservative 0.94 it draws 0.419 W, 1.05x. **The cited idleness band brackets break-even**, and at its idle end a gated ReRAM module of this design would draw less than DDR5 for the same capacity. So the arithmetic does not forbid parity for that deployment class, and on the optimistic reading of the

band it implies it. What forbids this book from claiming it is that **ReRAM's power-down energy is an explicit no-savings placeholder** (Section 4.1): the model cannot price a gated ReRAM module, and no citable ReRAM power-gating characterization exists to price it with. Every per-gigabyte ratio in this book is therefore an upper bound on the gap a gated module would show, and closing it is a measurement this project could not make rather than a possibility it has excluded - and equally, the parity these lines bracket is arithmetic rather than a result. Two honesty bounds and one physical asymmetry frame this: bandwidth utilization overstates achievable gating, since power-down entry and exit residency consume part of every idle window; DDR5's own power-down states are exercised, so its floor already carries its gating credit (Section 3.1.6, item 5), which keeps these thresholds like-for-like; and both sides are counted as memory devices only, with no register clock driver, power-management IC, PHY or ReRAM controller on either (Section 3.1.2), which cuts *against* the bracketed parity above rather than for it. A fixed per-module overhead is carried by 8 GiB of ReRAM against 16 GiB of DDR5, so adding the same overhead O to both sides leaves $(1-f)\times 6.937 + 0.003 + O = (0.797 + O)/2$, and the required gating fraction *rises* with O: 94.3% at $O = 0$, **97.9% at $O = 0.5$ W per module**, and beyond $\boldsymbol{O = 0.79}$ **W** no gating fraction reaches parity at all. Equivalently, the 0.257 W margin the $f = 0.98$ line above enjoys shrinks by $O/2$ and is gone at $O = 0.51$ W, which is below any plausible register-plus-PMIC figure. Two distinct statements have to be kept apart here, because they point in opposite directions: the same overhead *compresses the ungated per-gigabyte ratio* of Section 3.1.2, which is a ratio of totals and falls from 17.4x toward an asymptote of 2x as O grows, and it *tightens the gated comparison*, which is a break-even fraction. The first flatters ReRAM and the second does not, and it is the second that this paragraph's arithmetic depends on. And the floor asymmetry is genuinely physical - an idle DRAM module must keep refreshing to retain data, while a gated non-volatile module retains it at zero power, which is why the zero-refresh asset survives every DRAM roadmap even though it does not, by itself, pay for the leakage. These are bounding calculations, not a real characterization; sourcing a real ReRAM power-gating energy characterization remains a future-work item (Section 4.1), but it is no longer the item on which the power verdict turns.

Table 7 places every axis of the comparison side by side - the tabular summary of the entire evaluation.

Table 7: Cross-technology summary, full-DIMM configurations.

Technology	GCC latency (ns)	GCC power (W / W per GiB)	Geo-mean PDP (W·ns)	Die density (× DDR5)	Projected lifetime @128 GiB	Role
DDR5-4800	83.1	0.797 / 0.0498	136.21	1.00	n/a (volatile)	commodity baseline; sets every bar
PCM (legacy)	12,168.2	0.046 / 0.0115	173.6	1.25*	not evaluated	floor-power reference; inherited power constants, two service-limited workloads, loses on latency by about 150x
1T1R SLC	41.4	6.939 / 0.8674	641.0	0.36	7.1 yr	most write-symmetric ReRAM track; smallest write penalty
1S1R SLC	36.8	6.940 / 0.8674	594.5	1.24	7.1 yr	fastest reader, 3.4x smaller die; conditional on selector quality
1T1R MLC	44.5	6.941 / 0.4338	722.8	0.73	7.1 yr	capacity tier; halves the per-GiB power penalty
1S1R MLC	37.9	6.941 / 0.4338	645.4	2.47	7.1 yr	densest configuration; read-dominant roles only

Latency, power and PDP from results/rev2026-09_primary_csv/ processed_bar_chart_metrics.csv and processed_geometric_means.csv. ***Power*** *is given as module watts / watts per gigabyte; only the second is comparable, because the modules are 16 GiB (DDR5, ReRAM MLC), 8 GiB (ReRAM SLC) and 4 GiB (PCM), and both sides count memory devices only, with no register clock driver, power-management IC, PHY, termination or ReRAM controller (Section 3.1.2).* ******* *The PCM density figure is a fixed constant in the metrics pipeline with no device or datasheet source behind it, not a characterized die area; it is reported*

here for completeness and should be read as not characterized*, like the lifetime cell beside it. The published-silicon counterweights of Table 2 are deliberately not repeated here: no row of this table describes a fabricated part. Die-level density is NVSim-characterized die area per GB against a commodity DDR5 baseline (Figure 26) at the matched 2048 x 2048 organization; node-independent cell-level bounds are DRAM 6F²/bit, 1T1R 20F²/bit and 1S1R 4F²/bit (two bits per cell as MLC), with 1T1R's 20F² a planar/FinFET logic-transistor assumption rather than a hard ceiling (Appendix A, ref [33]).* ***Projected lifetime*** *is the LBM worst case at 128 GiB under* ideal *uniform leveling at 10^6 cycles per cell, and is identical across all four ReRAM tracks because every track now completes the identical request population and therefore wears at the same 9.54 M writes/s. It is an upper bound: under a single-region Start-Gap remapper the same projection is 23.2 hours (Table 5). The claim in earlier versions that the selector variant outlives the transistor variant has been withdrawn.*

3.4. Near and Far Memory

Everything above compares this book's modeled module against a DDR5 DIMM in the *near* memory position: the slot the processor reaches over its own latency-controlled interconnect, where latency is the figure of merit and a replacement has to win on it. There is a second position in a modern server, reached over PCI Express rather than over the memory bus, where capacity and density matter more than the last fifty nanoseconds. This section asks where the projected device of this book would sit if it were aimed at that position instead, and answers it with measured numbers from the literature rather than with an aspiration.

One piece of vocabulary has to come first, because without it the comparison is a category error. **A *media* number describes the storage element and its array; a *device* number describes a whole product - media plus controller, plus error correction, plus interface, plus, in some designs, a DRAM cache in front of it.** Every latency this book reports for ReRAM is a media number: it is NVSim's array time carried through an NVMain protocol model at an assumed interface clock, with no controller, no coding block and no PHY overhead (Section 2.3, Section 3.1.2). Every Optane and CXL number in this section is a device number, measured on shipped hardware from the processor's point of view. The two are separated by exactly the stack this project does not model, and they are never compared as if they were the same measurement. In the same vocabulary, 3D XPoint is the *media* - a phase-change element in series with an ovonic threshold switch, in a double-stacked cross-point die, announced by Intel and Micron in 2015 **[100]** - and Optane is the *device* brand built on it.

3.4.1. The measured far-memory window

Real systems accept a surprisingly narrow band of far-memory latency, and the band is measured rather than declared. Local DRAM at idle sits at 80 to 115 ns, with DDR5 measured at 111 to 114 ns on two-socket servers and platform baselines of 78 and 115 ns reported in Microsoft's Pond study **[84]**, **[85]**. Crossing to the far socket of the same machine costs 140 to 193 ns on two sockets and 411 ns on eight **[84]**. Shipped CXL-attached DRAM expanders measure about **214, 239 and 271 ns** idle load-to-use for ASIC-based devices and about **394 ns** for an FPGA-based one **[84]**, **[86]**; memory pooling adds roughly 70 to 90 ns at eight to sixteen sockets and about

180 ns rack-scale **[85]**. Above that band the mechanism changes rather than degrades: Google's software-defined far memory, which moves cold pages behind a page fault, tolerates single-digit microseconds **[98]**.

The one shipped non-volatile main memory in that range is Optane persistent memory, measured at **305 ns** idle random read and 169 ns sequential read, against 81 ns for local DRAM on the same platform, at 6.6 GB/s read and 2.3 GB/s write per module **[94]**. No open figure exists for its latency under load, which is a real gap in the comparison and not an omission here.

What does *not* survive contact with that data is a single blanket latency threshold for what counts as acceptable far memory. Pond's own workload study finds applications losing measurable performance at latency increases well below the width of the window above, while most tolerate an increase of that size **[85]**; acceptability is a per-workload property, not a number. A second confusion is worth settling in the same place, because the two bands are often swapped in discussion: **the few-hundred-nanosecond band measured above belongs to CXL-attached DRAM *devices*, not to local DRAM.** Local DRAM idle latency is far below it - 111 to 114 ns of measured DDR5 on two-socket servers, and platform baselines of 78 and 115 ns **[84]**, **[85]** - so a figure drawn from the device band cannot be used as a DRAM reference, and a device built to sit inside that band is not competing with local DRAM at all.

This book's two latency lines sit on opposite sides of that window, and neither is inside it. The projected media latency of the four ReRAM tracks is 36.8 to 44.5 ns under GCC and 353.1 to 652.9 ns on the write burst (Table 2), so on CPU traffic the media is *below* the entire window, with room for a controller and an ECC stack before it would reach the low end of it. The published-silicon counterweights are 11.9 us and 379 us under GCC, which is 30x to 1,000x above the window and slower even than the page-fault-based software-defined far memory that tolerates about 6 us **[98]**. The gap between those two lines is the whole state of the technology: nothing shipped is close, and nothing in the device physics says it could not be.

3.4.2. How Linux would see such a device

A CXL Type 3 memory region **[87]** surfaces to Linux as a CPU-less NUMA node, with the ACPI Heterogeneous Memory Attribute Table supplying per initiator-target-pair latency and bandwidth figures so the kernel can rank it against local memory **[88]**. What the kernel then does with that ranking is **page placement, not per-request routing**: pages move between tiers at 4 KiB granularity on a millisecond-to-second timescale under the memory-tiering policy (`numa_balancing` mode 2, with a configurable promotion rate limit **[89]**, an abstract distance of 512 and demotion disabled by default **[90]**). Meta's Transparent Page Placement work describes the same mechanism at production scale and reports CXL adding about 50 to 100 ns over local DRAM, with FPGA-based cards about 250 ns higher **[91]**. **There is no mechanism that inspects an individual load or store and routes it by projected access time**; the system's intelligence is in which pages live where, which is why a far memory can afford to be slower than DRAM but cannot afford to be slower by a factor that page migration cannot amortize.

3.4.3. What CXL is and is not worth in bandwidth

The common claim that a CXL expander carries more bandwidth than a DIMM does not hold as stated. A x8 CXL device is published at up to 36 GB/s **[92]**, which is at or below a single DDR5-4800 channel's 38.4 GB/s **[86]**; a x16 link is only partly ahead, and no expander approaches a socket's own aggregate, about 307 GB/s across eight DDR5-4800 channels **[93]**. Link efficiency on sequential reads is lower as well: about 70% for DDR5, 46 to 47% for a CXL ASIC device and about 20% for an FPGA-based one **[86]**. A shipped example sets the shape of the product class: a CXL 1.1 / 2.0 smart memory controller at x16 and 32 GT/s carries two DDR5 channels and up to 2 TB behind one slot, and publishes no latency figure at all **[99]** - which is exactly why the device latencies of Table 10 come from independent measurement papers rather than from vendor material. The correct framing is therefore not that CXL beats a DIMM channel, but that **CXL adds capacity and bandwidth from a slot that otherwise carries no memory at all**, at a real efficiency cost per link. For this project that framing matters because the density dividend of Section 3.3 is exactly the quantity such a slot is bought for.

3.4.4. What the one shipped precedent actually did

Optane persistent memory shipped as one product line, the 100 and 200 series, in two modes **[95]**. In **Memory Mode** the platform's DRAM acts as a direct-mapped cache in front of the Optane media and the operating system never sees the Optane directly; that is the far-memory-like mode, and it is the architectural template a ReRAM main memory would most plausibly follow. In **App Direct** the media is exposed byte-addressable in 64-byte lines, the same access width this project uses throughout (Section 2.4). A 300 series was planned and cancelled in January 2023 **[96]**, no CXL-attached version ever shipped, and Intel exited the business in 2022 citing the industry's move to CXL **[97]**. The precedent is therefore both an existence proof and a warning: the architecture works, and the product did not survive it.

3.4.5. Device reference row

The table below is the one place in this book where this project's modeled tracks are set beside measured hardware. Every row carries the label that says which kind of number it is, and no row is a substitute for another.

Row	Kind	Latency	Bandwidth / note	Source
1T1R SLC, full DIMM	media	GCC 41.4014 to 48.3061 ns; write burst 353.1069 to 696.6112 ns	modeled at an assumed 800 MHz interface; no controller, ECC or PHY	primary and bounding CSVs
1T1R MLC, full DIMM	media	GCC 44.5279 to 65.7738 ns; write burst 512.0013 to 1,649.9 ns	as above, twice the capacity per chip	primary and bounding CSVs
1S1R SLC, full DIMM	media	GCC 36.7661 to 47.3896 ns; write burst 391.4819 to 916.5738 ns	as above; conditional on a selector better than production OTS	primary and bounding CSVs
1S1R MLC, full DIMM	media	GCC 37.8691 to 72.3981 ns; write burst 652.9494 to 2,498.7 ns	as above; densest track at 2.47x DDR5	primary and bounding CSVs

Row	Kind	Latency	Bandwidth / note	Source
DDR5-4800, this book's baseline	media (modeled)	GCC 83.1115 ns; write burst 234.2819 ns	two 32-bit subchannels, 64 B per access, JEDEC timings	primary CSV
DDR5 on a two-socket server	device	111 to 114 ns idle; 140 to 193 ns to the far socket	38.4 GB/s per channel; about 307 GB/s per 8-channel socket	**[84]**, **[86]**, **[93]**
CXL-attached DRAM expanders	device	214, 239, 271 ns (ASIC) and about 394 ns (FPGA) idle load-to-use	x8 device up to 36 GB/s; 46 to 47% link efficiency (ASIC)	**[84]**, **[86]**, **[92]**
Optane PMem 100 / 200 series	device	305 ns idle random read; 169 ns sequential read	6.6 GB/s read and 2.3 GB/s write per module; no open loaded-latency figure	**[94]**

Table 10: Device reference row - this book's modeled media beside measured devices. *The four ReRAM ranges and the DDR5 row are the* `Latency_ns` *column,* `full_dimm`*, of results/rev2026-09_primary_csv/processed_bar_chart_metrics.csv and, for the second figure of each ReRAM range, of results/system_rev2026-09_writepulse/_csv/processed_bar_chart_metrics.csv (Table 9). "Write burst" is the AlexNet layer 1 output map. The DDR5 baseline appears twice on purpose: once as the modeled media number this book compares against, and once as the measured device latency of a real DDR5 server, which is about a third higher.* ***No ReRAM row in this table is a device number, and no device row is a prediction for ReRAM.***

The conditional statement that follows from the table is the honest one, and it is conditional in both directions. **ReRAM could serve as far memory only if a *device* built on it - controller, error correction, and a DRAM cache in front of it as in Optane's Memory Mode - landed in the same few-hundred-nanosecond class as the measured CXL-attached DRAM devices above. No published ReRAM silicon does.** What the projected numbers of this book do say is which half of that problem is open: at 36.8 to 44.5 ns of media latency under CPU traffic, with 1.24x to 2.47x DDR5's die-level density on the selector track, the device physics is not the obstacle. The interface, the error-correction stack and the controller are. That is a different research programme from the one this book ran, and Section 4.2 names its first two pieces.

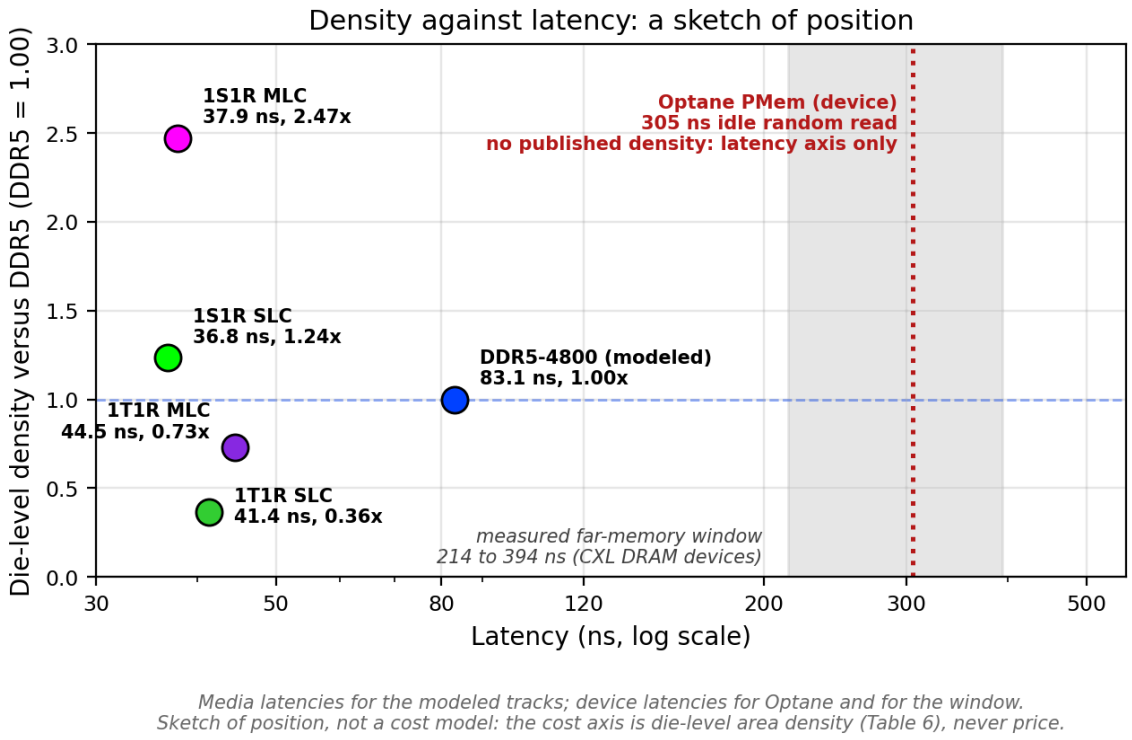


Figure 28: Density against latency - a sketch of position.

This is a sketch of position, not a cost model. *The cost axis is die-level area density against DDR5 = 1.00 (Table 6,* `Area_Density_Ratio` *of results/rev2026-09_primary_csv/processed_hero_metrics.csv), which is this project's standing cost proxy; no price appears on it, here or anywhere else in this book. The latency axis carries each modeled track's GCC average in default mode (*`Latency_ns`*,* `full_dimm`*, primary CSV) -* media *numbers - and, separately labelled, Optane PMem's 305 ns idle random read as a* device *number* ***[94]****; Optane has no published die-level density figure, so it is drawn on the latency axis only and carries no height. The shaded band is the measured far-memory window, 214 to 394 ns of idle load-to-use latency on shipped CXL-attached DRAM devices* ***[84]****,* ***[86]****. Nothing in this figure is fitted, projected forward in time, or priced. Rendered by* `tools/plot_far_memory_sketch.py`*.*

4. Conclusion and Future Work

This research delivers two things: a corrected cross-layer characterization of 22nm ReRAM as a DDR5 alternative, and a quantified fidelity audit of the standard NVSim-to-NVMain toolchain in which thirteen of fourteen discovered failure modes were repaired, spanning the ReRAM configurations, the metrics pipeline and both non-ReRAM baselines, each repair validated by exact anchor arithmetic and an independent blind re-verification, with the affected simulations re-run. The word "corrected" is load-bearing: this revision withdraws three of the headline claims the 3 September version made, and it withdraws them because the experiments that supported them were not controlled. Using a cross-layer simulation pipeline spanning device physics (NVSim), cycle-accurate memory simulation (NVMain 2.0) and regenerated workload

traces from gem5 and SCALE-Sim, I evaluated 20 memory configurations across 6 workloads at one matched array organization.

The key quantitative findings are four. **Latency:** at NVSim-projected device timings the ReRAM DIMM is faster than DDR5-4800 on five of six workloads - 41.4 ns (1T1R SLC) and 36.8 ns (1S1R SLC) against 83.1 ns under compute-bound GCC. The direction is robust and the margin is model-dependent: only 11.25 ns of the ReRAM figure is NVSim array time charged as tRCD + tCAS, and roughly three quarters of it is NVMain's default protocol cycle counts at this book's assumed, uncited 800 MHz interface, which is why the same configuration reports 19.0 ns at 2400 MHz (Sections 2.3, 3.1.1). On the write-dominated AlexNet OFMAP burst it is 1.5x to 2.8x slower. That result is bounded by the published-silicon configurations run on the same traces, which land at 11.9 us and 379 us under GCC and complete 18% and 1% of the LBM window: the projected array core is fast, and nothing fabricated is (Section 3.1.1). **Power:** at a matched 2048 x 2048 organization NVSim gives both cell types identical peripheral leakage, 108.384 mW per 1 Gb chip, so the transistor-versus-selector leakage gap this book previously called its deciding architectural fact *does not exist as a device effect*; it was an organization artifact (Appendix D). The full ReRAM DIMM draws 6.94 W for 8 GiB, 99.96% static, which is 12.7x to 17.4x DDR5's power per gigabyte for the SLC tracks and 6.4x to 8.7x as MLC, while DDR5 spends 6.5-9.1% of its own module power on refresh it can never shed. Both module figures count memory devices only, with no register clock driver, power-management IC, PHY, termination or ReRAM-side controller on either side, so a fixed per-module overhead would compress this per-gigabyte ratio - and, being carried by half as much ReRAM capacity as DDR5 capacity, would at the same time raise the gating fraction break-even needs (Section 3.3). That ratio is also an **upper bound**: the DDR5 figure carries its JEDEC power-down credit while ReRAM's power-down energy is an explicit no-savings placeholder, so the model cannot price a ReRAM module that gates its periphery. Gating is no longer arithmetically out of reach, which is a change of tone from the 3 September version and from the figures this book carried before the corrections of Appendix A: break-even needs 94.3% of the static component gated at 22nm and 80.8% at a 12nm-class port, and the 2-6% memory-bandwidth utilization Malladi et al. report for web-serving [21] corresponds to 94-98% idleness, a band that *brackets* that threshold - at its idle end a gated 22nm module would draw 0.14 W against DDR5's 0.40 W for the same 8 GiB. Ungated, node scaling alone reaches parity only at about a 5 nm feature size, on an area-proportional first-order scaling that Section 3.3 argues is optimistic. What this project can state is the ungated measurement and the threshold; what it cannot state is a gated ReRAM number, because no citable power-gating characterization exists, so neither the parity nor its impossibility is demonstrated here (Sections 3.1.2, 3.3, 4.1). **Endurance:** every lifetime here is a projection from a measured write distribution, and it is decided by the wear leveling rather than by the cell. At 64 GiB and 10^6 cycles, ideal uniform leveling gives 38.6 years (GCC), 3.6 (LBM) and 4.3 (STREAM), while no leveling gives 7.7, 23.2 and 69.5 hours, and a faithful single-region Start-Gap remapper recovers none of that gap because the hottest line fails before the gap rotates once. Randomizing the region recovers part of it (0.20 and 0.96 years). The endurance a cell would need for a ten-year life at 64 GiB is 2.6×10^5 to 2.8×10^6 under ideal leveling and 4.3 to 6.5 x

10^6 under randomized Start-Gap, against a best-sourced production planning value of 10^6 (Section 3.1.4). **Density:** the selector cross-point delivers 1.24x DDR5's die-level density as SLC and 2.47x as MLC at 22nm, compounding quadratically under node scaling and multiplicatively under deck stacking (Table 6); the transistor-gated variant reaches 0.36x and 0.73x.

The deployment recommendation that follows is narrower than the one this book previously drew, and it no longer turns on the access device. At a matched organization the choice between 1T1R and 1S1R decides die area (3.4x in the selector's favor) and write symmetry (1.5x against 5.2x write-to-read at the device, in the transistor's favor), and decides essentially nothing about power or system efficiency. Both are conditional: 1S1R on a selector better than today's production ovonic threshold switch, which cannot keep a 2048-cell tile valid, and 1T1R on a cell area no fabricated logic-process part has reached. What 22nm ReRAM is, on this evidence, is a *dense, non-volatile, latency-competitive, leakage-expensive* medium: a plausible capacity tier behind a DRAM front end, particularly as MLC for read-dominant storage such as frozen model weights, and not a drop-in DDR5 replacement. Making it one requires attacking the peripheral-leakage floor directly, which is a circuit problem in the sense amplifiers and decoders, not a choice between access devices. Finally, the mbmm_master.py orchestration framework, which bridges NVSim device characterization to NVMain system simulation in a single reproducible pipeline - now with forced-organization gates, provenance sidecars on every trace, per-location wear counters and an end-to-end latency statistic - is contributed as an open-source Physical-to-System memory simulator for the non-volatile memory research community.

Each results axis resolves to a one-line verdict. Latency (3.1.1): the projected array core beats DDR5 everywhere except under sustained write pressure, at an assumed interface clock that supplies about three quarters of the modeled figure, and the published-silicon counterweight is three to four orders of magnitude away. Power (3.1.2): leakage is identical between access devices at a matched organization, it is 99.96% of the module total, and per gigabyte it is 12.7x to 17.4x DDR5 for the SLC tracks, device-for-device - the central negative result of this evaluation. Efficiency (3.1.3): with leakage equal, the four ReRAM tracks fall within 1.2x of one another and 4.4x behind DDR5 at module level, 4.7x to 9.4x behind per gigabyte. Endurance (3.1.4): wear leveling, not the cell, decides viability, and a single-region Start-Gap is not enough. Robustness (3.1.5): read latency is invariant and system PDP moves under 4% across a ±20% ReadVoltage sweep, on the pre-revision dataset. Scaling (3.2): chip count buys capacity and pays leakage for it exactly linearly, and buys latency only where a workload is queue-bound and then only through the second channel, which a one-channel control run measures directly; rank depth bought at most 7.5%, on the three offered-rate-limited CPU traces only, and exactly nothing on the three AI traces.

The scope of these claims is bounded by four disclosed limitations, gathered here deliberately in one place: validation is internal to the toolchain - parameters are anchored to real silicon and vendor datasheets, but no end-to-end result is checked against measured hardware (Section 2.2); every latency figure for the projected cells rests on NVSim's array-core model, which no fabricated part has matched, which is why the published-silicon rows accompany every latency

table (Sections 3.1.1, 2.3); every endurance figure is a projection from a measured write distribution rather than a measured lifetime, and the wear-leveling schemes are modeled analytically rather than simulated end to end (Section 3.1.4); and one AI trace cannot be attributed to a preserved generator configuration, while all three AI traces are microsecond bursts rather than sustained workloads (Sections 2.4, 3.1.6 item 6). Two of these bound a headline rather than qualifying it, and are stated as such: without a fabricated part at NVSim's timings there is no latency advantage, and without a wear leveling scheme substantially better than single-region Start-Gap there is no ten-year lifetime. That boundary is, by construction, the future-work agenda of Sections 4.1-4.2.

4.1. Future Work: Simulator and Infrastructure Enhancements

Power-Down Restoration (done this cycle; one gap remains): NVMain's previously-disabled power-down state machine (`MemoryController::HandleLowPower()`) has been restored - a shared mechanism, so it now fires for every technology, not ReRAM alone. DDR5 already carried real, JEDEC-datasheet-backed power-down currents and realizes a genuine, measured power reduction from them under GCC, so the per-gigabyte gap separating ReRAM from DDR5 (Section 3.1.2) is compared against a real, gated DDR5 number rather than an ungated one. The remaining gap is on ReRAM's side: no NVSim datapoint decomposes its leakage into a gatable-periphery-vs- ungatable-crossbar split, and a dedicated literature search (Appendix A) found no citable ReRAM power-gating energy or fraction to characterize what a real gated ReRAM module would actually draw - so ReRAM's power-down energy is set to an explicit, disclosed placeholder equal to its own standby energy (no modeled savings), keeping the mechanism mechanically live (86-89% of cycle-slots gated, confirmed directly) while claiming no unearned benefit. **Sourcing a real ReRAM power-gating characterization - what fraction of its leakage is peripheral and gatable, and at what entry/exit energy cost - remains the item on which the per-gigabyte power verdict turns**. Section 3.3's bounding arithmetic shows why: break-even against DDR5 needs 94.3% of the static component gated at 22nm, which sits inside the 94-98% idleness the web-serving profile of [21] reports, so the gap is not arithmetically out of reach for that deployment class, and at the idle end of that band the gated module would draw less than DDR5 for the same capacity. This book cannot claim it is closed, because the model has no gated ReRAM energy to price it with; equally, it must not claim it is unreachable. Running alongside it, and independent of any gating policy, is **reducing the peripheral leakage itself** - the 108.384 mW per 1 Gb chip that NVSim charges to decoders, muxes, prechargers and sense amplifiers - through fewer and larger mats, lower-leakage sense circuitry, or a different sensing scheme entirely. PCM shows no power-down activity at all in either run, plausibly because its `FRFCFS-WQF` controller's write-queue-flush design keeps its request queue non-empty far more of the time than the plain `FRFCFS` controller ReRAM/DDR5 use, starving the power-down entry condition - a second, smaller open item, not yet root-caused to full confidence.

- **Parameter Optimization**: Conduct a sensitivity analysis on NVSim parameters, such as ReadVoltage and WritePulseWidth, to identify the efficiency-optimal operating region for LOP (Low Operating Power) devices that maximizes sensing margin while minimizing energy.

- **Native MLC Logic**: Resolve the NVSim C++ "Floating Point Exception" (FPE) specifically within the Mat.cpp sensing logic to replace my current "Analytical Penalty Method" with native circuit-level characterization of iterative sensing.
- **A Workload That Would Actually Use Rank Depth**: *This item replaces the 2026-09 revision's "Chip-Count Matrix Under Restored Idle-Gating", which is closed: the primary matrix ran all four architectures in one pass with the power-down mechanism live, so Section 3.2 and Tables 2-4 are already on the same dataset.* What Section 3.2 could not answer is what rank depth is worth, because none of the six workloads asks for it. The three CPU traces span all eight ranks and are offered-rate limited, so extra ranks have no backlog to relieve; the three AI traces are queue-bound but their footprints (64.0 to 132.2 KiB) never leave the first 128 KiB rank span. The decisive case is a trace with a *large* address footprint *and* a sustained arrival rate above one controller's service rate, which is a full-model AI-inference capture or a multi-programmed CPU mix rather than either of the two the suite carries. Acquiring one would convert the section's strongest negative result from "this suite could not use rank depth" into a measurement of what rank depth is worth.
- **Representative AI-Inference Trace**: Both AI-inference traces available to this study are single-layer SCALE-Sim captures whose ~64-68 KB address footprints never leave one rank (Section 3.2, Appendix A). A full-model trace is needed before any claim about how AI inference scales with rank count.
- **Standalone Device-to-System Simulator Wrapper**: Generalizing "The Bridge" (Section 2.1) - the cross-layer ETL pipeline that translates NVSim's device-level physical metrics into NVMain's cycle-accurate architectural parameters - into a reusable, project-independent simulator wrapper. The toolchain audit (Section 3.1.6) already demonstrated that this translation layer is precisely where correctness risk concentrates; packaging it as a standalone, hardened tool would let other device-level-to-system-level simulation pairings benefit from the same repairs and validation discipline, beyond this project's specific NVSim/NVMain pairing.
- **FPGA-Based Hardware-in-the-Loop Validation of Power-Down Policy**: Every result in this book is validated only against NVMain's own internal consistency (Section 2.2) - no fabricated ReRAM hardware exists to check system-level, as opposed to device-level, behavior against. A tractable next step, complementary to Power-Down Restoration above, is an FPGA emulation of just the idle-gating/power-down state machine - using the same literature-calibrated device parameters already characterized in this book, but exercised under real hardware clock-edge timing rather than software simulation - to check whether the controller's power-down and wake behavior matches its simulated prediction. This would strengthen the system-level validation claim without requiring access to fabricated ReRAM silicon, which remains out of reach; it would not, by itself, validate NVSim's own device-level circuit numbers against real silicon, since the emulated cell still runs on the same literature-derived parameters.

4.2. Future Work: Architectural Extensions and System Scaling

- **The L3 Cache Mitigation:** Evaluating the integration of a large last-level cache (LLC) or near-memory write buffer to absorb and coalesce ReRAM write penalties, a mitigation strategy historically proven essential for legacy phase-change memory **[11]**, specifically for write-intensive AI workloads like AlexNet OFMAP.
- **Endurance-Aware Scheduling:** Implementing wear-leveling algorithms at the controller level to protect the finite lifecycle of the 22nm cells.
- **Macro-Scale 1T1R Array Limits:** This evaluation forces both cell types to a matched 2048 x 2048 subarray so that the transistor-versus-selector comparison is controlled (Section 2.3), but 1T1R arrays are not bound by the sneak-path limits that cap the cross-point, and the matched organization is therefore a constraint on 1T1R rather than a description of its ceiling. Because chip leakage follows mat count, enlarging the 1T1R subarray is also the most direct attack available on the peripheral-leakage floor that Section 3.1.2 identifies as the technology's binding obstacle. Future research should evaluate the latency, power and routing tradeoffs of 1T1R arrays at commercial foundry macro limits, reporting leakage per gigabyte as the primary metric rather than latency.
- **Recessed-Channel 1T1R Density:** this evaluation models 1T1R with a planar/FinFET, width-driven CMOS access transistor ($20F^2$, Appendix A), the only access-device topology NVSim supports for this project. A commercial part has demonstrated a DRAM-process buried recessed-channel access transistor instead, reaching a $6F^2$ cell - parity with DRAM [33] - though with a different switching-layer material (full discussion in Appendix A). Closing this gap for the oxide-RRAM material system modeled in this book would require either patching NVSim with a recessed-channel access-device model or deriving an equivalent RC/ drive-current model outside NVSim and feeding it back in; either is real device-physics work, not a configuration change, and could overturn this book's "1T1R is not density-competitive" verdict if the recessed-channel geometry proves compatible with oxide-RRAM integration.
- **Node Scaling and 3D Stacking**: this evaluation deliberately fixed ReRAM at 22nm - a manufacturability-anchored choice [8] - while DRAM competes from 10 nm-class nodes with no remaining capacitor runway [5], [17]. Future work should project the $4F^2$ cross-point's quadratic density dividend at 1x-nm feature sizes, quantify the multiplicative effect of back-end-of-line deck stacking [9], and re-derive the NVSim characterization at scaled nodes, where the peripheral-leakage floor that Section 3.1.2 identifies as the binding obstacle is the quantity to track for *both* access devices: its per-transistor density is expected to worsen at advanced nodes, against the area-proportional first order Table 6 assumes. Density gains convert into lifetime only under ideal uniform leveling (Section 3.1.4), so a wear-leveling scheme better than single-region Start-Gap is a precondition for that part of the dividend, not a detail.
- **Interface Parity: Faster-PHY ReRAM**: Every ReRAM configuration in this book runs behind a deliberately conservative 800 MHz interface (Section 2.3) - a modeling choice made to isolate the device-technology comparison from a confounding interface-engineering

difference, not a limitation of the memristor media itself. Channel count is already matched to DDR5 in the primary configuration (two channels), so what remains open is the clock: 800 MHz against the DDR4-2666 electricals that carried Optane's DDR-T protocol [53], and against DDR5-4800 itself. That axis has now been run, at 800 / 1333 / 2400 MHz, and it quantifies the choice rather than leaving it open (Section 2.3): the full DIMM's 1T1R SLC average under GCC falls from 41.4 ns to 27.9 and then to 19.0, and 1S1R SLC from 36.8 to 23.6 to 14.9, against a DDR5 baseline of 83.1 ns; on the write-dominated AlexNet output-map burst the same triple gives 353.1 / 263.1 / 200.4 ns in the controller and 486.1 / 366.5 / 283.3 µs end to end. So the interface is worth 2.2x on compute-bound traffic and only 1.7x where the queue is the binding term, and it costs nothing in the modeled static power, which does not depend on the interface clock. What remains genuinely open is therefore not the magnitude but the feasibility: no ReRAM PHY specification at any of these three rates exists in the literature this project could find (Section 2.3), so the faster two points are the same kind of assumption the 800 MHz point is, measured rather than justified. Sourcing or bounding a real ReRAM interface rate is the work this item now names.

- **Wide Internal Word Behind a Write-Combining Buffer**: A dedicated study of access granularity concluded that 128-byte access is worse than 64-byte on every axis that could be modeled honestly here - 57-75% more latency, 43-50% less lifetime, no device-level gain
  - and that NVMain's address translator is hardwired to a 64-byte grid

  in any case. The idea that survives is Optane's own: keep a 64-byte bus transaction in front of a *wider internal word*, with a write-combining buffer absorbing the mismatch [53]. That is the structure a real ReRAM DIMM controller would need in order to amortize write recovery, and it is the same mechanism that would attack the write asymmetry of Section 3.1.1 and the wear concentration of Section 3.1.4. It cannot be evaluated in NVMain without a controller-side buffer model, which is the work item.

- **CXL Type 3 Device Model with Capacity-Bound Traces**: Every result in this book places the module in the *near* memory position, on the processor's own memory bus. Section 3.4 shows that the projected media latency would clear the measured far-memory window with room to spare, and that the open question is the device stack rather than the device physics. The work item is to model the far position directly: a CXL Type 3 region behind a link model, exposed as a CPU-less NUMA node with HMAT attributes [87], [88], and driven by traces that are *capacity*-bound rather than latency-bound, which none of the six traces in this book is. Two things would then become measurable that are not measurable now: what page-level tiering [90], [91] does to the effective latency of a resistive tier, and whether the density dividend of Section 3.3 converts into a system-level win at a link efficiency of 46 to 47% [86]. The deliverable is the far-memory counterpart of Table 10, with this project's own rows measured rather than positioned.

- **ECC Decode on the Read Path**: Section 3.1.2 establishes that a ReRAM main memory needs SSD-class rather than DRAM-class correction - the literature's design point is a raw rate of 2 x 10^{-4} corrected with BCH(2312,2048,22) at about 27% coding overhead [74] - and that *none* of

the latencies in this book carries the decode time that code would cost. That is the single largest disclosed optimism left in the read path after the corrections of Section 3.1.6. The work item is to price it: a decoder latency model on the read path, a capacity cost of 27% applied to the density figures of Table 6, and the resulting re-derivation of the latency and density verdicts together, since the two move in opposite directions. It needs a code-level latency estimate rather than a simulator change, and it would turn the “excludes ECC decode” caveat into a number.

- **Write-Pulse-Aware MLC Modeling Beyond the Bracket**: The bounding run of Section 3.1.1 brackets the write-path cost between two mappings of one cell-write number, and the bracket is wide - up to 1.53x SLC on CPU traffic and 2.73x on a write burst. Neither end of it models what an MLC part actually does, which is a *program and verify loop*: published 1T1R data reports an average of 49 pulses per level move [55], and the closest two-bit array result spends more than 70 iterations on a 9.7 us write [57]. The work item is to put that loop structure, rather than a single scalar pulse, on the request path: an iteration-count distribution per level transition, a verify-read cost, and the early-termination behaviour a smart-write scheme [63] provides. That is the only way the MLC latency question stops being a modeling-choice question, and it is also the natural place to attach the endurance and energy consequences that Sections 3.1.4 and 3.1.2 currently carry separately.
- **Optane as a Real-Hardware Reference Row**: The only NVM main memory module ever shipped remains the field’s one genuine hardware anchor, and this book should carry it as a carefully scoped reference row rather than as a simulated configuration. **Section 3.4.5, Table 10, is the first version of that row**, with its measurement conditions stated and every entry labelled media or device; what follows is what would still have to be added to it. What is comparable, with its measurement conditions stated, is density per DIMM, power envelope per gigabyte from the published maximum TDP, endurance in petabytes written with the access size and mix stated, bandwidth envelope, and the software-visible read-latency ratio against DRAM on the same platform [53], [54]. What is *not* comparable, and must never be tabulated beside this book’s figures, is absolute latency (which includes CPU, memory controller and on-DIMM controller), write latency (hidden by the write queue), and energy per operation or idle power, neither of which Intel published. No NVMain configuration can honestly be called Optane: its media timing was never published, the only validated public model of it is a controller-and-buffer simulator with no power model at all, and a Lee-2009-style PCM model of the kind this book’s own baseline inherits scores 65.6% against real Optane.

4.3 Data Availability and Reproducibility

The complete simulation stack (the mbmm_master.py orchestration script, the patched NVSim and NVMain C++ sources, the JSON configurations) is public at https://github.com/uvKogan/Memristor-based-dram. The two simulators are git submodules of that repository, pinned at the NVMain fork `uvKogan/NVmain`, branch `ddr5-timing-fix`, commit `c89988d`, and the NVSim fork `uvKogan/MBMM`, commit `9a90d44`; Appendix C gives the full statement.

References

[1] M. Ahmad, "NAND Flash's Reversal of Fortune Amid the AI Boom," *EE Times*, Jan. 26, 2026. [Online]. Available: https://www.eetimes.com/nand-flashs-reversal-of-fortune-amid-the-ai-boom/. [Accessed: Apr. 25, 2026].

[2] A. J. Fleischer, "How AI Broke the Memory Market," *Octopart Pulse*, Mar. 13, 2026. [Online]. Available: https://octopart.com/pulse/p/how-ai-broke-memory-market. [Accessed: Apr. 25, 2026].

[3] X. Dong et al., "NVSim: A Circuit-Level Performance, Energy, and Area Model for Emerging Nonvolatile Memory," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 31, no. 7, pp. 994-1007, July 2012.

[4] M. Poremba, T. Zhang, and Y. Xie, "NVMain 2.0: A user-friendly memory simulator to model (non-)volatile memory systems," IEEE Comput. Archit. Lett., vol. 14, no. 2, pp. 140-143, July-Dec. 2015.

[5] International Roadmap for Devices and Systems (IRDS), "More Moore," 2024. [Online]. Available: https://irds.ieee.org/images/files/pdf/2024/2024IRDS_MM.pdf.

[6] L. R. Upton et al., "EMBER: A 100 MHz, 0.86 mm2, multiple-bits-per-cell RRAM macro in 40 nm CMOS with compact peripherals and 1.0 pJ/bit read circuitry," in Proc. IEEE 49th Eur. Solid State Circuits Conf. (ESSCIRC), Sep. 2023, pp. 469-472.

[7] C. Matsui, A. Yamada, N. Misawa, and K. Takeuchi, "ReRAM resistance design of LRS and HRS for ultrahigh-capacity digital memory and analog Computation-in-Memory," IEICE Trans. Fundamentals, vol. E109.A, no. 3, pp. 596-603, Mar. 2026. DOI: 10.1587/transfun.2025VLP0011

[8] C. Xue et al., "16.1 A 22nm 4Mb 8b-Precision ReRAM Computing-in-Memory Macro with 11.91 to 195.7TOPS/W for Tiny AI Edge Devices," *2021 IEEE International Solid-State Circuits Conference (ISSCC)*, San Francisco, CA, USA, 2021, pp. 245-247.

[9] S. H. Jo et al., "3D-stackable crossbar resistive memory based on Field Assisted Superlinear Threshold (FAST) selector," *2014 IEEE International Electron Devices Meeting (IEDM)*, San Francisco, CA, USA, 2014, pp. 6.7.1-6.7.4.

[10] JEDEC Solid State Technology Association, "DDR5 SDRAM Standard," JESD79-5D, Nov. 2025. [Online]. Available: https://www.jedec.org/standards-documents/docs/jesd79-5d.

[11] B. C. Lee, E. Ipek, O. Mutlu, and D. Burger, "Architecting phase change memory as a scalable dram alternative," in *Proc. 36th Annu. Int. Symp. Comput. Archit. (ISCA)*, 2009, pp. 2-13.

[12] A. Samajdar, J. M. Joseph, Y. Zhu, P. Whatmough, M. Mattina, and T. Krishna, "A Systematic Methodology for Characterizing Scalability of DNN Accelerators Using SCALE-Sim," in *Proc. IEEE Int. Symp. Perform. Anal. Syst. Softw. (ISPASS)*, 2020, pp. 58-68.

[13] B. Q. Le et al., “Resistive RAM With Multiple Bits Per Cell: Array-Level Demonstration of 3 Bits Per Cell,” *IEEE Trans. Electron Devices*, vol. 66, no. 1, pp. 641-646, Jan. 2019.

[14] H.-S. P. Wong et al., “Metal-Oxide RRAM,” *Proc. IEEE*, vol. 100, no. 6, pp. 1951-1970, June 2012. DOI: 10.1109/JPROC.2012.2190369

[15] M. Lanza et al., “Standards for the Characterization of Endurance in Resistive Switching Devices,” *ACS Nano*, vol. 15, no. 11, pp. 17214-17231, Nov. 2021. DOI: 10.1021/acsnano.1c06980

[16] Data Center Knowledge, “Data Center Hardware Refresh Cutback by Microsoft - What’s Next?,” Aug. 25, 2022. [Online]. Available: https://www.datacenterknowledge.com/hyperscalers/data-center-hardware-refresh-cutback-by-microsoft-what-s-next-. [Accessed: Jul. 10, 2026].

[17] TrendForce, "Micron Races Ahead in 10nm-Class DRAM with 1γ DDR5 Samples Delivered to Intel & AMD," Feb. 26, 2025. [Online]. Available: https://www.trendforce.com/news/2025/02/26/news-micron-races-ahead-in-10nm-class-dram-with-1γ-ddr5-samples-delivered-to-intel-amd/. [Accessed: Jul. 11, 2026]. **[18]** J. Choe, "Comparing DDR5 Memory From Micron, Samsung, SK Hynix," EE Times, Feb. 15, 2022. [Online]. Available: https://www.eetimes.com/comparing-ddr5-memory-from-micron-samsung-sk-hynix/. [Accessed: Jul. 12, 2026]. **[19]** A. Shilov, "Samsung Puts 3D DRAM on the Roadmap, Stacked DRAM to Follow," Tom's Hardware, Apr. 3, 2024. [Online]. Available: https://www.tomshardware.com/pc-components/dram/samsung-outlines-plans-for-3d-dram-which-will-come-in-the-second-half-of-the-decade. [Accessed: Jul. 12, 2026]. **[20]** J. Yang et al., "A 14nm-FinFET 1Mb Embedded 1T1R RRAM with a 0.022μm² Cell Size Using Self-Adaptive Delayed Termination and Multi-Cell Reference," in Proc. IEEE Int. Solid-State Circuits Conf. (ISSCC), 2021. DOI: 10.1109/ISSCC42613.2021.9365945 **[21]** K. T. Malladi, F. A. Nothaft, K. Periyathambi, B. C. Lee, C. Kozyrakis, and M. Horowitz, "Towards Energy-Proportional Datacenter Memory with Mobile DRAM," in Proc. 39th Annu. Int. Symp. Computer Architecture (ISCA), 2012, pp. 37-48. DOI: 10.1109/ISCA.2012.6237004 **[22]** A. Gholami, Z. Yao, S. Kim, C. Hooper, M. W. Mahoney, and K. Keutzer, "AI and Memory Wall," IEEE Micro, vol. 44, no. 3, pp. 33-39, May/June 2024. DOI: 10.1109/MM.2024.3373763 **[23]** Tom's Hardware, "Intel Kills Optane Memory Business Entirely, Pays $559 Million to Exit," Jul. 28, 2022. [Online]. Available: https://www.tomshardware.com/news/intel-kills-optane-memory-business-for-good. [Accessed: Jul. 13, 2026]. **[24]** TechInsights, "Advanced TSMC 22ULL Embedded RRAM Chip Unveiled." [Online]. Available: https://www.techinsights.com/blog/advanced-tsmc-22ull-embedded-rram-chip-unveiled. [Accessed: Jul. 13, 2026]. **[25]** Tom's Hardware, "Neo Semiconductor's Revolutionary 3D X-DRAM for AI Processors Has Passed Proof-of-Concept Validation," Apr. 24, 2026. [Online]. Available: https://www.tomshardware.com/tech-industry/artificial-intelligence/neo-semiconductors-revolutionary-3d-x-dram-for-ai-processors-has-passed-proof-of-concept-validation-company-secures-funding-to-develop-next-gen-memory-hbm-alternative. [Accessed: Jul. 13, 2026]. **[26]** N. Binkert et al., "The gem5 Simulator," ACM SIGARCH Computer Architecture News, vol. 39, no. 2, pp. 1-7, May 2011. DOI: 10.1145/2024716.2024718 **[27]** J. D. McCalpin, "Memory Bandwidth and Machine Balance in Current High Performance

Computers," IEEE Computer Society Technical Committee on Computer Architecture (TCCA) Newsletter, pp. 19-25, Dec. 1995. **[28]** Standard Performance Evaluation Corporation, "SPEC CPU 2017 Benchmark Suite." [Online]. Available: https://www.spec.org/cpu2017/. [Accessed: Jul. 13, 2026]. **[29]** Micron Technology, Inc., "16Gb DDR5 SDRAM Addendum: MT60B4G4, MT60B2G8, MT60B1G16, Die Revision A," Doc. No. CCM005-0005-1684161373-30, Rev. D, Feb. 2023. [Online]. Available: https://www.micron.com/products/memory/dram-components/ddr5-sdram/part-catalog **[30]** SK hynix Inc., "16Gb DDR5 SDRAM," datasheet, n.d. [Online]. Available (registration required): https://product.skhynix.com/support/downloads.go

[31] A. Levy, L. R. Upton, M. D. Scott, D. Rich, W.-S. Khwa, Y.-D. Chih, M.-F. Chang, S. Mitra, B. Murmann, and P. Raina, "EMBER: Efficient Multiple-Bits-Per-Cell Embedded RRAM Macro for High-Density Digital Storage," *IEEE J. Solid-State Circuits*, vol. 59, no. 7, pp. 2081-2092, July 2024. DOI: 10.1109/JSSC.2024.3387566

[32] J. Izraelevitz, J. Yang, L. Zhang, J. Kim, X. Liu, A. Memaripour, Y. J. Soh, Z. Wang, Y. Xu, S. R. Dulloor, J. Zhao, and S. Swanson, "Basic Performance Measurements of the Intel Optane DC Persistent Memory Module," arXiv:1903.05714, Aug. 2019.

[33] R. Fackenthal, M. Kitagawa, W. Otsuka, K. Prall, D. Mills, K. Tsutsui, J. Javanifard, K. Tedrow, T. Tsushima, Y. Shibahara, and G. Hush, "19.7 A 16Gb ReRAM with 200MB/s Write and 1GB/s Read in 27nm Technology," in Proc. IEEE Int. Solid-State Circuits Conf. (ISSCC), Feb. 2014, pp. 338-339. DOI: 10.1109/ISSCC.2014.6757460

[34] Intel Corporation, "Ultra High Bandwidth Memory with Backend Transistors," U.S. Patent Application Publication No. US 2026/0191095 A1, Appl. No. 19/001,921, filed Dec. 26, 2024, published Jul. 2, 2026. [Online]. Available: https://www.freepatentsonline.com/y2026/0191095.html. [Accessed: Aug. 22, 2026].

[35] TrendForce, "Intel Patent Reveals XBM Matching HBM4 Footprint Without Interposers; Commercialization Seen After 2030," Jul. 8, 2026. [Online]. Available: https://www.trendforce.com/news/2026/07/08/news-intel-patent-reveals-xbm-matching-hbm4-footprint-without-interposers-commercialization-seen-after-2030/. [Accessed: Aug. 22, 2026].

[36] B. C. Lee, E. Ipek, O. Mutlu, and D. Burger, "Architecting Phase Change Memory as a Scalable DRAM Alternative," in *Proc. 36th Annu. Int. Symp. Computer Architecture (ISCA)*, 2009, pp. 2-13. DOI: 10.1145/1555754.1555758

[37] M. K. Qureshi, V. Srinivasan, and J. A. Rivers, "Scalable High Performance Main Memory System Using Phase-Change Memory Technology," in *Proc. 36th Annu. Int. Symp. Computer Architecture (ISCA)*, 2009, pp. 24-33. DOI: 10.1145/1555754.1555760

[38] P. Zhou, B. Zhao, J. Yang, and Y. Zhang, "A Durable and Energy Efficient Main Memory Using Phase Change Memory Technology," in *Proc. 36th Annu. Int. Symp. Computer Architecture (ISCA)*, 2009, pp. 14-23. DOI: 10.1145/1555754.1555759

[39] C. Xu, D. Niu, N. Muralimanohar, R. Balasubramonian, T. Zhang, S. Yu, and Y. Xie, "Overcoming the Challenges of Crossbar Resistive Memory Architectures," in *Proc. 21st IEEE*

Int. Symp. High Performance Computer Architecture (HPCA), 2015, pp. 476-488. DOI: 10.1109/HPCA.2015.7056056

[40] E. Kültürsay, M. Kandemir, A. Sivasubramaniam, and O. Mutlu, "Evaluating STT-RAM as an Energy-Efficient Main Memory Alternative," in *Proc. IEEE Int. Symp. Performance Analysis of Systems and Software (ISPASS)*, 2013, pp. 256-267. DOI: 10.1109/ISPASS.2013.6557176

[41] Y. Choi et al., "A 20nm 1.8V 8Gb PRAM with 40MB/s Program Bandwidth," in Proc. IEEE Int. Solid-State Circuits Conf. (ISSCC), San Francisco, CA, Feb. 2012, pp. 46-48.

[42] D. Kau, S. Tang, I. V. Karpov, R. Dodge, B. Klehn, J. A. Kalb, J. Strand, A. Diaz, N. Leung, J. Wu, S. Lee, T. Langtry, K.-W. Chang, C. Papagianni, J. Lee, J. Hirst, S. Erra, E. Flores, N. Righos, H. Castro, and G. Spadini, "A Stackable Cross Point Phase Change Memory," in *2009 IEEE International Electron Devices Meeting (IEDM)*, Baltimore, MD, Dec. 2009, pp. 617-620. DOI: 10.1109/IEDM.2009.5424263

[43] International Technology Roadmap for Semiconductors, 2011 Edition, Process Integration, Devices, and Structures (PIDS), Semiconductor Industry Association, 2011, pp. 10-11 (subthreshold source/drain leakage current design targets: HP logic 100 nA/μm, LOP logic 5 nA/μm, LSTP logic 10 pA/μm).

[44] C. Auth et al., "A 22nm High Performance and Low-Power CMOS Technology Featuring Fully-Depleted Tri-Gate Transistors, Self-Aligned Contacts and High Density MIM Capacitors," in *2012 Symposium on VLSI Technology (VLSIT) Digest of Technical Papers*, June 2012, pp. 131-132.

[45] Y. Y. Chen, B. Govoreanu, L. Goux, R. Degraeve, A. Fantini, G. S. Kar, D. J. Wouters, G. Groeseneken, J. A. Kittl, M. Jurczak, and L. Altimime, "Balancing SET/RESET Pulse for $>10^{10}$ Endurance in HfO_2/Hf 1T1R Bipolar RRAM," *IEEE Trans. Electron Devices*, vol. 59, no. 12, pp. 3243-3249, Dec. 2012. DOI: 10.1109/TED.2012.2218607

[46] M. Hellenbrand, I. Teck, and J. L. MacManus-Driscoll, "Progress of emerging non-volatile memory technologies in industry," *MRS Communications*, vol. 14, pp. 1099-1112, 2024. DOI: 10.1557/s43579-024-00660-2

[47] Y. Chen, "ReRAM: History, Status, and Future," *IEEE Trans. Electron Devices*, vol. 67, no. 4, pp. 1420-1433, Apr. 2020.

[48] T.-y. Liu et al., "A 130.7-mm² 2-Layer 32-Gb ReRAM Memory Device in 24-nm Technology," *IEEE J. Solid-State Circuits*, vol. 49, no. 1, pp. 140-153, Jan. 2014.

[49] J. Zahurak et al., "Process Integration of a 27nm, 16Gb Cu ReRAM," in *Proc. IEEE Int. Electron Devices Meeting (IEDM)*, Dec. 2014.

[50] C.-C. Chou et al., "A 22nm 96Kx144 RRAM Macro with a Self-Tracking Reference and a Low Ripple Charge Pump to Achieve a Configurable Read Window and a Wide Operating Voltage Range," in *Symp. VLSI Technology and Circuits Dig. Tech. Papers*, 2020.

[51] J. Zhou, K.-H. Kim, and W. Lu, "Crossbar RRAM Arrays: Selector Device Requirements During Read Operation," *IEEE Trans. Electron Devices*, vol. 61, no. 5, pp. 1369-1376, May 2014.

[52] M. K. Qureshi, J. Karidis, M. Franceschini, V. Srinivasan, L. Lastras, and B. Abali, "Enhancing Lifetime and Security of PCM-Based Main Memory with Start-Gap Wear Leveling," in *Proc. 42nd Annu. IEEE/ACM Int. Symp. Microarchitecture (MICRO)*, 2009, pp. 14-23.

[53] J. Yang, J. Kim, M. Hoseinzadeh, J. Izraelevitz, and S. Swanson, "An Empirical Guide to the Behavior and Use of Scalable Persistent Memory," in *Proc. 18th USENIX Conf. File and Storage Technologies (FAST)*, 2020, pp. 169-182.

[54] Intel Corporation, "Intel Optane Persistent Memory 200 Series Brief," product brief, 2020.

[55] Program-and-verify pulse-count characterization of multi-level-cell 1T1R RRAM, preprint, arXiv:2108.07879 (average 49 pulses per level move, median 36). [Online]. Available: https://arxiv.org/pdf/2108.07879.

[56] Self-terminating single-pulse multi-level-cell write technique for RRAM, in *Proc. Design, Automation and Test in Europe (DATE)*, 2022. ACM DL record 10.5555/3539845.3540141. [Online]. Available: https://dl.acm.org/doi/10.5555/3539845.3540141.

[57] Program-and-verify characterization of multi-level-cell phase-change memory (more than 70 iterations for a 9.7 us total write; under 1.6 us with the verify step skipped), PMC6680789. [Online]. Available: https://pmc.ncbi.nlm.nih.gov/articles/PMC6680789/.

[58] Kawahara et al., "An 8Mb multi-layered cross-point ReRAM macro with 443MB/s write throughput," in *Proc. IEEE Int. Solid-State Circuits Conf. (ISSCC)*, 2012. *The 17.2 ns write-cycle figure quoted in Section 3.1.1 is taken from the published abstract; the full text was requested and has not been read.* [Online]. Available: https://www.researchgate.net/publication/254023933_An_8Mb_multi-layered_cross-point_ReRAM_macro_with_443MBs_write_throughput.

[59] Kim et al., "High-performance, cost-effective 2z nm two-deck cross-point memory integrated by self-align scheme for 128 Gb SCM," in *Proc. IEEE Int. Electron Devices Meeting (IEDM)*, 2018 (SK hynix). *The array-level set and read figures quoted in Section 3.1.1 are second-hand; the full text was requested and has not been read.* [Online]. Available: https://ieeexplore.ieee.org/document/8614680/.

[60] Channel-model bit error rates for a 1S1R cross-point array (1024 x 1024, 10 ohm per junction, ideal selector; read and write error rates by cell position), preprint, arXiv:1912.02963. [Online]. Available: https://arxiv.org/abs/1912.02963.

[61] BCH coding over 128- and 256-bit blocks with location-dependent interleaving, applied to the 1S1R channel model of **[60]**, preprint, arXiv:2104.14011. [Online]. Available: https://arxiv.org/abs/2104.14011.

[62] Raw bit error rate of 1T1R HfO_2 RRAM arrays under cycling, including a 2T2R variant, preprint, arXiv:1902.02528. [Online]. Available: https://arxiv.org/pdf/1902.02528.

[63] Weebit Nano, smart-write algorithm technical resource (128 kb 1T1R macro: below 10^{-7} after 10^6 cycles, 28.1 dB read margin). [Online]. Available: https://www.weebit-nano.com/technical_resources/smart-write-algorithm-to-enhance-performances-and-reliability-of-an-rram-macro/.

[64] Error-rate analysis of embedded RRAM, ICCAD 2023 (raw bit error rate about 10^{-6} corrected to an uncorrectable rate of 2 x 10^{-15} by a 2-bit-per-128-bit code). [Online]. Available: https://cs.stanford.edu/ anjiang/papers/ICCAD23PBA.pdf.

[65] Endurance of TSMC 28 nm embedded RRAM under 2-bit and 3-bit error correction (500,000 cycles at 2-bit, 1,000,000 at 3-bit), IEEE Xplore document 9830374. [Online]. Available: https://ieeexplore.ieee.org/document/9830374/.

[66] Reliability of a 1S1R selector stacked with STT-MRAM (10^{-9} per-read disturb rate, 10^9 read-disturb-free cycles), IEDM 2023. [Online]. Available: https://ieeexplore.ieee.org/document/10413809/.

[67] IBM Research, "Enhancing 3D XPT/SOM reliability: strategies for mitigating spike current and improving read endurance," IMW 2025 (cell failure rate 9% falling to 0.02% after read-endurance optimization). [Online]. Available: https://research.ibm.com/publications/enhancing-3d-xptsom-reliability-strategies-for-mitigating-spike-current-and-improving-read-endurance.

[68] Alfaro Robayo et al. (CEA-Leti), OxRAM-plus-OTS 1S1R reliability, IEDM 2019 (five decades of selectivity, more than 10^6 program cycles, 10^8 read cycles, 10^9 read-disturb-free cycles). [Online]. Available: https://ieeexplore.ieee.org/document/8993439/.

[69] Subthreshold-read reliability of OTS phase-change cross-point cells (10^{12} disturb-free reads), *Microelectronic Engineering*, ScienceDirect S0167931724000807. [Online]. Available: https://www.sciencedirect.com/science/article/abs/pii/S0167931724000807.

[70] Read-disturb model for 28 nm embedded RRAM (per-bit rate below about 10^{-18}), IRPS 2020. [Online]. Available: https://ieeexplore.ieee.org/document/9265060/.

[71] Review of threshold-voltage drift in ovonic threshold switches (under 40 mV per decade in typical material stacks), *J. Semiconductors*, DOI 10.1088/1674-4926/43/10/104101. [Online]. Available: https://www.jos.ac.cn/article/doi/10.1088/1674-4926/43/10/104101.

[72] Saturation of ovonic-threshold-switch threshold-voltage drift near 10^3 seconds, preprint, arXiv:2102.07264. [Online]. Available: https://arxiv.org/abs/2102.07264.

[73] imec, "The promise of OTS-only memories for next-gen compute system architectures" (half-bias nonlinearity in the low thousands up to about 10^4). [Online]. Available: https://www.imec-int.com/en/articles/promise-ots-only-memories-next-gen-compute-system-architectures-0.

[74] Zhang, Sridharan, and Jian, chipkill-correct for persistent memory, *Proc. 51st Annu. IEEE/ACM Int. Symp. Microarchitecture (MICRO)*, 2018; consulted via ar5iv:2309.00304 (2 x 10^{-4} raw design point, BCH(2312,2048,22), about 27% coding overhead). [Online]. Available: https://ar5iv.labs.arxiv.org/html/2309.00304.

[75] ArchShield, an architectural framework for DRAM with high bit-error rates; consulted via ar5iv:1704.03991 (DRAM on-die SECDED tolerates about 10^{-6} raw). [Online]. Available: https://ar5iv.labs.arxiv.org/html/1704.03991.

[76] Gugnani et al., *Proc. VLDB Endowment*, vol. 14, p. 626, 2021 (Intel Optane's 256-byte "XPLine" error-correction block). [Online]. Available: https://vldb.org/pvldb/vol14/p626-gugnani.pdf.

[77] Intel Corporation, Intel Optane SSD 905P series product specifications (uncorrectable bit error rate under 1 sector per 10^{17} bits). [Online]. Available: https://www.intel.com/content/www/us/en/products/sku/147526/intel-optane-ssd-905p-series-1-5tb-2-5in-pcie-x4-3d-xpoint/specifications.html.

[78] Schechter et al., "Use ECP, Not ECC, for Hard Failures in Resistive Memories," in *Proc. 37th Annu. Int. Symp. Comput. Archit. (ISCA)*, 2010. DOI: 10.1145/1815961.1815980 [Online]. Available: https://dl.acm.org/doi/10.1145/1815961.1815980.

[79] Yoon et al., "FREE-p: Protecting Non-Volatile Memory against Both Hard and Soft Errors," in *Proc. IEEE Int. Symp. High Performance Computer Architecture (HPCA)*, 2011. [Online]. Available: https://www.researchgate.net/publication/221574466_FREE-p_Protecting_non-volatile_memory_against_both_hard_and_soft_errors.

[80] Read-margin analysis of 1R against 1S1R crossbar arrays, establishing the 10% minimum read-margin criterion, arXiv:2407.10466. [Online]. Available: https://arxiv.org/html/2407.10466v1.

[81] Flocke and Noll, "Fundamental analysis of resistive nano-crossbars," in *Proc. 33rd Eur. Solid-State Circuits Conf. (ESSCIRC)*, 2007. *Consulted through its bibliographic record; the full proceedings text was requested and has not been read.* [Online]. Available: https://www.semanticscholar.org/paper/Fundamental-analysis-of-resistive-nano-crossbars-in-Flocke-Noll/2c600f18e5709b537af8d140910cc019617b420e.

[82] A. Chen, crossbar-array model resolving line resistance and nonlinear device characteristics, *IEEE Trans. Electron Devices*, 2013 (line resistance as the binding constraint on read margin). [Online]. Available: https://ieeexplore.ieee.org/document/6473873/.

[83] "Effect of threshold switching selectors on one-selector one-resistor crossbar arrays" (a 10% read margin needs a half-bias nonlinearity around 8,000 and an on/off window of 5), ResearchGate record 351467888. [Online]. Available: https://www.researchgate.net/publication/351467888_Effect_of_Threshold_Switching_Selectors_on_One-Selector_One-Resistor_Crossbar_Arrays.

[84] Measured idle latency of local and remote-socket DDR5 and of CXL-attached DRAM devices, preprint, arXiv:2409.14317. [Online]. Available: https://arxiv.org/abs/2409.14317.

[85] "Pond": CXL-based memory pooling for cloud platforms (Microsoft Azure), preprint, arXiv:2203.00241 (local DDR5 platform baselines, pooling latency overheads, workload latency-acceptability study). [Online]. Available: https://arxiv.org/abs/2203.00241.

[86] Sun et al., "Demystifying CXL Memory with Genuine CXL-Ready Systems and Devices," preprint, arXiv:2303.15375. [Online]. Available: https://arxiv.org/abs/2303.15375.

[87] Compute Express Link Consortium, "About CXL" (CXL.io, CXL.cache and CXL.mem; Type 3 memory devices). [Online]. Available: https://computeexpresslink.org/about-cxl/.

[88] Linux kernel documentation, "NUMA performance" (ACPI HMAT latency and bandwidth attributes exposed per initiator-target pair). [Online]. Available: https://docs.kernel.org/admin-guide/mm/numaperf.html.

[89] Linux kernel documentation, kernel sysctl reference (numa_balancing mode 2 and the promotion rate limit). [Online]. Available: https://docs.kernel.org/admin-guide/sysctl/kernel.html.

[90] LWN.net, "Memory tiering" (abstract distance of 512; demotion disabled by default). [Online]. Available: https://lwn.net/Articles/902881/.

[91] Meta Platforms, Transparent Page Placement for CXL-enabled tiered memory, preprint, arXiv:2206.02878 (CXL adds about 50 to 100 ns over local DRAM; FPGA-based cards about 250 ns higher). [Online]. Available: https://arxiv.org/abs/2206.02878.

[92] Micron Technology, "Micron launches memory expansion module portfolio to accelerate CXL 2.0 adoption," Aug. 7, 2023 (CZ120, x8, up to 36 GB/s). [Online]. Available: https://www.globenewswire.com/news-release/2023/08/07/2719764/14450/en/Micron-Launches-Memory-Expansion-Module-Portfolio-to-Accelerate-CXL-2-0-Adoption.html.

[93] Intel Corporation, 4th Gen Intel Xeon Scalable processor overview (eight DDR5-4800 channels per socket). [Online]. Available: https://www.intel.com/content/www/us/en/developer/articles/technical/fourth-generation-xeon-scalable-family-overview.html.

[94] Izraelevitz et al., performance measurements of the Intel Optane DC persistent memory module, preprint, arXiv:1903.05714, 2019 (305 ns idle random read, 169 ns sequential read, 6.6 / 2.3 GB/s read/write per module). [Online]. Available: https://arxiv.org/abs/1903.05714.

[95] Intel Corporation, second-generation Intel Xeon Scalable processor family technical overview (Optane PMem Memory Mode and App Direct; 64-byte byte-addressable lines). [Online]. Available: https://www.intel.com/content/www/us/en/developer/articles/news/second-generation-intel-xeon-processor-scalable-family-technical-overview.html.

[96] Intel Corporation, support article: discontinuation of Intel Optane persistent memory 300 series, Jan. 31, 2023. [Online]. Available: https://www.intel.com/content/www/us/en/support/articles/000093792/memory-and-storage/intel-optane-persistent-memory.html.

[97] The Register, report on Intel’s Optane wind-down and inventory write-off, July 29, 2022. [Online]. Available: https://www.theregister.com/2022/07/29/intel_optane_memory_dead/.

[98] Google Research, software-defined far memory in warehouse-scale computers (single-digit microsecond tolerance). [Online]. Available: https://research.google/pubs/pub48551/.

[99] Astera Labs, CM5162 Leo smart memory controller product page (CXL 1.1 / 2.0, x16 at 32 GT/s, two DDR5 channels, up to 2 TB; no published latency figure). [Online]. Available: https://www.asteralabs.com/product-details/cm5162p/.

[100] Intel Corporation and Micron Technology, 3D XPoint technology announcement, July 2015. [Online]. Available: https://www.intc.com/news-events/press-releases/detail/324/intel-and-micron-produce-breakthrough-memory-technology.

[101] Micron Technology, Inc., "DDR5 SDRAM," core product data sheet, Doc. No. CCM005-1684161373-23, Rev. D, Oct. 2022, 537 pp. (speed bins Table 361; command and address timing Table 429; power-down timing Table 438). [Online]. Available: https://www.farnell.com/datasheets/3980285.pdf (accessed 2026-10-19).

Appendix A: Simulation Parameters and Literature Grounding

- **Process Node:** 22nm FinFET LOP (Low Operating Power). *Justification:* LOP was selected over High Performance (HP) to guarantee electrical convergence in the NVSim solver and to accurately reflect the strict thermal envelopes required for high-capacity Main Memory modules.
- **Resistance Targets:** $10^5\Omega$ LRS and $10^9\Omega$ HRS. *Reference:* The $10^5\Omega$ LRS floor is **Matsui et al.'s [7]** direct recommendation for high-capacity *digital* ReRAM memory, to mitigate bitline IR-drop on dense arrays; the paired $10^9\Omega$ HRS is adopted from the same paper's analog Computation-in-Memory (CiM) design point as a representative high-resistance target, since [7] does not separately specify an HRS floor for the digital-memory case. *Sensitivity:* A dedicated NVSim sweep across HRS $\in \{25, 100, 1000, 10000\} \times$ LRS (i.e. $2.5 \times 10^6\Omega$ to $10^9\Omega$, LRS held fixed at $10^5\Omega$) shows this choice has negligible practical consequence: modeled leakage power is bit-for-bit identical across the full two-orders-of-magnitude range, and read latency varies by at most 1.7% across it. The sweep was run at the pre-revision organization, so its absolute values belong to that dataset; its conclusion holds a fortiori at the matched 2048 x 2048 organization, where NVSim charges the cell no leakage at all and chip leakage is entirely peripheral. The exact HRS value is therefore not load-bearing for any finding in this book. Note that the same sweep was previously offered as evidence that the transistor-versus-selector leakage separation of that era was insensitive to HRS. That reading was correct but beside the point: the separation was insensitive to HRS because it was not a cell effect at all (see the next entry, and Appendix D). *Bitline length:* Matsui et al. set the $10^5\Omega$ LRS floor at a 1024-cell bitline, where the retained fraction of ideal bitline current is $\alpha = 0.99$ at 1 Ω per cell pitch (22nm single-level Cu). The 2048-cell bitlines simulated here give $\alpha = 0.980$ single-level and $\alpha = 0.990$ double-level, so the organization meets the source's own criterion with a double-level line.
- **Array Organization (and the withdrawal of the leakage-gap figure):** Both ReRAM cell types are characterized at one matched organization: **2048 x 2048 subarrays, sense-amp column mux 64**, forced through NVSim's `-ForceBank`, `-ForceMat` and `-ForceMuxSenseAmp` inputs and verified against the geometry NVSim itself prints, on every run, by an anchored gate that fails the run on any mismatch. *Justification:* the side length is anchored to fabricated parts of both topologies - Micron and Sony's 27nm 16 Gb 1T1R [49] and SanDisk and Toshiba's 24nm 32 Gb cross-point, whose block is 2K bitlines by 4K wordlines [48] - and to the resistance-design source's bitline model (entry above). The mux value has no chip source and is labeled an inference. *Why this entry replaces the previous "Access-Device Leakage Model":* earlier versions of this book reported a two-orders-of-magnitude leakage separation between the two cells (Appendix D) and justified it from two independently sourced leakage currents, the ITRS 22nm LOP subthreshold target [43] and Intel's measured 22nm FinFET off-state band [44] on the transistor side, against the $10^9\Omega$ HRS target [7] on the selector side. **NVSim uses none of those inputs for that number.** Its source assigns memristor cells zero leakage for every access type (`SubArray.cpp`), and accumulates chip leakage from row

decoders, bitline and sense-amp mux decoders, prechargers, muxes and sense amplifiers only. Chip leakage therefore scales with the number of mats, and the retired figure was the arithmetic consequence of two hand-set column muxes (32 and 256) driving NVSim to 256 mats for 1T1R against 2 for 1S1R. At a matched organization the two cells leak identically (108.384 mW per 1 Gb chip). [43] and [44] remain correct as external statements about transistor leakage; they simply do not corroborate a number NVSim never computed from them. The physical question they speak to - what an unselected 1T1R access transistor actually leaks in a real array - is *not modeled by NVSim at all*, and remains open.

- **Analytic Selector Layer:** NVSim models no selector physics, so the sneak-path behavior of the 1S1R cross-point is supplied by a standalone analytic module rather than by the simulator. *Method:* read margin is a reduced network solve of the read model of Zhou, Kim and Lu [51], which publishes no closed-form equation and is solved in HSPICE; the solve is validated against thirteen points of that paper's Fig. 3(b) to within 0.3 percentage points and against the array power of its Fig. 8(a). Tile validity is the handbook rule that a tile side may not exceed $I_{\mathrm{on}}/(6I_{\mathrm{leak}})$. Sneak leakage counts the half-selected cells on the active lines of the sixteen subarrays a request activates, derived from the forced organization above. *Operating point:* read voltage 1.4 V with a V/2 half-select scheme, R_{on} $10^5\Omega$ and R_{off} $10^9\Omega$ from the cell file, and 1 Ω per cell pitch for a 22nm single-level Cu line. *Bounds:* an ovonic threshold switch at nonlinearity 10^4, I_{on} 100 uA, I_{leak} 10 nA (present-day production practice), and a Crossbar-class FAST selector at 10^6 and 0.1 nA (best published) [9]. *Results:* the standby adder is **exactly zero** at both bounds, because unselected, unbiased crossbar lines have no voltage across their cells; access-time sneak power is 7.56 mW per chip (OTS) or 0.076 mW (FAST), which is an instantaneous figure during an access and must be duty-cycled before comparison, and must never be added to the 108.384 mW standby figure; read margin is 24.0% and 23.2% against a 10% requirement; and the maximum valid tile side is **1666 cells at the OTS bound** against 166,666 at the FAST bound. The simulated 2048-cell tile is therefore valid only at the better bound, which is why every 1S1R result in this book is stated as conditional on selector quality.
- **Trace Time Base and Window:** Every trace replayed in this book carries a machine-readable provenance sidecar (`benchmarks/<trace>.nvt.sidecar.json`) recording the generator command line, the source log's hash and size, the region boundaries, the timestamp unit and a full accounting of every line read. *Unit:* one trace cycle is one NVMain cycle at `CPUFreq 3000`, that is 1/3 ns, converted by exact integer arithmetic from gem5 ticks. This is a correction: the traces used in earlier versions of this book carried timestamps whose unit the parser had never established, and the window this book previously quoted was in fact 250 ms of program time (Appendix D). *Region:* gem5 runs fast-forward 500 M instructions, then switches to the detailed out-of-order CPU for 400 M (gcc) or 300 M (lbm, STREAM) instructions with L1 and L2 caches enabled; the first 10 ms after the switch is discarded as cache warm-up and the 250 ms replay window begins after it. In earlier versions the entire replayed window lay inside gem5's atomic fast-forward, before the detailed region began, which is why LBM's write rate

was read off its program-initialization burst. *Validation:* each regenerated gem5 trace reconciles exactly with gem5's own memory-controller request counters, kept plus skipped equalling `readReqs` + `writeReqs`, and a standalone validator checks alignment, duplicate records and window coverage on every trace before it is used.

- **NVSim Calibration:** The NVSim build used here was checked against upstream and against published reference points before the organization above was adopted. *Software drift: none.* On every case run - the upstream sample configurations for NVM macro, PCRAM, SLC NAND, STT-RAM cache and RRAM at both access types, plus the calibration targets and the planned re-run configurations - this build's output is identical to upstream's once local debug-print lines are removed. The only local changes are a `-std=c++11` build flag, debug prints, and a CAM-to-RAM redirect in `main.cpp`. *Against an external reference:* the NVMExplorer fork reproduces its own published 22nm LOP tutorial figures exactly, while this build and upstream differ from them, and every difference traces to that fork's rewritten sense-amplifier model (sub-22nm sense-amp delay 0.1 ns against upstream's 1.45 ns, different sense-amp energy, and zero sense-amp leakage against upstream's non-zero value). This is a *model* difference, not a defect in either build, and it is disclosed because sense-amp leakage is precisely the term that dominates this book's static-power result. *Against fabricated silicon:* calibration to a published 4 Mb 1T1R part is dominated by organization - free and forced organizations span 1.3 to 25.8 ns against a 7.2 ns measurement - which is the same sensitivity this revision's matched-organization decision addresses. NVSim's 1T1R write time is decoder delay plus the input pulse width, so write timing is not a calibration at all. A cross-point calibration target was rejected because it would require approximating a two-layer chip in two dimensions with no sneak-path or IR-drop model. *Net position:* the 1T1R characterization is organization-sensitive and anchored; the 1S1R characterization is **uncalibrated** against any fabricated part, which is the reason the published-silicon configurations of Section 3.1.1 exist.
- **DDR5 Timing and Power Corrections of This Revision:** Two defects in the DDR5 baseline were found after the fidelity audit of Section 3.1.6 had been closed, and both are corrected here. They move only DDR5: every ReRAM and PCM row in this book is byte-identical before and after, which is itself the check that the corrections are confined to where they belong. *Background power, an upstream NVMain accounting defect.* In NVMain 2.0's current-mode energy model - which only the DDR5 configurations use - a rank accumulates its background energy for all of its devices, but then divides by the device count when converting that energy to power, while activate, burst and refresh power are multiplied by the device count as they should be. The rank's printed background power, and therefore its printed total power, carry *one device's* standby draw against the whole rank's other three components. The defect is in upstream code from 2014 and is internally self-consistent, which is why the pipeline's own power-reconciliation residual could never see it. It is corrected in *post-processing*: each rank's background power is multiplied by the device count derived from the run's own configuration (BusWidth / DeviceWidth), cross-checked against NVMain's own printed "Creating N banks in all D devices" line and independently against the rank's background-energy counter. The simulator's C++ is deliberately left unpatched and the statistics files are left exactly as the runs

printed them, so the correction is reproducible from the frozen stats. For the primary two-subchannel baseline the factor is 4 (two ranks of four devices, eight on the 16 GiB module). Under GCC it therefore multiplies the module's background power by four, giving a static power of 0.724 W inside a module total of 0.797 W. The sanity check that needs no source reading: the configuration's own lowest standby current, EIDD2P0 at 46.87 mA and 1.1 V across eight devices, is a 0.412 W floor, and the uncorrected figure sat *below* its own floor. ReRAM and PCM run under NVMain's other energy models, which never perform that division, so their factor is 1 and no ReRAM or PCM power figure moved. *Write-path, power-down and activate-window timings.* The DDR5 configuration's core timings (tCAS, tRCD, tRP, tRAS, tRFC, tREFW) were already sourced, but its write path, its power-down path and its activate window were still DDR3-1333 template cycle counts running at 2400 MHz. They are now taken from JESD79-5 for the DDR5-4800 bin, in interface cycles at tCK = 0.4167 ns: **tCWD 7 → 38** (CWL = CL - 2, Table 471), **tWR 10 → 72** (30 ns, Table 521), **tRTP 5 → 18** (max(12 nCK, 7.5 ns)), **tWTR 5 → 6** (tWTR_S = max(4 nCK, 2.5 ns)), **tPD and tXP 6 → 18** (max(7.5 ns, 8 nCK), Table 272), **tRDPDEN 24 → 49** and **tWRPDEN 19 → 119** (Table 272's formulas evaluated for this configuration), **tRRD 5 → 8** (tRRD_S) and **tFAW 20 → 32** (tFAW_1K = max(32 nCK, 13.312 ns), Table 521). Four keys are left at their template values with reasons: tXPDLL, because DDR5 defines no DLL-off slow-exit power-down and NVMain never reaches that branch under the fast-exit mode used here; tXS and tXSDLL, because NVMain models no self-refresh entry at all; and tOST and tRTRS, which are NVMain-specific bus-turnaround terms with no JEDEC counterpart. *Scope and provenance.* Where NVMain keeps one rank-wide value and JEDEC splits the parameter short-versus-long by bank group - tCCD, tWTR and tRRD - the **short** value is used, which is the DDR5-friendly choice and is recorded as a judgement rather than a transcription (Section 2.3). NVMain also charges tRRD, and advances its four-activate window, on every refresh as well as every activate, which JEDEC does not do for an all-bank refresh; that is template behavior and makes the baseline slightly pessimistic. The values were read from the JESD79-5 tables through a public mirror; on 2026-10-19 every one of them was checked against Micron's DDR5 SDRAM core data sheet [101], the vendor's full transcription of JESD79-5 (speed bin Table 361, command timing Table 429, power-down Table 438), and against two DDR5-4800 module datasheets, and **all agree**, the power-down entry formulas included; Micron's 16 Gb DDR5 addendum [29] separately confirms the 4800B speed bin, CL 40, tRCD, tRP and tCK. The JEDEC text itself remains unread (access-gated). *Effect.* DDR5's average total latency rose by between 6.4% (AlexNet IFMAP) and 47.5% (AlexNet OFMAP) across the six traces; module power moved by at most 3%, in both directions, because this is a timing correction and not a power one. Both corrections were verified end to end through `mbmm_master.py`.

- **MLC Penalties:** 1.5x Read Latency, 3.263x Write Latency, 1.1x Read Energy, 3.0x Write Energy (2 bits/cell vs. 1 bit/cell). *Reference:* Directly measured on the same EMBER macro, across its two publications by the same author group. Read energy (1.0/1.1 pJ/bit at 1/2 b per cell) is from **Upton et al. [6]** (ESSCIRC 2023), Table I. Read latency, write-verify bandwidth, and write-verify energy are all from the journal follow-up, **Levy et al. [31]** (IEEE JSSC 2024):

read bandwidth 2.4/1.6 Gbps at 1/2 b per cell (Section V.A / Abstract) gives the 1.5x read-latency multiplier via the same bandwidth-ratio method as write-latency; write-verify bandwidth 12.4/3.8 Mbps and write-verify energy 0.40/1.2 nJ/bit at 1/2 b per cell (Section V.B) give the 3.263x and 3.0x multipliers. The ESSCIRC conference paper reports no write-verify data split by bits-per-cell (only an aggregate, non-split SET/RESET pulse-energy estimate) and no 2-bit/cell read-latency figure at all - its Table I “Read Time (1 b)” row lists only 1-bit/cell numbers for every compared macro, including EMBER’s own. Earlier drafts of this book used unsourced 3x/4x placeholder multipliers attributed to “EMBER Macro analytical heuristics” that do not exist in either EMBER publication; a subsequent correction replaced these with 1.917x/3.263x/1.1x/3.0x, but the 1.917x read-latency figure was itself a data error - it paired EMBER’s own 12 ns read time with a “23 ns” figure that Table I attributes to a different, competing macro’s 1-bit/cell measurement, not EMBER’s 2-bit/cell number. The 1.5x figure above corrects this using EMBER’s own bandwidth data (Section 3.1.6, items 13 and 14 document both correction rounds and the resulting re-simulations).

- **Bit-Cell Area:** 20 F^2 for 1T1R, 4 F^2 for 1S1R. *Justification:* 20 F^2 reflects the physical reality of CMOS-compatible access transistors, whereas 4 F^2 represents the idealized limit of cross-point selector technology. The 1T1R figure is transistor-limited, not memristor-limited: the memristor occupies the back-end-of-line above the cell, while the access transistor must be sized wide enough to source the SET/RESET programming current [14]. A production-grade 14nm FinFET embedded 1T1R macro reports a 0.022 µm² cell - about 112F² at that node [20] - so 20F² is generous relative to planar/FinFET logic-transistor integration, which makes the density verdict against it conservative *for that architecture*. It is not generous in absolute terms: DRAM-process integration using a buried recessed-channel access transistor rather than a planar logic transistor has been demonstrated at 6F² - parity with DRAM's own 6F² cell - in a commercial 16Gb, 27nm part [33]. That device pairs the recessed transistor with a Cu-filament CBRAM switching layer rather than the HfOx/TaOx oxide-RRAM family modeled here, and NVSim's access-device model in this project (width-driven CMOS or diode only, no recessed-channel option) cannot represent it, so no simulated results are claimed for that configuration anywhere in this book (see Section 4.2 for the future-work case). [33] stands as an existence proof that 1T1R density is architecture-dependent, not a validated data point: 20F² should be read as a mid-range, logic-compatible assumption, not a technology ceiling. Note that the 4F² figure is the array-level cell footprint; peripheral circuitry (sense amplifiers, row decoders, column multiplexers) adds overhead, and the periphery-inclusive die-level density ratios are reported in Section 3.3 and Figure 26 (1S1R: 1.24x SLC / 2.47x MLC versus DDR5 at the matched organization; earlier versions reported 1.92x / 3.84x from a two-mat organization whose periphery was implausibly small).
- **Memory Controller Queue Depth:** The FRFCFS memory controller (`MemControl/FRFCFS/FRFCFS.cpp`) reads a single combined read+write `QueueSize` key, falling back to a hardcoded 32 entries when unset. A separate `ReadQueueSize`/`WriteQueueSize` pair exists only in the different FRFCFS-WQF controller, which this project's ReRAM and DDR5 configurations

never use - only the PCM baseline runs under FRFCFS-WQF. Every configuration generated by this project's pipeline left `QueueSize` unset prior to this study, silently relying on that hardcoded default rather than a documented, deliberate choice; `3_gen_nvmain_config.py` now writes it explicitly for the ReRAM configurations it generates (still defaulting to 32, so no headline result in this book changes). **The DDR5 baseline is a static configuration file, not a generated one**, and the only queue keys it carries are the `ReadQueueSize`/`WriteQueueSize` pair that plain FRFCFS ignores, so DDR5 runs on the hardcoded 32 throughout this book and the `--queue-size` flag never reaches it. *Sensitivity:* the sweep was re-run at the corrected configurations and on the regenerated traces, at depths 8, 32 and 128, on GCC, LBM and the AlexNet output map. The pre-revision sweep is superseded and its completion figures are retired, because they came from a trace that offered roughly four times LBM's real request rate (Appendix D). *Completion is invariant:* every ReRAM configuration completes the identical request population at every depth - 519,568 under GCC, 5,564,704 under LBM and 135,423 under the AlexNet output map - so depth changes timing only, never service. *Latency on the CPU traces is invariant too:* 1T1R SLC averages 41.4 ns under GCC and 43.1 ns under LBM at all three depths, 1S1R SLC 36.8 and 38.1, 1T1R MLC 44.5 and 42.9, 1S1R MLC 37.9 and 35.5, in each case to the last digit the CSV carries. At 2.1 and 22.3 million requests per second (519,568 and 5,564,704 admitted over the 250 ms window) the queue never fills, so its depth cannot matter. *On the write-dominated AlexNet output-map burst the two latency measures move in opposite directions*, which is the result that justifies reporting end-to-end latency at all:

Track	Total, Q8	Total, Q32	Total, Q128	E2E, Q8	E2E, Q32	E2E, Q128
1T1R SLC	124.9 ns	353.1 ns	1,049.2 ns	680.1 µs	486.1 µs	432.5 µs
1S1R SLC	135.7 ns	391.5 ns	1,161.6 ns	753.0 µs	543.2 µs	479.0 µs
1T1R MLC	142.8 ns	512.0 ns	1,675.0 ns	830.7 µs	780.2 µs	765.0 µs
1S1R MLC	172.1 ns	652.9 ns	2,160.3 ns	1,039.3 µs	1,010.7 µs	991.3 µs

A shallower queue reports a faster average over the few requests it admits while the rest of the burst waits outside the controller, where the total-latency counter cannot see it; a deeper queue admits the backlog, so the in-controller average rises by 8.4x from depth 8 to 128 while the time the workload actually waits falls by 1.6x. *Scope:* the `--queue-size` flag reaches the generated ReRAM configurations only. The DDR5 and PCM baselines are static configuration files and keep their own queue depths across the entire sweep, so no DDR5 or PCM number in this book varies with it. *Conclusion:* this book's headline results retain the documented 32-entry default throughout, and the depth it is set to changes no conclusion, only which of the two latency statistics flatters the configuration.

- **The Write-Pulse Placement Flag (`--write-pulse-on-path`):** *What it changes.* By default `3_gen_nvmain_config.py` writes the NVSim cell write latency into `tWR`, the write recovery time, and leaves NVMain's `tWP` at its default of zero. With this flag the same cycle count is written into `tWP` instead and **`tWR` is held at one cycle**, not at zero. That one-cycle floor is the

tWR ruling of this flag, and its effect is that the flag *moves* the pulse rather than duplicating it: the two datasets differ by where a single number is booked and by nothing else. Both mappings are enforced by `3_gen_nvmain_config.validate_config()`, which re-reads the timings out of the rendered configuration text before the file is written, so a run in either mode fails loudly rather than silently charging the cell twice or not at all. *Mechanism.* `tWP` is what NVMain's `WriteCellData` returns and so becomes the write timer, which means every write's own completion carries the pulse; in the plain write branch the subarray's next read and next write carry it too, while after a row-closing write the subarray's next activation is unchanged to within one cycle (Section 2.3, Section 3.1.1). *Scope.* Like every other generator flag, it reaches the generated ReRAM and published-silicon configurations only - the DDR5 and PCM baselines are static files, so their rows are identical in the two datasets - while the `Run_Write_Pulse_On_Path` provenance column describes the *run* and therefore reads `true` on those baseline rows as well. *Dataset.* `results/system_rev2026-09_writepulse/`, with the processed metrics under its `_csv/` subdirectory; the results are Table 9 and the analysis is Section 3.1.1. *Reproduction note.* That dataset's `run_manifest.json` carries the schema string `mbmm_run_manifest/1` rather than the current one, because the manifest schema was bumped while this run was in progress: its configurations and its manifest were written before the edit landed. The manifest is otherwise complete and the run is not affected; a reproduction that validates manifests against the current schema string should expect this one file to differ.

- **Module Capacity and the Address Footprint (the mechanism behind Section 3.2's scaling result):** *Re-derived 2026-09 against the corrected module capacity and the regenerated traces; the pre-revision version of this entry reasoned over a decoded address space 64x the physical module and its arithmetic is retired (Appendix D). Capacity.* `3_gen_nvmain_config.py`'s `geometry()` now sizes each architecture so that the decoded capacity equals the physical module, verified against NVMain's own per-channel capacity print: 1 GiB at eight chips, 2 GiB at sixteen (two channels of 1 GiB) and 8 GiB at the 64-chip full DIMM for the SLC tracks, with MLC twice each. The one exception is the single-chip architecture, where the generator keeps a `max(..., 65536)` row-count floor and NVMain consequently decodes 32 GiB from one 1 Gb device: that point is a compatibility floor, not a module, and Section 3.2 excludes it from every conclusion. *Interleave granularity.* With `COLS` = 1024 and 64-byte lines, one rank of one channel covers $2^{10} \times 2^{6}$ = 65,536 bytes of consecutive addresses. The channel field sits below the rank field, so a two-channel module alternates channels first and rolls into the next rank only after 128 KiB. *Footprints, counted over the requests admitted inside the 250 ms window (the three AI traces end inside the window, so for them the in-window and whole-trace figures coincide; for the three CPU traces they do not, and the in-window figures are the ones that govern):* GPT-2 IFMAP 1,024 distinct 64-byte lines over 64.0 KiB; AlexNet layer 1 IFMAP 1,094 lines over 68.4 KiB; AlexNet layer 1 OFMAP 2,116 lines over 132.2 KiB; GCC 276,294 lines over a 63.0 MiB span; LBM 1,648,067 lines over 345.4 MiB; STREAM 3,248,128 lines over 229.5 MiB. (Whole-trace, the three CPU figures are 289,137, 3,137,944 and 3,750,024 lines, which is what a scan without a window filter reports.) *Prediction and confirmation.* The model predicts that the two AI read bursts occupy exactly one rank per

channel, that the AlexNet output map occupies two, and that the three CPU traces occupy all four. NVMain's own per-rank counters at the full DIMM confirm all three, for channel 0: GPT-2 IFMAP 32,767 reads on rank 0 and zero on ranks 1 to 3; AlexNet IFMAP 634,815 on rank 0 and zero elsewhere; AlexNet OFMAP 27,008 writes on rank 0 and 40,703 on rank 1 (40,704 on channel 1), zero on ranks 2 and 3; GCC 34,697 to 38,926 reads across the eight ranks of the two channels, a spread of 12%; LBM and STREAM even to within 0.3%. *What this does and does not explain.* It explains why rank depth changes nothing for the AI bursts: the addresses never reach the ranks. It does *not* explain the CPU traces, which do span all eight ranks and still gain at most 7.5% from the sixteen-to-sixty-four-chip step - those are limited by the rate at which the trace offers requests, not by the module (their delivered bandwidth is identical at all four architectures). So the corrected statement is narrower than the pre-revision one: address footprint is a *necessary* condition for a scaling benefit, not a sufficient one. *Channel versus rank, isolated.* Because the eight-to-sixteen-chip step doubles channels and chips at once, the two AI input maps were re-run with the channel count pinned at one: the eight-, sixteen- and 64-chip points then collapse onto one latency (240.6 ns at all three under GPT-2 IFMAP; 241.2, 242.0, 242.0 under AlexNet IFMAP), queue term flat at 211.5 to 212.7 ns and device term flat at 29.1 to 29.3 ns. With two channels the sixteen- and 64-chip points fall to 143.3 and 143.6 ns, the queue term halving while the device term does not move. The 1.68x is therefore admission queueing at a second controller, measured, not inferred. *Queue depth ruled out, again.* The re-run QueueSize sweep (entry above) changes no CPU-trace latency at any depth and changes the AlexNet output-map burst identically at every architecture, so it is not an alternative explanation for any of this. *Scope.* The two AI traces are single-layer SCALE-Sim captures, and a 1,024-line footprint is a property of that capture rather than of AI inference as a workload class; a representative full-model trace remains future work (Section 4.1).

Appendix B: Execution Pipeline and Command Interface

To facilitate replication of the system scaling and trace execution, the following command-line interface (CLI) instructions document the exact mbmm_master.py parameters used to generate the results in Section 3.

B.1: Configuration Generation. The system architecture factory maps hardware constraints (for example forcing a 64-bit bus width for single-chip models), sets the DIMM clock to 800 MHz, pins the host reference frequency to `CPUFreq 3000` for every configuration, forces the 2048 x 2048 subarray organization through NVSim, and validates each configuration it writes against four explicit rules, refusing any configuration whose interface clock exceeds the host reference clock, whose tCCD is below tBURST, whose tRAS is below tRCD + tBURST, or whose tRCD + tCAS differs from the device read latency rounded up to whole interface cycles, *ceil(read latency in ns / cycle time in ns)*:

python3 3_gen_nvmain_config.py --freq 800

B.2: The primary matrix. One master invocation generates the primary dataset behind Tables 1 to 4 and Table 8, and the figures rendered from it (Figures 1 to 18 and 20 to 27). It does *not* generate Table 5, the DIMM-wide wear figures of Section 3.1.4, the selector bounds of Section 3.1.2 or Figure 19, each of which comes from a separate tool; Appendix C says which. The window is specified in nanoseconds and each technology's cycle budget is derived from its own clock, so every configuration replays the identical 250 ms of program time:

python3 mbmm_master.py --all \

--trace gcc_spec2017.nvt lbm_spec2017.nvt stream.nvt \

gpt2_ifmap.nvt alexnet_layer1_ifmap.nvt alexnet_layer1_ofmap.nvt \

--window-ns 250000000 --ddr5-model DDR5_4800_DRAM_subchannel \

--channels 2 --decoder StartGap --endurance-model RowModel --silicon

This expands to 120 runs: four ReRAM cell tracks (1T1R and 1S1R, SLC and MLC) across four chip-count architectures, the two published-silicon configurations, the DDR5-4800 subchannel baseline and the PCM baseline, each against six traces. `\-\-silicon` adds the two fabricated-timing configurations of Section 3.1.1; `\-\-decoder StartGap` selects the wear-leveling remapper of Section 3.1.4; `\-\-endurance-model RowModel` enables the per-location write counters. Post-processing reads the clocks from each stats file rather than from a table, and any unparsable stats file fails the run.

B.3: Endurance and wear analysis. The projections of Table 5 are produced from the primary matrix's statistics, not from the simulator directly:

python3 tools/aggregate_wear.py results/system_rev2026-09_primary/
stats_<model>_<trace>.out

python3 endurance_sensitivity.py

The first prints and stores DIMM-wide wear (touched locations, total writes, maximum writes to one location, mean over touched locations, hot-spot factor and the top sixteen locations); the second sweeps endurance rating, capacity, wear-leveling policy and write reduction, and emits `results/endurance_table.csv`.

Appendix C: Reproducibility Statement

Repository: the superproject is https://github.com/uvKogan/Memristor-based-dram (branch `master`). The complete simulation stack (NVSim 22nm FinFET LOP models, NVMain 2.0 configurations, workload traces, and all post-processing scripts) is available under the MIT License.

Software versions: NVSim (patched; fork https://github.com/uvKogan/MBMM, branch `master`, commit `9a90d44`), NVMain 2.0 (patched; fork https://github.com/uvKogan/NVmain, branch `ddr5-timing-fix`, commit `c89988d`), gem5 v25.1, SCALE-Sim v2, Python 3.10+, matplotlib 3.9, pandas 2.2. Both simulators are carried as git submodules of the superproject, whose `.gitmodules` names the two forks, so the commits are pinned by the superproject rather than quoted from a file. Neither commit exists in the upstream SEAL-UCSB repositories: both trees carry this project's patches. The NVMain pin is `c89988d`, not the `53cea70` quoted in earlier copies of this appendix. The two commits hold the same C++ sources and differ only in the DDR5 configuration files (the JEDEC DDR5-4800B timing corrections of 2026-09-21, Appendix D), so the ReRAM and PCM results, run at `53cea70`, and the DDR5 results, run with the corrected configurations committed as `c89988d`, are all reproduced from `c89988d`.

Recipe. `git clone --recursive https://github.com/uvKogan/Memristor-based-dram.git` checks out both pins; the B.2 command then regenerates the primary matrix; and `typst compile --font-path fonts Project_Book.typ`, and the same for `Technical_Report.typ`, rebuild the two documents from `documents/MBMM_Book_Typst/`.

Pipeline interface. The master orchestration script mbmm_master.py takes the replay window in *nanoseconds* (`--window-ns`), converting it to each technology's own cycle budget from that technology's own clock; a `--cycles` override, which earlier versions of this appendix documented, is retired, because it defeats the matched-window correction of Section 3.1.6, item 11. The other flags used in this book are `--trace` (one or more NVMain trace files), `--models`, `--ddr5-model`, `--channels`, `--decoder`, `--endurance-model`, `--freq`, `--queue-size`, `--organization` and `--silicon`. The last four reach the generated ReRAM configurations only; the DDR5 and PCM baselines are static configuration files.

The runs behind this book. The primary matrix is the B.2 command: 120 runs (20 configurations x 6 workloads), executed on 2026-09-20 in **87 minutes** on a standard Linux workstation, into `results/system_rev2026-09_primary/` with post-processed CSVs in `results/rev2026-09_primary_csv/`. Seven sensitivity axes followed the same day, each the same command with one axis flag changed and with three traces rather than six (gcc, lbm and the

AlexNet output map: one light, one heavy, one write-dominated burst), 54 runs each, three and a half hours in total: decoder Default, one channel, the DDR5 64 B cross-check, interface clock 1333 and 2400 MHz, and queue depth 8 and
128. Two further runs stand outside that series because they need their

own staging: the 1024 x 1024 organization axis (`--organization 1024`, 54 runs, half an hour) and a two-trace one-channel control on the AI input maps (36 runs). Every run exited 0.

What the master produces, and what it does not. The master executes NVSim characterization, metric extraction, configuration generation, NVMain execution, metric processing and three figure generators (`visualize_results.py`, `visualize_pareto.py`, `visualize_hero_graphs.py`), so Tables 1 to 4, Table 8 and Figures 1 to 18 and 20 to 27 are produced under it. Four analyses are *not*: Table 5 comes from `endurance_sensitivity.py`, the DIMM-wide wear figures of Section 3.1.4 from `tools/aggregate_wear.py`, the selector bounds of Section 3.1.2 from `selector_layer.py`, and the deck charts from `visualize_slides.py`. Figure 19 comes from neither: it is retained from the pre-revision ReadVoltage sweep (`results/sweep_voltage_results.json`), which this revision did not re-run (Section 3.1.5). The four scripts above are invoked directly and write into `results/`, so the gate-keeper discipline of Section 2.2 covers the simulation pipeline and not them; each was instead validated against unit tests and, for the endurance projection, against an independent write-by-write simulator (Section 3.1.4).

Traces. Every trace replayed in this book ships a machine-readable provenance sidecar (`benchmarks/<trace>.nvt.sidecar.json`) recording the generator command line, the source log's hash and size, the region boundaries, the timestamp unit and a full accounting of every line read (Appendix A). **The `.nvt` trace files themselves are not distributed, and neither are the raw gem5 logs**: the SPEC CPU2017-derived material cannot be redistributed under its license, and the logs run to tens of gigabytes. The sidecars are, so any trace can be regenerated from its recorded command line and checked against its recorded hash.

All statistical outputs (processed_bar_chart_metrics.csv, processed_pareto_metrics.csv, processed_hero_metrics.csv, processed_geometric_means.csv) and generated figures are written to results/ and are fully reproducible from the raw NVMain stats files, which the frozen dataset folders above retain.

Appendix D: What Changed Since the 3 September Version

This revision corrects the 3 September 2026 version of this book. The old PDF is preserved unchanged in the project repository and the pre-revision result set is frozen under `results/archive_2026-09_pre_revision/`. Every row below names the superseded value, the corrected value, the mechanism that produced the error and where the investigation is recorded. Evidence paths are relative to `documents/MBMM_Book_Typst/` unless they begin with `results/` or `benchmarks/`. **This appendix is the only place in this book where the retired headline values are tabulated.** Since the part-B revision, Section 3.2 and both of the Appendix A entries that previously carried superseded figures have been rewritten from the primary dataset. Two dataset notes remain outside this appendix, both on standalone NVSim sweeps that this revision did not re-run and both stating their own scope in place: Figure 19 and Section 3.1.5 (the ReadVoltage sweep), and the HRS sensitivity entry of Appendix A.

One reading note applies to every DDR5 row below. **The 3 September column prints what that version printed**, and every DDR5 power, PDP and latency figure in it carried the two defects described in Appendix A, “DDR5 Timing and Power Corrections of This Revision”: its module background power was reported for one device instead of the eight that configuration carried per rank, and its write-path, power-down and activate-window timings were DDR3-1333 template cycle counts. No corrected 3 September figure is given, because no run ever produced one; the old column is a record of what was published, not a quantity to be compared arithmetically against the new one.

Claim	3 September value	This version	Cause	Evidence
Array organization	1T1R at mux 32 (256 mats), 1S1R at mux 256 (2 mats); never stated in the book	Both forced to 2048 x 2048 subarrays at mux 64, verified against NVSim’s printed geometry on every run	The two cells had been characterized at hand-set, unequal column muxes, so every device-level comparison confounded cell type with organization	research_notes/leakage_47x_organization_artifact.md, sections 1-5 and 12
Per-chip leakage	794.7 mW (1T1R) against 16.9 mW (1S1R): a 47x gap, called the deciding architectural fact of the evaluation	108.384 mW for both cell types, identical to three decimals	NVSim assigns memristor cells zero leakage; chip leakage is peripheral circuitry and scales with mat count, so the 47x gap was the 128x mat gap	same note, sections 1 and 5; results/rev2026-09_primary_csv/hardware_metrics.json
NVSim device latency	1T1R 32.13 ns read / 32.28 ns write; 1S1R 52.13 / 72.89, “the selector costs 1.6x on reads”	1T1R 10.12 / 15.26; 1S1R 4.70 / 24.59 - the selector is the faster reader	Latency is organization-dependent in NVSim; the read ordering inverts at a matched organization	Table 1; same note, section 5, finding 3
Module power, GCC, full DIMM	1T1R 50.870 W, 1S1R 1.118 W, DDR5 0.623 W; “1T1R infeasible even power-gated”	6.939 W (1T1R), 6.940 W (1S1R), 0.797 W (DDR5); per gigabyte, ReRAM SLC is 12.7-17.4x DDR5 and MLC 6.4-8.7x	The leakage correction above, plus the DDR5 module correction below and the DDR5 background-power correction two rows down, which the 3 September figure also	Section 3.1.2, Table 3; processed_bar_chart_metrics.csv

Claim	3 September value	This version	Cause	Evidence
			carried; both modules were also the wrong capacity	
Die-level density vs DDR5	1S1R 1.92x SLC / 3.84x MLC; 1T1R 0.22x / 0.44x	1S1R 1.24x / 2.47x; 1T1R 0.36x / 0.73x	Die area is organization-dependent; at the matched organization the areas are 3.540 and 12.008 mm² per 1 Gb chip	Section 3.3, Table 6; processed_hero_metrics.csv
Selector sneak-path physics	Not modeled; the selector's advantage was asserted from NVSim leakage	Analytic layer at two bounds: zero standby adder; 7.56 mW (OTS) or 0.076 mW (FAST) per chip during an access; the OTS bound cannot keep a 2048-cell tile valid (ceiling 1666)	NVSim models no selector physics at all, so the omission is closed by a standalone analytic module rather than by the simulator	Review_Fixes_Tracker.md, T2.7 entry; results/ selector_layer.json
Read latency path	The full NVSim device read latency was written into both tRCD and tCAS	tRCD + tCAS sums exactly to the device read latency rounded up to whole interface cycles; the generator's validation refuses any configuration in which tRCD + tCAS differs from ceil(read latency / cycle time)	A generator defect: every read was charged the device twice. The largest single correction of this revision	research_notes/ interface_clock_study.md; research_notes/ revision_plan.md decision 37
Average total latency, GCC, full DIMM	DDR5 90.2 ns, 1T1R SLC 136.2 ns (1.51x DDR5), 1S1R SLC 192.3 ns	DDR5 83.1 ns, 1T1R SLC 41.4 ns (0.498x DDR5), 1S1R SLC 36.8 ns	Read double-count, trace regeneration, the organization change, the corrected DDR5 module and the corrected DDR5 JEDEC write path, compounding	Section 3.1.1, Table 2
AI-inference latency deficit	"Trailing DDR5 by 4.6x under high-parallelism AI inference"	Faster than DDR5 under GPT-2 (143.3 and 130.1 ns against 222.1); every technology is queue-bound there, with end-to-end latencies of 98-139 us	Read double-count plus the tenfold AI-trace undercount below	Section 3.1.1
Published-silicon reference	Absent; NVSim's nanosecond projections stood alone	1T1R SILICON and 1S1R SILICON run on every trace: 11.9 us and 379 us under GCC, completing 18% and 1% of the LBM window	Added because no fabricated ReRAM part reads or writes in nanoseconds, and the book had no counterweight to say so	Section 3.1.1, Table 2; research_notes/ revision_plan.md decisions 9 and 19
Geometric-mean PDP	DDR5 103.3, 1S1R SLC 738.1, 1T1R SLC 21,508.4 W·ns; a 29x intra-ReRAM gap	DDR5 136.21, 1S1R SLC 594.5, 1T1R SLC 641.0 W·ns; a 1.08x intra-ReRAM gap. Per	The intra-ReRAM gap was the leakage artifact propagated into the power term. DDR5's own figure rose rather than fell, because	Section 3.1.3, Table 4; processed_geometric_means.csv

Claim	3 September value	This version	Cause	Evidence
		gigabyte the ratios to DDR5 are 4.7x to 9.4x	its 3 September value carried both the background-power undercount and the DDR3-template write path (two rows below)	
Trace time base and window	Trace cycles interpreted on a basis the parser never established; the window was stated as 83.33 ms	Exact integer NVMain cycles at CPUFreq 3000 (1 cycle = 1/3 ns); the window is 250 ms, recorded in a sidecar per trace	The parser never established the unit it emitted, and the book's stated window followed from that assumption	research_notes/ trace_timebase_investigation.md
Trace capture and region	"Uncached raw CPU-to-memory stream, first 10M instructions"; no region of interest, no warm-up	Full out-of-order runs with L1 and L2 caches, 500 M instructions fast-forwarded, 400 M (gcc) or 300 M (lbm, STREAM) detailed, first 10 ms discarded as warm-up	The documented caveat was contradicted by the pipeline's own evidence; the traces were regenerated from scratch	Section 3.1.6 item 6; benchmarks/ sidecar files
gem5 trace record counts	gcc 236,562 reads / 170,800 writes; lbm 9,481,308 / 6,965,793; the parser kept about four records per real request	gcc 545,566 records; lbm 10,779,425; STREAM 48,439,056, each reconciling exactly with gem5's own readReqs plus writeReqs	The old line-matching rule matched several debug lines per request, duplicating every access fourfold	Section 3.1.6 item 6; Revision_Workflow_2026-09.md session log, 2026-09-20
STREAM provenance	A hand-written synthetic copy kernel, 99,999 reads / 100,000 writes, described as the industry benchmark	The real STREAM benchmark, statically linked, OpenMP off, traced through gem5: 3,996,254 reads / 1,998,124 writes in window	The stand-in had been described as the real benchmark	Section 2.4; research_notes/ revision_plan.md decisions 10 and 34
AI trace address counts	GPT-2 IFMAP 6,554 records; AlexNet IFMAP 184,320; AlexNet OFMAP 13,543; padding markers written as real addresses	65,536; 1,269,600; 135,424 - undercounts of 10.0x, 6.9x and 10.0x respectively, with padding dropped	The SCALE-Sim parser kept one address per trace row instead of every address in the row, and wrote the padding marker as an address	Revision_Workflow_2026-09.md session log, 2026-09-20 (T3.3)
AlexNet layer identity	"AlexNet Layer 1", implying the first convolution layer	SCALE-Sim's layer1 folder is zero-indexed: it is AlexNet's second convolution layer (Conv2), in this and every earlier version	A naming assumption, settled by reproducing the old traces byte for byte from the known folder	Section 2.4; same session log
AI trace duration	Treated as sustained workloads	Microsecond bursts: 2.2 us (GPT-2), 60.5 us (AlexNet IFMAP), 4.5 us (AlexNet OFMAP) inside a 250	The spans were never stated, so burst rates were read as sustained requirements	Section 2.4; Section 3.1.4, Table 5 note

Claim	3 September value	This version	Cause	Evidence
		ms window; every derived rate is flagged burst-derived		
Seventh workload (mcf)	Planned as 505.mcf_r and named in the revision plan	Absent. gem5 panics inside the benchmark's own input reader 0.13 ms into the detailed region, identically on two attempts	A benchmark-binary fault, not a pipeline fault; recovering it needs a rebuilt binary	Section 2.4; Revision_Workflow_2026-09.md session log, 2026-09-20
SPEC benchmark names	602.gcc and 619.lbm (the speed versions)	502.gcc_r and 519.lbm_r (the rate versions), which are the binaries actually built and traced	The book named benchmarks that were never built in this environment	Section 2.4; research_notes/ trace_timebase_investigation.md section 8
LBM completion	DDR5 100%, 1T1R SLC 39.8%, 1S1R SLC 28.0%, 1T1R MLC 22.4%, 1S1R MLC 13.5%, PCM 3.9%; read as a device-speed ordering	DDR5 and all four ReRAM tracks 100% (5,564,704 requests); PCM 50.2%; published silicon 18.2% and 1.0%	The old trace offered roughly four times LBM's real request rate, from inside its start-up burst	Section 3.1.1; processed_bar_chart_metrics.csv, Completed_Requests
Simulated module capacity	The ReRAM full DIMM decoded 512 GB of address space against an 8 GB physical module, described as a harmless generator artifact	8 GiB SLC (16 GiB MLC), matching the physical module exactly	It was not harmless: it changed which rank an address decoded to, which is the mechanism behind the scaling results	research_notes/ revision_plan.md, NVMain prototype findings, config defect 1
DDR5 baseline module	A single 64-bit channel, 32 devices and 32 GiB, while the prose claimed two 32-bit subchannels and BL16	Two independent 32-bit subchannels of 32 banks each, 8 devices, 16 GiB, 64 B as a 16-beat burst: prose and configuration now agree	The configuration had never implemented the model the book described	Section 2.3; research_notes/ revision_plan.md decision 41
Access granularity and channels	Granularity unstated; ReRAM single-channel against dual-channel DDR5	64 B for every technology; two channels primary, matched to DDR5's two subchannels, for the 16-chip and full-DIMM ReRAM architectures only	A 128 B study found it worse on every modelable axis and unmodelable in NVMain; channels are formed by splitting ranks, so one-rank modules stay single-channel	research_notes/ access_granularity_study.md; research_notes/ revision_plan.md decisions 38-39
DDR5 refresh share	46.6% of module power under GCC; a 33-47% band	9.1% under GCC; a 6.5-9.1% band	The share is computed against the corrected 16 GiB module total, and that total now carries the corrected current-mode background power, which is four times the figure NVMain printed	Section 3.1.2, Table 3; Appendix A, "DDR5 Timing and Power Corrections of This Revision"

Claim	3 September value	This version	Cause	Evidence
DDR5 background power	Understated: NVMain's current-mode energy model divides each rank's background power by the rank's device count and never multiplies it back, so the 3 September DDR5 figures carried one device's standby draw against the whole rank's activate, burst and refresh power	Each rank's background power multiplied by its device count, read from the run's own configuration and cross-checked against NVMain's printed device count and its background-energy counter: DDR5 static 0.724 W and module total 0.797 W under GCC	An upstream NVMain 2.0 defect of 2014, affecting the current-mode model only, which is the model only the DDR5 configurations use. Corrected in post-processing; the simulator's source and its printed statistics are left untouched. The uncorrected figure sat below the floor the configuration's own EIDD2P0 current allows (0.412 W for eight devices). ReRAM and PCM use other energy models and did not move	Appendix A, "DDR5 Timing and Power Corrections of This Revision"; results/ rev2026-09_primary_csv/ processed_bar_chart_metrics.csv, column Background_Power_Device_Factor
DDR5 write-path, power-down and activate-window timings	DDR3-1333 template cycle counts running at 2400 MHz: tCWD 7, tWR 10, tRTP 5, tWTR 5, tPD 6, tXP 6, tRRD 5, tFAW 20	JESD79-5 DDR5-4800: tCWD 38 (CWL = CL - 2), tWR 72 (30 ns), tRTP 18, tWTR 6 (tWTR_S), tPD and tXP 18, tRRD 8 (tRRD_S), tFAW 32. DDR5's average total latency rose by 6.4% to 47.5% across the six traces, most on the write-dominated burst	The audit of Section 3.1.6 item 12 sourced tCAS, tRCD and tRP but left the write path, the power-down path and the activate window at their template values, and said so without saying they were unsourced. Three short-versus-long choices (tCCD, tWTR, tRRD) are DDR5-friendly judgements, not transcriptions	Section 2.3; Appendix A, "DDR5 Timing and Power Corrections of This Revision"; configs/ DDR5_4800_DRAM_subchannel.config
Endurance rating basis	SLC 10^7 and MLC 10^6 cycles, cited to Wong et al. [14]	10^6 primary, cited to Chen [47]; 10^4 and 10^7 carried as bounds	[14] gives no SLC or MLC rating at all; the citation has been withdrawn for this purpose	research_notes/ endurance_deep_dive.md section 3.5
LBM write rate	39.1 M writes/s (3,257,597 writes per 83.33 ms)	9.54 M writes/s sustained (2,384,804 writes per 250 ms)	The old window sat inside LBM's start-up burst on a fourfold-duplicated trace	Section 3.1.4; benchmarks/ lbm_spec2017.nvt.sidecar.json
Endurance projection	1.09 years at 8 GB and 8.7 years at 64 GB under LBM, assuming ideal uniform leveling, hot-spot exposure "bounded but not simulated"	At 64 GiB and 10^6: 3.6 years ideal, 0.20 years randomized Start-Gap, 23.2 hours with no leveling or with single-region Start-Gap	Wear is now counted per location in the simulator, and three leveling policies are projected from that distribution instead of one being assumed	Section 3.1.4, Table 5; results/ endurance_table.csv
"The selector lives longer"	1S1R SLC 24.8 yr against 1T1R SLC 17.4 yr at 128 GB	Withdrawn. All four ReRAM tracks wear identically (7.1 yr at 128 GiB under ideal leveling)	The apparent difference was the slower configuration completing fewer writes inside a service-limited window	Section 3.1.4; Table 7 note
Wear measurement	No per-location data existed; the simulator's worst-case	Per-location write counters, DIMM-wide aggregation and a	The simulator had no usable wear output at all, so	research_notes/ revision_plan.md, NVMain prototype

Claim	3 September value	This version	Cause	Evidence
	endurance statistic printed an empty-map sentinel	faithful Start-Gap decoder, all added to NVMain for this revision	uniformity had to be assumed	findings B; results/ wear_1T1R_SLC files
Die area per 1 Gb chip	1T1R 19.802 mm², 1S1R 2.276 mm²: an 8.7x selector advantage	12.008 and 3.540 mm²: a 3.4x advantage	Die area is organization-dependent; the same 128x mat gap that produced the leakage figure above produced this one	Table 1; results/ archive_2026-09_pre_revision/ hardware_metrics.json
Start-Gap on versus off	Not run; the decoder did not exist	Wear statistics and latency identical with the decoder on and off on GCC, LBM and the AlexNet output map, so no before-and-after hot-spot figure is shown	A 250 ms window completes about 3 x 10^{-5} (GCC) of one rotation over the 67,108,864-line Start-Gap region, so leveling has not yet acted; its effect is carried by the projection instead	Section 3.1.4; results/ system_rev2026-09_decoder_default
Gap-move traffic	Not applicable; no wear-leveling decoder was modeled	Charged as wear in the Table 5 projection but *not* issued as traffic in the simulation (one line move per 100 writes, about 1% write overhead)	The two accountings deliberately differ, so no latency or power figure in this book carries the cost of running Start-Gap	Section 3.1.4; research_notes/ revision_plan.md decision 22
Dead configuration lines	DECODER MigratingDecoder (wrong key and unknown value), STATS_OUT (ignored by traceSim), EnduranceDist missing (segfault when the model was set without it)	Decoder, StatsFile, EnduranceModel with EnduranceDist: all live keys, verified against the simulator's own parameter readers	Keys written by the generator had never been checked against the keys NVMain reads	research_notes/ revision_plan.md decision 25; Section 3.1.6 item 10
Chip-count scaling verdict	gcc improves ~14% (152.58 to 130.91 ns, 1T1R SLC) as chip count scales while GPT-2 IFMAP is exactly flat at 458.41 ns; scaling presented as a real latency lever for footprint-spanning workloads	gcc moves 44.7 to 41.4 ns between the one-chip and full-DIMM points (7.4%), with a spread of 11.6% across all four points on LBM and no monotonic trend; GPT-2 IFMAP moves 223.1 to 143.3 ns, all of it at the second channel; rank depth is worth at most 7.5%, on the three CPU traces only, and exactly nothing on the three AI traces	The pre-revision matrix ran on the old traces, the double-counted read path and a decoded address space 64x the physical module. On the corrected matrix the three CPU traces are offered-rate limited (delivered bandwidth is identical at all four architectures) and the AI bursts are queue-limited, so within this chip-count matrix the only scaling dimension that moves latency is channel count, and only for queue-bound traffic. The interface clock and the controller queue depth are	Section 3.2, Table 8; Appendix A "Module Capacity and the Address Footprint"; results/ rev2026-09_primary_csv/ processed_bar_chart_metrics.csv; results/ system_rev2026-09_channels1_ai/

Claim	3 September value	This version	Cause	Evidence
			separate levers, quantified in Section 2.3 and Section 3.1.1. A one-channel control run on the two AI input maps measures the channel attribution directly	
Two-leakage-tier scaling picture	Section 3.2 and Figures 20-25 plotted the transistor-gated family scaling "linearly into infeasibility" at 51.0 W while the selector-gated family held a 1.25-1.31 W envelope	Both families sit on the same static floor at every chip count: 0.108384, 0.867072, 1.734144 and 6.936576 W at 1, 8, 16 and 64 chips, identical for all four ReRAM tracks	The same organization artifact as the per-chip leakage row above, propagated into every scaling trajectory. Static power is exactly chip count times 108.384 mW	Section 3.2; Table 8 note; processed_bar_chart_metrics.csv column Static_Power
Per-rank address span	64 KiB per rank, reasoned over a decoded address space of 512 GB; "GPT-2 IFMAP's footprint lands 6 bytes short of the boundary"	64 KiB per rank *per channel*, so a two-channel module rolls into the next rank after 128 KiB; GPT-2 IFMAP (64.0 KiB) and AlexNet IFMAP (68.4 KiB) both sit inside one rank span, AlexNet OFMAP (132.2 KiB) crosses one boundary	The mechanism was right and its arithmetic was not: the capacity was wrong and the channel field, which sits below the rank field, was not accounted for. Confirmed directly from NVMain's per-rank counters rather than inferred	Appendix A "Module Capacity and the Address Footprint"; results/ system_rev2026-09_primary/ stats_reram_22nm_1t1r_slc_full_dimm_.out
Queue-depth sensitivity	A 16/32/64 sweep on the pre-revision traces, quoted in Appendix A as 468.1 / 670.6 / 858.6 ns with completion figures from a trace that offered roughly 4x LBM's real request rate	An 8/32/128 sweep on the regenerated traces: no effect at all on gcc or lbm at any depth, and on the AlexNet output burst the in-controller average rises 124.9 to 1,049.2 ns while the end-to-end average falls 680.1 to 432.5 us. Completion is identical at every depth	The old sweep measured a saturated trace. On the corrected traces the CPU workloads never fill the queue, and on the write burst the two latency statistics move in opposite directions, which is the case for reporting end-to-end latency	Section 3.1.1; Appendix A "Memory Controller Queue Depth"; results/ system_rev2026-09_queue8 and _queue128
Interface-clock sensitivity	Declared as an open sensitivity axis; no measured 1333 or 2400 MHz result existed	Measured: 1T1R SLC gcc 41.4 / 27.9 / 19.0 ns at 800 / 1333 / 2400 MHz, 1S1R SLC 36.8 / 23.6 / 14.9; on the AlexNet output burst 353.1 / 263.1 / 200.4 ns and 486.1 / 366.5 / 283.3 us end to end	New measurement rather than a correction. The interface is worth 2.2x on compute-bound traffic and 1.7x where the queue binds, and does not enter the leakage figure at all	Section 2.3; Section 4.2; results/ system_rev2026-09_freq1333 and _freq2400

Claim	3 September value	This version	Cause	Evidence
DDR5 64 B cross-check	Not run; the access-granularity effect and the module-shape effect were not separated	Nearly equal on the CPU traces (79.6 vs 83.1 ns under gcc, 75.9 vs 78.3 under lbm) but 3.4x faster in the controller and 3.3x faster end to end on the dense AI write burst (69.2 ns / 63.6 us against 234.3 ns / 209.9 us)	New measurement. The two-subchannel model stays primary because it is the organization JESD79-5 specifies. The cross-check is a legacy-shape reference rather than a JEDEC DDR5 configuration - its tCCD of 4 cycles is below the JEDEC floor of 8, kept so the 64-bit, 8-beat channel shape it exists to reproduce survives - so its speed advantage is optimistic and every DDR5 figure in the dense-burst regime is a conservative baseline	Section 2.3; results/ system_rev2026-09_ddr5_64b
Book figures	All 27 figure files rendered the pre-revision dataset; 15 caption blocks carried interim warnings and the List of Figures carried a dataset note	Figures 1-18 and 20-27 regenerated from the primary dataset and each checked against the table beside it; the six Pareto figures plot all four architectures of Table 8; Figure 19 retained with a caption naming its dataset, because the ReadVoltage sweep behind it was not re-run	Figure regeneration was deferred to part B of the book revision while the tables were corrected first. A second pass then restored the 8-chip point to the Pareto figures, whose generator had excluded it under a comment that was true only of the pre-revision dataset, and removed the em-dashes baked into the figure titles	results/ book_figures_rev2026-09/; Review_Fixes_Tracker.md 2026-09-12 entry for the figure-to-media mapping
Where the cell write time is charged	The 3 September version made no statement at all about the write-path mapping. *Stated earlier in this revision, and now corrected:* the cell write pulse was described as being off the request path entirely, with NVSim's write latency said to block the bank after the burst rather than delay the request	Booked as tWR, the cell write time reaches the request path *only when a write closes its row*: a write-with-precharge adds tWR to that subarray's next activation and maxes its next precharge, next read and next write against it, while a write followed by another queued request to the same open row charges nothing beyond the burst and tWTR. Neither branch delays the write's own completion, and the unit that waits is the	The earlier wording was read off NVMain's tWP default alone. The source was then traced in full (MemoryController::IsLastRequest under ClosePage, the SubArray WRITE_PRECHARGE and WRITE branches, and nextActivate / nextPrecharge / nextRead / nextWrite as SubArray members): the answer is branch-conditional, and two earlier attempts at this correction were themselves wrong - the first because it said the write time delays only the next precharge, the second because it made the	Section 2.3; Section 3.1.1; Section 3.1.3; configs/CLAUDE.md, "What the two write modes actually differ in, and for which writes"

Claim	3 September value	This version	Cause	Evidence
		write's own subarray, not the whole bank	completion effect unconditional	
The write-path cost: a single value becomes a bracket	Not run. *Stated earlier in this revision:* the write-burst deficit as single values - DDR5 1.507x faster than 1T1R SLC, 1.671x than 1S1R SLC, 2.185x and 2.787x against the two MLC tracks - with the ReRAM side declared a lower bound and no upper end measured	Each is now a bracket between the two placements of the same cell write number: 1.507x to 2.973x (1T1R SLC), 1.671x to 3.912x (1S1R SLC), 2.185x to 7.042x (1T1R MLC), 2.787x to 10.665x (1S1R MLC). MLC against SLC on GCC is 1.076 to 1.362 (1T1R) and 1.030 to 1.528 (1S1R). "Below DDR5 on the CPU traces" survives at the upper end for all four projected tracks, worst case 1S1R MLC under GCC at 0.871x of DDR5; Wear_Max_Writes is identical in both modes for all four	New measurement, not a correction. The whole primary matrix was re-run with `--write-pulse-on-path`, which books the cell write time on the write's own completion instead of as write recovery. On the three CPU traces the rise equals the write fraction times the pulse to within 1%, because those traces have almost no row-buffer hits; on the AlexNet output burst it is 21x to 23x that term, because 76% of its writes find an open row - the branch the default mode left uncharged. The bracket is an upper end of the write-path cost in this model, not a prediction	Section 3.1.1, "The write pulse on the request path: a bounding run", Table 9; Appendix A, "The Write-Pulse Placement Flag"; results/ system_rev2026-09_writepulse/ _csv/ processed_bar_chart_metrics.csv